

No.	Questions
82	How to create react class components without ES6?
83	Is it possible to use react without JSX?
84	How do you create HOC using <code>render props</code>?
85	What is react scripts?
86	What are the features of create react app?
87	What is the purpose of <code>renderToNodeStream</code> method?
88	How do you get redux scaffolding using <code>create-react-app</code>?
89	What is state mutation and how to prevent it?

Core React

1. What is React?

React (aka React.js or ReactJS) is an **open-source front-end JavaScript library** for building user interfaces based on components. It's used for handling the view layer in web and mobile applications, and allows developers to create reusable UI components and manage the state of those components efficiently.

React was created by [Jordan Walke](#), a software engineer at Facebook (now Meta). It was first deployed on Facebook's News Feed in 2011 and on Instagram in 2012. The library was open-sourced in May 2013 and has since become one of the most popular JavaScript libraries for building modern user interfaces.

[↑ Back to Top](#)

2. What is the history behind React evolution?

The history of ReactJS started in 2010 with the creation of **XHP**. XHP is a PHP extension which improved the syntax of the language such that XML document fragments become valid PHP expressions and the primary purpose was used to create custom and reusable HTML elements.

The main principle of this extension was to make front-end code easier to understand and to help avoid cross-site scripting attacks. The project was successful to prevent the malicious content submitted by the scrubbing user.

But there was a different problem with XHP in which dynamic web applications require many roundtrips to the server, and XHP did not solve this problem. Also, the whole UI was re-rendered for small change in the application. Later, the initial prototype of React is created with the name **FaxJ** by Jordan inspired from XHP. Finally after sometime React has been introduced as a new library into JavaScript world.

► See deep-dive answer

Note: JSX, React's syntax extension, was indeed inspired by XHP's approach of embedding XML-like syntax in code.

[↑ Back to Top](#)

3. What are the major features of React?

React offers a powerful set of features that have made it one of the most popular JavaScript libraries for building user interfaces:

Core Features:

- **Component-Based Architecture:** React applications are built using components - independent, reusable pieces of code that return HTML via a render function. This modular approach enables better code organization, reusability, and maintenance.
- **Virtual DOM:** React creates an in-memory data structure cache, computes the resulting differences, and efficiently updates only the changed parts in the browser DOM. This approach significantly improves performance compared to direct DOM manipulation.
- **JSX (JavaScript XML):** A syntax extension that allows writing HTML-like code in JavaScript. JSX makes the code more readable and expressive while providing the full power of JavaScript.
- **Unidirectional Data Flow:** React follows a one-way data binding model where data flows from parent to child components. This makes the code more predictable and easier to debug.
- **Declarative UI:** React allows you to describe what your UI should look like for a given state, and it handles the DOM updates when the underlying data changes.

Advanced Features:

- **React Hooks:** Introduced in React 16.8, hooks allow using state and other React features in functional components without writing classes.
- **Context API:** Provides a way to share values between components without explicitly passing props through every level of the component tree.
- **Error Boundaries:** Components that catch JavaScript errors anywhere in their child component tree and display fallback UI instead of crashing.
- **Server-Side Rendering (SSR):** Enables rendering React components on the server before sending HTML to the client, improving performance and SEO.
- **Concurrent Mode:** A set of new features (in development) that help React apps stay responsive and gracefully adjust to the user's device capabilities and network speed.
- **React Server Components:** A new feature that allows components to be rendered entirely on the server, reducing bundle size and improving performance.
- **Suspense:** A feature that lets your components "wait" for something before rendering, supporting code-splitting and data fetching with cleaner code.

These features collectively make React powerful for building everything from small widgets to complex, large-scale web applications.

[↑ Back to Top](#)

4. What is JSX?

JSX stands for *JavaScript XML* and it is an XML-like syntax extension to ECMAScript. Basically it just provides the syntactic sugar for the `React.createElement(type, props, ...children)` function, giving us expressiveness of JavaScript along with HTML like template syntax.

In the example below, the text inside `<h1>` tag is returned as JavaScript function to the render function.

```
export default function App() {  
  return <h1 className="greeting">{"Hello, this is a JSX Code!"}</h1>;  
}
```

If you don't use JSX syntax then the respective JavaScript code should be written as below,

```
import { createElement } from "react";  
  
export default function App() {  
  return createElement(  
    "h1",  
    { className: "greeting" },  
    "Hello, this is a JSX Code!"  
  );  
}
```

► See Class

Note: JSX is stricter than HTML

[↑ Back to Top](#)

5. What is the difference between an Element and a Component?

Element:

- A React **Element** is a plain JavaScript object that describes what you want to see on the UI. It represents a DOM node or a component at a specific point in time.
- Elements are immutable: once created, you cannot change their properties. Instead, you create new elements to reflect updates.
- Elements can be nested within other elements through their `props`.
- Creating an element is a fast, lightweight operation—it does **not** create any actual DOM nodes or render anything to the screen directly.

Example (without JSX):

```
const element = React.createElement("button", { id: "login-btn" }, "Login");
```

Equivalent JSX syntax:

```
<button id="login-btn">Login</button>
```

The object returned by `React.createElement` :

```
{
  type: 'button',
  props: {
    id: 'login-btn',
    children: 'Login'
  }
}
```

Elements are then passed to the React DOM renderer (e.g., `ReactDOM.render()`), which translates them to actual DOM nodes.

Component:

- A **Component** is a function or class that returns an element (or a tree of elements) to describe part of the UI. Components can accept inputs (called **props**) and manage their own state (in case of class or function components with hooks).
- Components allow you to split the UI into independent, reusable pieces, each isolated and composable.
- You can define a component using a function or a class:

Example (Function Component with JSX):

```
const Button = ({ handleLogin }) => (
  <button id="login-btn" onClick={handleLogin}>
    Login
  </button>
);
```

When JSX is compiled, it's transformed into a tree of `React.createElement` calls:

```
const Button = ({ handleLogin }) =>
  React.createElement(
    "button",
    { id: "login-btn", onClick: handleLogin },
    "Login"
  );
```

In summary:

- **Elements** are the smallest building blocks in React—objects that describe what you want to see.
- **Components** are functions or classes that return elements and encapsulate logic, structure, and behavior for parts of your UI. Think of **elements** as the instructions for creating UI, and **components** as reusable blueprints that combine logic and structure to generate those instructions.

[↑ Back to Top](#)

6. How to create components in React?

Components are the building blocks of creating User Interfaces(UI) in React. There are two possible ways to create a component.

- Function Components:** This is the simplest way to create a component. Those are pure JavaScript functions that accept props object as the one and only one parameter and return React elements to render the output:

```
function Greeting({ message }) {
  return <h1>{`Hello, ${message}`}</h1>;
}
```

- Class Components:** You can also use ES6 class to define a component. The above function component can be written as a class component:

```
class Greeting extends React.Component {
  render() {
    return <h1>{`Hello, ${this.props.message}`}</h1>;
  }
}
```

[↑ Back to Top](#)

7. When to use a Class Component over a Function Component?

After the addition of Hooks(i.e. React 16.8 onwards) it is always recommended to use Function components over Class components in React. Because you could use state, lifecycle methods and other features that were only available in class component present in function component too.

But even there are two reasons to use Class components over Function components.

- i. If you need a React functionality whose Function component equivalent is not present yet, like Error Boundaries.
- ii. In older versions, If the component needs *state or lifecycle methods* then you need to use class component.

So the summary to this question is as follows:

Use Function Components:

- If you don't need state or lifecycle methods, and your component is purely presentational.
- For simplicity, readability, and modern code practices, especially with the use of React Hooks for state and side effects.

Use Class Components:

- If you need to manage state or use lifecycle methods.
- In scenarios where backward compatibility or integration with older code is necessary.

Note: You can also use reusable [react error boundary](#) third-party component without writing any class. i.e, No need to use class components for Error boundaries.

The usage of Error boundaries from the above library is quite straight forward.

Note when using react-error-boundary: ErrorBoundary is a client component. You can only pass props to it that are serializable or use it in files that have a `"use client";` directive.

```
"use client";

import { ErrorBoundary } from "react-error-boundary";

<ErrorBoundary fallback={<div>Something went wrong</div>}>
  <ExampleApplication />
</ErrorBoundary>;
```

[↑ Back to Top](#)

8. What are Pure Components?

Pure components are the components which render the same output for the same state and props. In function components, you can achieve these pure components through memoized `React.memo()` API wrapping around the component. This API prevents unnecessary re-renders by comparing the previous props and new props using shallow comparison. So it will be helpful for performance optimizations.

But at the same time, it won't compare the previous state with the current state because function component itself prevents the unnecessary rendering by default when you set the same state again.

The syntactic representation of memoized components looks like below,

```
const MemoizedComponent = memo(SomeComponent, arePropsEqual?);
```

Below is the example of how child component(i.e., EmployeeProfile) prevents re-renders for the same props passed by parent component(i.e.,EmployeeRegForm).

```
import { memo, useState } from "react";

const EmployeeProfile = memo(function EmployeeProfile({ name, email }) {
  return (
    <>
```

```

    <p>Name:{name}</p>
    <p>Email: {email}</p>
  </>
);
});
export default function EmployeeRegForm() {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  return (
    <>
      <label>
        Name:{ " "}
        <input value={name} onChange={(e) => setName(e.target.value)} />
      </label>
      <label>
        Email:{ " "}
        <input value={email} onChange={(e) => setEmail(e.target.value)} />
      </label>
      <hr />
      <EmployeeProfile name={name} />
    </>
  );
}

```

In the above code, the email prop has not been passed to child component. So there won't be any re-renders for email prop change.

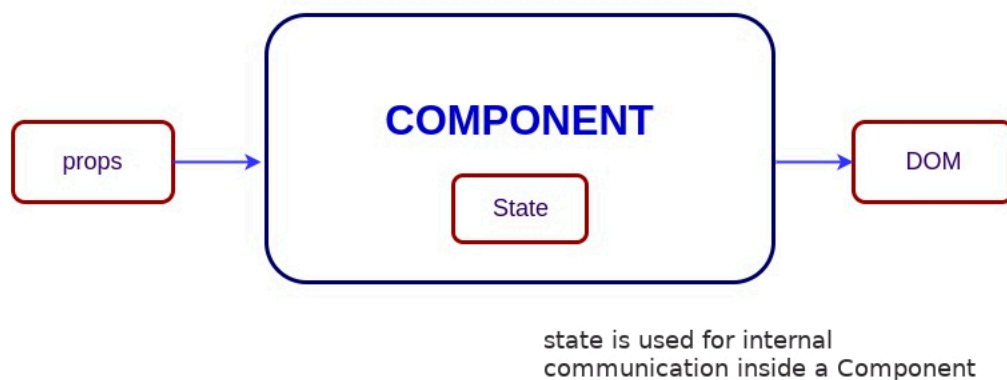
In class components, the components extending `React.PureComponent` instead of `React.Component` become the pure components. When props or state changes, `PureComponent` will do a shallow comparison on both props and state by invoking `shouldComponentUpdate()` lifecycle method.

Note: `React.memo()` is a higher-order component.

[📄 Back to Top](#)

9. What is state in React?

State of a component is an object that holds some information that may change over the lifetime of the component. The important point is whenever the state object changes, the component re-renders. It is always recommended to make our state as simple as possible and minimize the number of stateful components.



Let's take an example of **User** component with `message` state. Here, `useState` hook has been used to add state to the User component and it returns an array with current state and function to update it.

```

import { useState } from "react";

function User() {
  const [message, setMessage] = useState("Welcome to React world");

  return (
    <div>
      <h1>{message}</h1>
    </div>
  );
}

```

```
);
}
```

Whenever React calls your component or access `useState` hook, it gives you a snapshot of the state for that particular render.

► See Class

State is similar to props, but it is private and fully controlled by the component ,i.e., it is not accessible to any other component till the owner component decides to pass it.

[↑ Back to Top](#)

10. What are props in React?

Props are inputs to components. They are single values or objects containing a set of values that are passed to components on creation similar to HTML-tag attributes. Here, the data is passed down from a parent component to a child component.

The primary purpose of props in React is to provide following component functionality:

- i. Pass custom data to your component.
- ii. Trigger state changes.
- iii. Use via `this.props.reactProp` inside component's `render()` method.

For example, let us create an element with `reactProp` property:

```
<Element reactProp={"1"} />
```

This `reactProp` (or whatever you came up with) attribute name then becomes a property attached to React's native props object which originally already exists on all components created using React library.

```
props.reactProp;
```

For example, the usage of props in function component looks like below:

```
import React from "react";
import ReactDOM from "react-dom";

const ChildComponent = (props) => {
  return (
    <div>
      <p>{props.name}</p>
      <p>{props.age}</p>
      <p>{props.gender}</p>
    </div>
  );
};

const ParentComponent = () => {
  return (
    <div>
      <ChildComponent name="John" age="30" gender="male" />
      <ChildComponent name="Mary" age="25" gender="female" />
    </div>
  );
};
```

The properties from props object can be accessed directly using destructuring feature from ES6 (ECMAScript 2015). It is also possible to fallback to default value when the prop value is not specified. The above child component can be simplified like below.

```
const ChildComponent = ({ name, age, gender = "male" }) => {
  return (
    <div>
      <p>{name}</p>
      <p>{age}</p>
      <p>{gender}</p>
    </div>
  );
};
```

Note: The default value won't be used if you pass `null` or `0` value. i.e, default value is only used if the prop value is missed or `undefined` value has been passed.

► See Class

↑ [Back to Top](#)

11. What is the difference between state and props?

In React, both **state** and **props** are plain JavaScript objects, but they serve different purposes and have distinct behaviors:

State

- **Definition:**
State is a data structure that is managed within a component. It represents information that can change over the lifetime of the component.
- **Mutability:**
State is mutable, meaning it can be changed using the setter function (`setState` in class components or the updater function from `useState` in functional components).
- **Scope:**
State is local to the component where it is defined. Only that component can modify its own state.
- **Usage:**
State is typically used for data that needs to change in response to user actions, network responses, or other dynamic events.
- **Re-rendering:**
Updating the state triggers a re-render of the component and its descendants.

Props

- **Definition:**
Props (short for "properties") are inputs to a component, provided by its parent component.
- **Mutability:**
Props are read-only. A component cannot modify its own props; they are immutable from the component's perspective.
- **Scope:**
Props are used to pass data and event handlers down the component tree, enabling parent components to configure or communicate with their children.
- **Usage:**
Props are commonly used to make components reusable and configurable. They allow the same component to be rendered with different data or behavior.
- **Analogy:**
Think of props as arguments to a function, whereas state is like variables declared inside the function.

Summary Table

Feature	State	Props
Managed by	The component itself	Parent component
Mutable	Yes	No (read-only)
Scope	Local to the component	Passed from parent to child
Usage	Manage dynamic data and UI changes	Configure and customize component
Update	Using <code>setState</code> / <code>useState</code>	Cannot be updated by the component

↑ [Back to Top](#)

12. What is the difference between HTML and React event handling?

Below are some of the main differences between HTML and React event handling,

- In HTML, the event name usually represents in *lowercase* as a convention:

```
<button onclick="activateLasers()"></button>
```



Whereas in React it follows *camelCase* convention:

```
<button onClick={activateLasers}>
```



ii. In HTML, you can return `false` to prevent default behavior:

```
<a
  href="#"
  onclick='console.log("The link was clicked."); return false;'
/>
```



Whereas in React you must call `preventDefault()` explicitly:

```
function handleClick(event) {
  event.preventDefault();
  console.log("The link was clicked.");
}
```



iii. In HTML, you need to invoke the function by appending `()` Whereas in react you should not append `()` with the function name. (refer "activateLasers" function in the first point for example)

[↑ Back to Top](#)

13. What are synthetic events in React?

`SyntheticEvent` is a cross-browser wrapper around the browser's native event. Its API is same as the browser's native event, including `stopPropagation()` and `preventDefault()`, except the events work identically across all browsers. The native events can be accessed directly from synthetic events using `nativeEvent` attribute.

Let's take an example of `BookStore` title search component with the ability to get all native event properties

```
function BookStore() {
  function handleTitleChange(e) {
    console.log("The new title is:", e.target.value);
    console.log('Synthetic event:', e); // React SyntheticEvent
    console.log('Native event:', e.nativeEvent); // Browser native event
    e.stopPropagation();
    e.preventDefault();
  }

  return <input name="title" onChange={handleTitleChange} />;
}
```



List of common synthetic events are:

- `onClick`
- `onChange`
- `onSubmit`
- `onKeyDown`, `onKeyUp`
- `onFocus`, `onBlur`
- `onMouseEnter`, `onMouseLeave`
- `onTouchStart`, `onTouchEnd`

[↑ Back to Top](#)

14. What are inline conditional expressions?

You can use either *if statements* or *ternary expressions* which are available in JS (and JSX in React) to conditionally execute or render expressions. Apart from these approaches, you can also embed any expressions in JSX by wrapping them in curly braces and then followed by JS logical operator `&&`. It is helpful to render elements conditionally within a single line and commonly used for concise logic, especially in JSX rendering.

```
<h1>Hello!</h1>;
{
  messages.length > 0 && !isLogin ? (
    <h2>You have {messages.length} unread messages.</h2>
  ) : (
    <h2>You don't have unread messages.</h2>
  )
}
```




```
);  
}
```

[↑ Back to Top](#)

15. What is "key" prop and what is the benefit of using it in arrays of elements?

A `key` is a special attribute you **should** include when mapping over arrays to render data. Key prop helps React identify which items have changed, are added, or are removed.

Keys should be unique among its siblings. Most often we use ID from our data as *key*:

```
const todoItems = todos.map((todo) => <li key={todo.id}>{todo.text}</li>);
```



When you don't have stable IDs for rendered items, you may use the item *index* as a *key* as a last resort:

```
const todoItems = todos.map((todo, index) => (  
  <li key={index}>{todo.text}</li>  
));
```



Benefits of key:

- Enables React to **efficiently update and re-render** components.
- Prevents unnecessary re-renders by **reusing** components when possible.
- Helps **maintain internal state** of list items correctly.

Note:

- Using *indexes* for *keys* is **not recommended** if the order of items may change. This can negatively impact performance and may cause issues with component state.
- If you extract list item as separate component then apply *keys* on list component instead of `li` tag.
- There will be a warning message in the console if the `key` prop is not present on list items.
- The key attribute accepts either string or number and internally convert it as string type.
- Don't generate the key on the fly something like `key={Math.random()} . Because the keys will never match up between re-renders and DOM created everytime.`

[↑ Back to Top](#)

16. What is Virtual DOM?

The *Virtual DOM* (VDOM) is a lightweight, in-memory representation of *Real DOM* used by libraries like React to optimize UI rendering. The representation of a UI is kept in memory and synced with the "real" DOM. It's a step that happens between the render function being called and the displaying of elements on the screen. This entire process is called *reconciliation*.

[↑ Back to Top](#)

17. How Virtual DOM works?

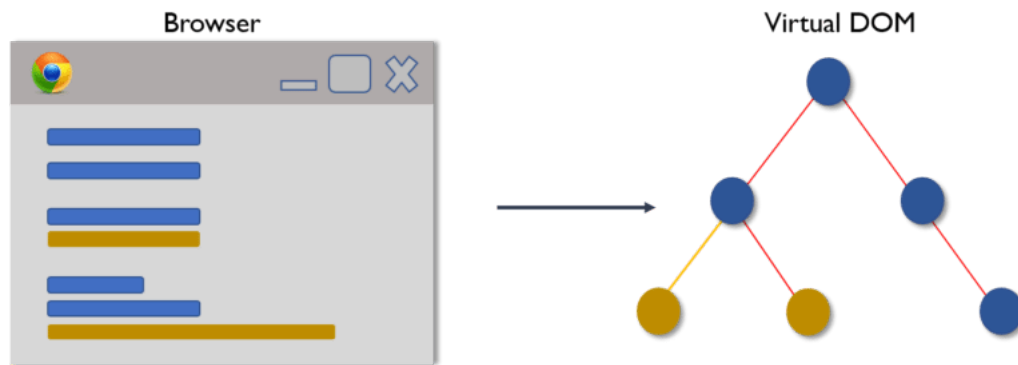
The *Virtual DOM* works in five simple steps.

1. Initial Render

When a UI component renders for the first time, it returns JSX. React uses this structure to create a Virtual DOM tree, which is a lightweight copy of the actual DOM. This Virtual DOM is then used to build and render the Real DOM in the browser.

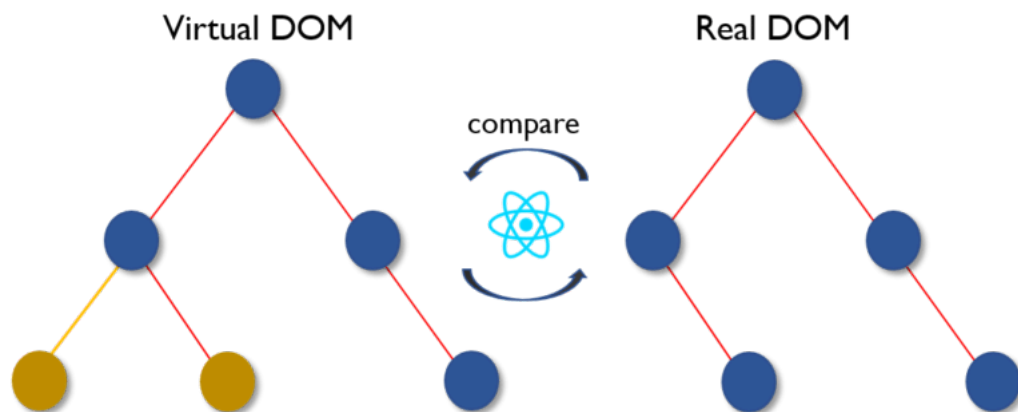
2. State or Props Change

When the component's state or props change, React creates a new Virtual DOM reflecting the updated UI. However, it doesn't immediately update the Real DOM; instead, it works in memory to prepare for an efficient update.



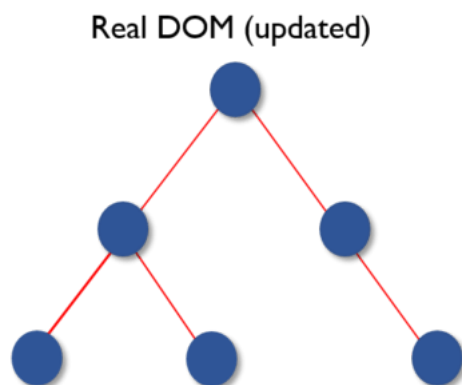
3. Diffing Algorithm

React then compares the new Virtual DOM with the previous one using a process called diffing. It determines what has changed between the two versions and identifies the minimal set of updates needed.



4. Reconciliation

Based on the diffing results, React decides which parts of the Real DOM should be updated. It avoids re-rendering the entire DOM and instead updates only the elements that actually changed.



5. Efficient DOM Updates

This entire process—working with the Virtual DOM, diffing, and selective updating—makes the UI rendering much faster and more efficient than manipulating the Real DOM directly.

[↑ Back to Top](#)

18. What is the difference between Shadow DOM and Virtual DOM?

The *Shadow DOM* is a browser technology designed primarily for scoping variables and CSS in *web components*. The *Virtual DOM* is a concept implemented by libraries in JavaScript on top of browser APIs.

The key differences in a table format shown below:

Feature	Shadow DOM	Virtual DOM
Purpose	Encapsulation for Web Components	Efficient UI rendering
Managed by	Browser	JS frameworks (e.g., React)

Feature	Shadow DOM	Virtual DOM
DOM Type	Part of real DOM (scoped)	In-memory representation
Encapsulation	Yes	No
Use Case	Web Components, scoped styling	UI diffing and minimal DOM updates

[↑ Back to Top](#)

19. What is React Fiber?

React Fiber is the **new reconciliation engine** in React, introduced in React 16. It's a complete rewrite of React's core algorithm(old stack-based algorithm) for rendering and updating the UI. Fiber enhances React's ability to handle **asynchronous rendering**, **prioritized updates**(assign priority to different types of updates), and **interruption**(ability to pause, abort, or reuse work) of rendering work, enabling smoother and more responsive user interfaces.

[↑ Back to Top](#)

20. What is the main goal of React Fiber?

The goal of *React Fiber* is to increase its suitability for areas like animation, layout, and gestures. Its headline feature is **incremental rendering**: the ability to split rendering work into chunks and spread it out over multiple frames.

Its main goals are:

- **Incremental Rendering** – Breaks work into chunks for smoother updates.
- **Interruptible Rendering** – Pauses and resumes rendering to keep the UI responsive.
- **Prioritization** – Handles high-priority updates (e.g. animations) before low-priority ones.
- **Concurrency Support** – Enables working on multiple UI versions simultaneously.
- **Better Error Handling** – Supports component-level error boundaries.
- **Suspense Support** – Allows waiting for async data before rendering.
- **Improved DevTools** – Enables better debugging and performance tracking.

[↑ Back to Top](#)

21. What are controlled components?

A **controlled component** is a React component that **fully manages the form element's state**(e.g, elements like `<input>`, `<textarea>`, or `<select>`)) using React's internal state mechanism. i.e, The component does not manage its own internal state — instead, React acts as the single source of truth for form data.

The controlled components will be implemented using the below steps,

- Initialize the state using `useState` hooks in function components or inside constructor for class components.
- Set the value of the form element to the respective state variable.
- Create an event handler(`onChange`) to handle the user input changes through `useState`'s updater function or `setState` from class component.
- Attach the above event handler to form element's change or click events

Note: React re-renders the component every time the input value changes.

For example, the name input field updates the username using `handleChange` event handler as below,

```
import React, { useState } from "react";

function UserProfile() {
  const [username, setUsername] = useState("");

  const handleChange = (e) => {
    setUsername(e.target.value);
  };

  return (
    <form>
      <label>
        Name:
        <input type="text" value={username} onChange={handleChange} />
      </label>
    </form>
  );
}
```

```
);
}
```

In these components, DOM does not hold the actual data instead React does.

Benefits:

- Easy to implement **validation**, **conditional formatting**, or **live feedback**.
- Full control over form data.
- Easier to test and debug because the data is centralized in the component's state.

[↑ Back to Top](#)

22. What are uncontrolled components?

The **Uncontrolled components** are form elements (like `<input>`, `<textarea>`, or `<select>`) that **manage their own state internally** via the **DOM**, rather than through React state. You can query the DOM using a `ref` to find its current value when you need it. This is a bit more like traditional HTML.

The uncontrolled components will be implemented using the below steps,

- Create a ref using `useRef` react hook in function component or `React.createRef()` in class based component.
- Attach this `ref` to the form element.
- The form element value can be accessed directly through `ref` in event handlers or `componentDidMount` for class components

In the below UserProfile component, the `username` input is accessed using ref.

```
import React, { useRef } from "react";

function UserProfile() {
  const usernameRef = useRef(null);

  const handleSubmit = (event) => {
    event.preventDefault();
    console.log("The submitted username is: " + usernameRef.current.value);
  };

  return (
    <form onSubmit={handleSubmit}>
      <label>
        Username:
        <input type="text" ref={usernameRef} />
      </label>
      <button type="submit">Submit</button>
    </form>
  );
}
```

Note: Here, DOM is in charge of the value. React only accesses the value when needed (via `ref`).

Benefits:

- **Less boilerplate** — no need for `useState` and `onChange`.
- Useful for **quick form setups** or when integrating with **non-React code**.
- Slightly better **performance** in very large forms (fewer re-renders).

In most cases, it's recommend to use controlled components to implement forms. In a controlled component, form data is handled by a React component. The alternative is uncontrolled components, where form data is handled by the DOM itself.

► See Class

[↑ Back to Top](#)

23. What is the difference between createElement and cloneElement?

Both `React.createElement` and `React.cloneElement` are used to work with React elements, but they serve different purposes.

createElement:

Creates a new React element from scratch. JSX elements will be transpiled to `React.createElement()` functions to create React elements which are going to be used for the object representation of UI. **Syntax:**

```
React.createElement(type, props, ...children)
```



Example:

```
React.createElement('button', { className: 'btn' }, 'Click Me')
```



cloneElement:

The `cloneElement` method is used to clone an existing React element and optionally adds or overrides props.

Syntax:

```
React.cloneElement(element, newProps, ...children)
```



Example:

```
const button = <button className="btn">Click Me</button>;
const cloned = React.cloneElement(button, { className: 'btn-primary' });
// Result: <button className="btn-primary">Click Me</button>
```



[↑ Back to Top](#)

24. What is Lifting State Up in React?

When several components need to share the same changing data then it is recommended to *lift the shared state up* to their closest common ancestor. That means if two child components share the same data from its parent, then move the state to parent instead of maintaining local state in both of the child components.

[↑ Back to Top](#)

25. What are Higher-Order Components?

A *higher-order component (HOC)* is a function that takes a component and returns a new enhanced component with additional props, behavior, or data. It's a design pattern based on React's compositional nature, allowing you to reuse logic across multiple components without modifying their internals.

We consider HOCs **pure components** because they don't mutate or copy behavior from the original component—they simply **wrap it**, enhance it, and pass through the necessary props. The wrapped component remains decoupled and reusable.

```
const EnhancedComponent = higherOrderComponent(WrappedComponent);
```



Let's take an example of a `withAuth` higher-order component (HOC) in React. This HOC will check if a user is authenticated and either render the wrapped component if authenticated or redirect (or show a message) if not.

withAuth HOC Example:

```
import React from 'react';
import { Navigate } from 'react-router-dom'; // For redirection (assuming React Router v6)

const isAuthenticated = () => {
  // e.g., check for a valid token in localStorage or context
  return !!localStorage.getItem('authToken');
};

function withAuth(WrappedComponent) {
  return function AuthenticatedComponent(props) {
    if (!isAuthenticated()) {
      // User is NOT authenticated, redirect to login page
      return <Navigate to="/login" replace />;
    }

    // User is authenticated, render the wrapped component
    return <WrappedComponent {...props} />;
  };
}

export default withAuth;
```



Usage

```
import React from 'react';
import withAuth from './withAuth';

function Dashboard() {
  return <h1>Welcome to the Dashboard!</h1>;
}

// Wrap Dashboard with withAuth HOC
export default withAuth(Dashboard);
```

HOC can be used for many use cases:

- i. Code reuse, logic and bootstrap abstraction (e.g., fetching data, permissions, theming).
- ii. Render hijacking (e.g., conditional rendering or layout changes).
- iii. State abstraction and manipulation (e.g., handling form logic).
- iv. Props manipulation (e.g., injecting additional props or filtering them).

Some of the real-world examples of HOCs in react eco-system:

- i. connect() from react-redux
- ii. withRouter() from React Router v5
- iii. withTranslation() from react-i18next
- iv. withApollo() from Apollo client
- v. withFormik from Formik library
- vi. withTheme from styled components

[↑ Back to Top](#)

26. What is children prop?

The `children` prop is a special prop in React used to pass elements between the opening and closing tags of a component. It is commonly used in layout and wrapper components.

A simple usage of children prop looks as below,

```
function MyDiv({ children }) {
  return (
    <div>
      {children}
    </div>;
  );
}

export default function Greeting() {
  return (
    <MyDiv>
      <span>{"Hello"}</span>
      <span>{"World"}</span>
    </MyDiv>
  );
}
```

Here, everything inside `<MyDiv>...</MyDiv>` is passed as children to the custom div component.

The children can be text, JSX elements, fragments, arrays and functions (for advance use case like render props).

► See Class

Note: There are several methods available in the legacy React API to work with this prop. These include `React.Children.map`, `React.Children.forEach`, `React.Children.count`, `React.Children.only`, `React.Children.toArray`.

[↑ Back to Top](#)

27. How to write comments in React?

The comments in React/JSX are similar to JavaScript Multiline comments but are wrapped in curly braces.

Single-line comments:

```
<div>
  {/* Single-line comments(In vanilla JavaScript, the single-line comments are represented by double slash(//)) */}
  {`Welcome ${user}, let's play React`}
</div>
```

Multi-line comments:

```
<div>
  {/* Multi-line comments for more than
  one line */}
  {`Welcome ${user}, let's play React`}
</div>
```

You can use `//` and `/* */` in JS logic, hooks, and functions.

[↑ Back to Top](#)

28. What is reconciliation?

Reconciliation is the process through which React updates the Browser DOM and makes React work faster. React use a `diffing` algorithm so that component updates are predictable and faster. React would first calculate the difference between the `real DOM` and the copy of DOM (`Virtual DOM`) when there's an update of components. React stores a copy of Browser DOM which is called `Virtual DOM`. When we make changes or add data, React creates a new Virtual DOM and compares it with the previous one. This comparison is done by `Diffing Algorithm`. Now React compares the Virtual DOM with Real DOM. It finds out the changed nodes and updates only the changed nodes in Real DOM leaving the rest nodes as it is. This process is called *Reconciliation*.

[↑ Back to Top](#)

29. Does the lazy function support named exports?

No, currently `React.lazy` function supports default exports only. If you would like to import modules which are named exports, you can create an intermediate module that reexports it as the default. It also ensures that tree shaking keeps working and don't pull unused components. Let's take a component file which exports multiple named components,

```
// MoreComponents.js
export const SomeComponent = /* ... */;
export const UnusedComponent = /* ... */;
```

and reexport `MoreComponents.js` components in an intermediate file `IntermediateComponent.js`

```
// IntermediateComponent.js
export { SomeComponent as default } from "./MoreComponents.js";
```

Now you can import the module using lazy function as below,

```
import React, { lazy } from "react";
const SomeComponent = lazy(() => import("./IntermediateComponent.js"));
```

[↑ Back to Top](#)

30. Why React uses `className` over `class` attribute?

React uses `className` instead of `class` because of a JavaScript naming conflict with the `class` keyword.

- i. `class` is a reserved keyword in JavaScript In JavaScript, `class` is used to define ES6 classes:

```
class Person {
  constructor(name) {
    this.name = name;
  }
}
```

If you try to use `class` as a variable or property name, it will throw a syntax error. Since JSX is just JavaScript with XML-like syntax, using `class` directly in JSX would break the parser.

- ii. JSX Is JavaScript

When you write JSX like this:

```
<div class="btn">Click</div>
```

It will be compiled to:

```
React.createElement('div', { class: 'btn' }, 'Click');
```

But `class` is invalid in this object literal context (since it clashes with the JS keyword), hence React instead uses `className`.

```
<div className="btn">Click</div>
```

which compiles to:

```
React.createElement('div', { className: 'btn' }, 'Click');
```

React then translates `className` to `class` in the final HTML DOM.

iii. Aligns with DOM APIs In vanilla JavaScript, you interact with element classes using:

```
element.className = 'my-class';
```

React follows this convention, staying consistent with the DOM API's property name rather than HTML's attribute.

[↑ Back to Top](#)

31. What are fragments?

It's a common pattern or practice in React for a component to return multiple elements. *Fragments* let you group a list of children without adding extra nodes to the DOM. You need to use either `<Fragment>` or a shorter syntax having empty tag (`<></>`).

Below is the example of how to use fragment inside *Story* component.

```
function Story({ title, description, date }) {
  return (
    <Fragment>
      <h2>{title}</h2>
      <p>{description}</p>
      <p>{date}</p>
    </Fragment>
  );
}
```

It is also possible to render list of fragments inside a loop with the mandatory `key` attribute supplied.

```
function StoryBook() {
  return stories.map((story) => (
    <Fragment key={story.id}>
      <h2>{story.title}</h2>
      <p>{story.description}</p>
      <p>{story.date}</p>
    </Fragment>
  ));
}
```

Usually, you don't need to use `<Fragment>` until there is a need of `key` attribute. The usage of shorter syntax looks like below.

```
function Story({ title, description, date }) {
  return (
    <>
      <h2>{title}</h2>
      <p>{description}</p>
      <p>{date}</p>
    </>
  );
}
```


[↑ Back to Top](#)

32. Why fragments are better than container divs?

Below are the list of reasons to prefer fragments over container DOM elements,

- Fragments are a bit faster and use less memory by not creating an extra DOM node. This only has a real benefit on very large and deep trees.
- Some CSS mechanisms like *Flexbox* and *CSS Grid* have a special parent-child relationships, and adding divs in the middle makes it hard to keep the desired layout.
- The DOM Inspector is less cluttered.

[↑ Back to Top](#)

33. What are portals in React?

A Portal is a React feature that enables rendering children into a DOM node that exists outside the parent component's DOM hierarchy, while still preserving the React component hierarchy. Portals help avoid CSS stacking issues—for example, elements with `position: fixed` may not behave as expected inside a parent with `transform`. Portals solve this by rendering content (like modals or tooltips) outside such constrained DOM contexts.

```
ReactDOM.createPortal(child, container);
```

- `child` : Any valid React node (e.g., JSX, string, fragment).
- `container` : A real DOM node (e.g., `document.getElementById('modal-root')`).

Even though the content renders elsewhere in the DOM, it still behaves like a normal child in React. It has access to context, state, and event handling.

Example:- Modal:

```
function Modal({ children }) {  
  return ReactDOM.createPortal(  
    <div className="modal">{children}</div>,  
    document.body  
  );  
}
```

The above code will render the modal content into the body element in the HTML, not inside the component's usual location.

[↑ Back to Top](#)

34. What are stateless components?

If the behaviour of a component is independent of its state then it can be a stateless component. You can use either a function or a class for creating stateless components. But unless you need to use a lifecycle hook in your components, you should go for function components. There are a lot of benefits if you decide to use function components here; they are easy to write, understand, and test, a little faster, and you can avoid the `this` keyword altogether.

[↑ Back to Top](#)

35. What are stateful components?

If the behaviour of a component is dependent on the *state* of the component then it can be termed as stateful component. These *stateful components* are either function components with hooks or *class components*.

Let's take an example of function stateful component which update the state based on click event,

```
import React, {useState} from 'react';  
  
const App = (props) => {  
  const [count, setCount] = useState(0);  
  handleIncrement() {  
    setCount(count+1);  
  }  
  
  return (  
    <>  
      <button onClick={handleIncrement}>Increment</button>  
      <span>Counter: {count}</span>  
    </>  
  )  
}
```

```
</>
)
}
```

► See Class

[↑ Back to Top](#)

36. How to apply validation on props in React?

When the application is running in *development mode*, React will automatically check all props that we set on components to make sure they have *correct type*. If the type is incorrect, React will generate warning messages in the console. It's disabled in *production mode* due to performance impact. The mandatory props are defined with `isRequired`.

The set of predefined prop types:

- i. `PropTypes.number`
- ii. `PropTypes.string`
- iii. `PropTypes.array`
- iv. `PropTypes.object`
- v. `PropTypes.func`
- vi. `PropTypes.node`
- vii. `PropTypes.element`
- viii. `PropTypes.bool`
- ix. `PropTypes.symbol`
- x. `PropTypes.any`

We can define `propTypes` for `User` component as below:

```
import React from "react";
import PropTypes from "prop-types";

class User extends React.Component {
  static propTypes = {
    name: PropTypes.string.isRequired,
    age: PropTypes.number.isRequired,
  };

  render() {
    return (
      <>
        <h1>{`Welcome, ${this.props.name}`}</h1>
        <h2>{`Age, ${this.props.age}`}</h2>
      </>
    );
  }
}
```

Note: In React v15.5 *PropTypes* were moved from `React.PropTypes` to `prop-types` library.

The Equivalent Functional Component

```
import React from "react";
import PropTypes from "prop-types";

function User({ name, age }) {
  return (
    <>
      <h1>{`Welcome, ${name}`}</h1>
      <h2>{`Age, ${age}`}</h2>
    </>
  );
}

User.propTypes = {
  name: PropTypes.string.isRequired,
  age: PropTypes.number.isRequired,
};
```

[↑ Back to Top](#)

37. What are the advantages of React?

Below are the list of main advantages of React,

- i. Increases the application's performance with *Virtual DOM*.
- ii. JSX makes code easy to read and write.
- iii. It renders both on client and server side (*SSR*).
- iv. Easy to integrate with frameworks (Angular, Backbone) since it is only a view library.
- v. Easy to write unit and integration tests with tools such as Jest.

[↑ Back to Top](#)

38. What are the limitations of React?

Apart from the advantages, there are few limitations of React too,

- i. React is just a view library, not a full framework.
- ii. There is a learning curve for beginners who are new to web development.
- iii. Integrating React into a traditional MVC framework requires some additional configuration.
- iv. The code complexity increases with inline templating and JSX.
- v. Too many smaller components leading to over engineering or boilerplate.

[↑ Back to Top](#)

39. What are the recommended ways for static type checking?

Normally we use *PropTypes library* (`React.PropTypes` moved to a `prop-types` package since React v15.5) for *type checking* in the React applications. For large code bases, it is recommended to use *static type checkers* such as Flow or TypeScript, that perform type checking at compile time and provide auto-completion features.

[↑ Back to Top](#)

40. What is the use of `react-dom` package?

The `react-dom` package provides *DOM-specific methods* that can be used at the top level of your app. Most of the components are not required to use this module. Some of the methods of this package are:

- i. `render()`
- ii. `hydrate()`
- iii. `unmountComponentAtNode()`
- iv. `findDOMNode()`
- v. `createPortal()`

[↑ Back to Top](#)

41. What is ReactDOMServer?

The `ReactDOMServer` object enables you to render components to static markup (typically used on node server). This object is mainly used for *server-side rendering* (SSR). The following methods can be used in both the server and browser environments:

- i. `renderToString()`
- ii. `renderToStaticMarkup()`

For example, you generally run a Node-based web server like Express, Hapi, or Koa, and you call `renderToString` to render your root component to a string, which you then send as response.

```
// using Express
import { renderToString } from "react-dom/server";
import MyPage from "./MyPage";

app.get("/", (req, res) => {
  res.write(
    "<!DOCTYPE html><html><head><title>My Page</title></head><body>"
  );
  res.write('<div id="content">');
  res.write(renderToString(<MyPage />));
  res.write("</div></body></html>");
});
```

```
res.end();  
});
```

[↑ Back to Top](#)

42. How to use innerHTML in React?

The `dangerouslySetInnerHTML` attribute is React's replacement for using `innerHTML` in the browser DOM. Just like `innerHTML`, it is risky to use this attribute considering cross-site scripting (XSS) attacks. You just need to pass a `__html` object as key and HTML text as value.

In this example `MyComponent` uses `dangerouslySetInnerHTML` attribute for setting HTML markup:

```
function createMarkup() {  
  return { __html: "First &middot; Second" };  
}  
  
function MyComponent() {  
  return <div dangerouslySetInnerHTML={createMarkup()} />;  
}
```

[↑ Back to Top](#)

43. How to use styles in React?

The `style` attribute accepts a JavaScript object with camelCased properties rather than a CSS string. This is consistent with the DOM style JavaScript property, is more efficient, and prevents XSS security holes.

```
const divStyle = {  
  color: "blue",  
  backgroundImage: "url(" + imgUrl + ")",  
};  
  
function HelloWorldComponent() {  
  return <div style={divStyle}>Hello World!</div>;  
}
```

Style keys are camelCased in order to be consistent with accessing the properties on DOM nodes in JavaScript (e.g. `node.style.backgroundImage`).

[↑ Back to Top](#)

44. How events are different in React?

Handling events in React elements has some syntactic differences:

- i. React event handlers are named using camelCase, rather than lowercase.
- ii. With JSX you pass a function as the event handler, rather than a string.

[↑ Back to Top](#)

45. What is the impact of indexes as keys?

Keys should be stable, predictable, and unique so that React can keep track of elements.

In the below code snippet each element's key will be based on ordering, rather than tied to the data that is being represented. This limits the optimizations that React can do and creates confusing bugs in the application.

```
{  
  todos.map((todo, index) => <Todo {...todo} key={index} />);  
}
```

If you use element data for unique key, assuming `todo.id` is unique to this list and stable, React would be able to reorder elements without needing to reevaluate them as much.

```
{  
  todos.map((todo) => <Todo {...todo} key={todo.id} />);  
}
```

Note: If you don't specify `key` prop at all, React will use index as a key's value while iterating over an array of data.

[↑ Back to Top](#)

46. How do you conditionally render components?

In some cases you want to render different components depending on some state. JSX does not render `false` or `undefined`, so you can use conditional *short-circuiting* to render a given part of your component only if a certain condition is true.

```
const MyComponent = ({ name, address }) => (
  <div>
    <h2>{name}</h2>
    {address && <p>{address}</p>}
  </div>
);
```

If you need an `if-else` condition then use *ternary operator*.

```
const MyComponent = ({ name, address }) => (
  <div>
    <h2>{name}</h2>
    {address ? <p>{address}</p> : <p>{"Address is not available"}</p>}
  </div>
);
```

[↑ Back to Top](#)

47. Why we need to be careful when spreading props on DOM elements?

When we *spread props* we run into the risk of adding unknown HTML attributes, which is a bad practice. Instead we can use prop destructuring with `...rest` operator, so it will add only required props.

For example,

```
const ComponentA = () => (
  <ComponentB isDisplay={true} className={"componentStyle"} />
);

const ComponentB = ({ isDisplay, ...domProps }) => (
  <div {...domProps}>{"ComponentB"}</div>
);
```

[↑ Back to Top](#)

48. How do you memoize a component?

There are memoize libraries available which can be used on function components.

For example `moize` library can memoize the component in another component.

```
import moize from "moize";
import Component from "../components/Component"; // this module exports a non-memoized component

const MemoizedFoo = moize.react(Component);

const Consumer = () => {
  <div>
    {"I will memoize the following entry:"}
    <MemoizedFoo />
  </div>
};
```

Update: Since React v16.6.0, we have a `React.memo`. It provides a higher order component which memoizes component unless the props change. To use it, simply wrap the component using `React.memo` before you use it.

```
const MemoComponent = React.memo(function MemoComponent(props) {
  /* render using props */
});
OR;
export default React.memo(MyFunctionComponent);
```

[↑ Back to Top](#)

49. How you implement Server Side Rendering or SSR?

React is already equipped to handle rendering on Node servers. A special version of the DOM renderer is available, which follows the same pattern as on the client side.

```
import ReactDOMServer from "react-dom/server";
import App from "./App";

ReactDOMServer.renderToString(<App />);
```



This method will output the regular HTML as a string, which can be then placed inside a page body as part of the server response. On the client side, React detects the pre-rendered content and seamlessly picks up where it left off.

[↑ Back to Top](#)

50. How to enable production mode in React?

You should use Webpack's `DefinePlugin` method to set `NODE_ENV` to `production`, by which it strip out things like `propType` validation and extra warnings. Apart from this, if you minify the code, for example, Uglify's dead-code elimination to strip out development only code and comments, it will drastically reduce the size of your bundle.

[↑ Back to Top](#)

51. Do Hooks replace render props and higher order components?

Both render props and higher-order components render only a single child but in most of the cases Hooks are a simpler way to serve this by reducing nesting in your tree.

[↑ Back to Top](#)

52. What is a switching component?

A *switching component* is a component that renders one of many components. We need to use object to map prop values to components.

For example, a switching component to display different pages based on `page` prop:

```
import HomePage from "./HomePage";
import AboutPage from "./AboutPage";
import ServicesPage from "./ServicesPage";
import ContactPage from "./ContactPage";

const PAGES = {
  home: HomePage,
  about: AboutPage,
  services: ServicesPage,
  contact: ContactPage,
};

const Page = (props) => {
  const Handler = PAGES[props.page] || ContactPage;

  return <Handler {...props} />;
};

// The keys of the PAGES object can be used in the prop types to catch dev-time errors.
Page.propTypes = {
  page: PropTypes.oneOf(Object.keys(PAGES)).isRequired,
};
```

[↑ Back to Top](#)

53. What are React Mixins?

Mixins are a way to totally separate components to have a common functionality. Mixins **should not be used** and can be replaced with *higher-order components* or *decorators*.

One of the most commonly used mixins is `PureRenderMixin`. You might be using it in some components to prevent unnecessary re-renders when the props and state are shallowly equal to the previous props and state:

```
const PureRenderMixin = require("react-addons-pure-render-mixin");

const Button = React.createClass({
  mixins: [PureRenderMixin],
  // ...
});
```

[↑ Back to Top](#)

54. What are the Pointer Events supported in React?

Pointer Events provide a unified way of handling all input events. In the old days we had a mouse and respective event listeners to handle them but nowadays we have many devices which don't correlate to having a mouse, like phones with touch surface or pens. We need to remember that these events will only work in browsers that support the *Pointer Events* specification.

The following event types are now available in *React DOM*:

- i. `onPointerDown`
- ii. `onPointerMove`
- iii. `onPointerUp`
- iv. `onPointerCancel`
- v. `onGotPointerCapture`
- vi. `onLostPointerCapture`
- vii. `onPointerEnter`
- viii. `onPointerLeave`
- ix. `onPointerOver`
- x. `onPointerOut`

[↑ Back to Top](#)

55. Why should component names start with capital letter?

If you are rendering your component using JSX, the name of that component has to begin with a capital letter otherwise React will throw an error as an unrecognized tag. This convention is because only HTML elements and SVG tags can begin with a lowercase letter.

```
function SomeComponent {
  // Code goes here
}
```



You can define function component whose name starts with lowercase letter, but when it's imported it should have a capital letter. Here lowercase is fine:

```
function myComponent {
  render() {
    return <div />;
  }
}

export default myComponent;
```



While when imported in another file it should start with capital letter:

```
import MyComponent from "./myComponent";
```

[↑ Back to Top](#)

56. Are custom DOM attributes supported in React v16?

Yes. In the past, React used to ignore unknown DOM attributes. If you wrote JSX with an attribute that React doesn't recognize, React would just skip it.

For example, let's take a look at the below attribute:

```
<div mycustomattribute={"something"} />
```



Would render an empty div to the DOM with React v15:

```
<div />
```

In React v16 any unknown attributes will end up in the DOM:

```
<div mycustomattribute="something" />
```

This is useful for supplying browser-specific non-standard attributes, trying new DOM APIs, and integrating with opinionated third-party libraries.

[↑ Back to Top](#)

57. How to loop inside JSX?

You can simply use `Array.prototype.map` with ES6 *arrow function* syntax.

For example, the `items` array of objects is mapped into an array of components:

```
<tbody>
  {items.map((item) => (
    <SomeComponent key={item.id} name={item.name} />
  ))}
</tbody>
```

But you can't iterate using `for` loop:

```
<tbody>
  for (let i = 0; i < items.length; i++) {
    <SomeComponent key={items[i].id} name={items[i].name} />
  }
</tbody>
```

This is because JSX tags are transpiled into *function calls*, and you can't use statements inside expressions. This may change thanks to `do` expressions which are *stage 1 proposal*.

[↑ Back to Top](#)

58. How do you access props in attribute quotes?

React (or JSX) doesn't support variable interpolation inside an attribute value. The below representation won't work:

```

```

But you can put any JS expression inside curly braces as the entire attribute value. So the below expression works:

```
<img className="image" src={"images/" + this.props.image} />
```

Using *template strings* will also work:

```
<img className="image" src={`images/${this.props.image}`} />
```

[↑ Back to Top](#)

59. What is React proptype array with shape?

If you want to pass an array of objects to a component with a particular shape then use `React.PropTypes.shape()` as an argument to `React.PropTypes.arrayOf()`.

```
ReactComponent.propTypes = {
  arrayWithShape: React.PropTypes.arrayOf(
    React.PropTypes.shape({
      color: React.PropTypes.string.isRequired,
      fontSize: React.PropTypes.number.isRequired,
    })
  )
}
```



```
    ).isRequired,
  };

```

[↑ Back to Top](#)

60. How to conditionally apply class attributes?

You shouldn't use curly braces inside quotes because it is going to be evaluated as a string.

```
<div className="btn-panel {this.props.visible ? 'show' : 'hidden'}">

```



Instead you need to move curly braces outside (don't forget to include spaces between class names):

```
<div className={'btn-panel ' + (this.props.visible ? 'show' : 'hidden')}>

```



Template strings will also work:

```
<div className={`btn-panel ${this.props.visible ? 'show' : 'hidden'}`}>

```



[↑ Back to Top](#)

61. What is the difference between React and ReactDOM?

The `react` package contains `React.createElement()`, `React.Component`, `React.Children`, and other helpers related to elements and component classes. You can think of these as the isomorphic or universal helpers that you need to build components. The `react-dom` package contains `ReactDOM.render()`, and in `react-dom/server` we have *server-side rendering* support with `ReactDOMServer.renderToString()` and `ReactDOMServer.renderToStaticMarkup()`.

[↑ Back to Top](#)

62. Why ReactDOM is separated from React?

The React team worked on extracting all DOM-related features into a separate library called *ReactDOM*. React v0.14 is the first release in which the libraries are split. By looking at some of the packages, `react-native`, `react-art`, `react-canvas`, and `react-three`, it has become clear that the beauty and essence of React has nothing to do with browsers or the DOM.

To build more environments that React can render to, React team planned to split the main React package into two: `react` and `react-dom`. This paves the way to writing components that can be shared between the web version of React and React Native.

[↑ Back to Top](#)

63. How to use React label element?

If you try to render a `<label>` element bound to a text input using the standard `for` attribute, then it produces HTML missing that attribute and prints a warning to the console.

```
<label for={'user'}>{'User'}</label>
<input type={'text'} id={'user'} />

```



Since `for` is a reserved keyword in JavaScript, use `htmlFor` instead.

```
<label htmlFor={'user'}>{'User'}</label>
<input type={'text'} id={'user'} />

```



[↑ Back to Top](#)

64. How to combine multiple inline style objects?

You can use *spread operator* in regular React:

```
<button style={{ ...styles.panel.button, ...styles.panel.submitButton }}>
  {"Submit"}
</button>

```



If you're using React Native then you can use the array notation:

```
<button style={[styles.panel.button, styles.panel.submitButton]}}>
  {"Submit"}
</button>
```

[↑ Back to Top](#)

65. How to re-render the view when the browser is resized?

You can use the `useState` hook to manage the width and height state variables, and the `useEffect` hook to add and remove the `resize` event listener. The `[]` dependency array passed to `useEffect` ensures that the effect only runs once (on mount) and not on every re-render.

```
import React, { useState, useEffect } from "react";
function WindowDimensions() {
  const [dimensions, setDimensions] = useState({
    width: window.innerWidth,
    height: window.innerHeight,
  });

  useEffect(() => {
    function handleResize() {
      setDimensions({
        width: window.innerWidth,
        height: window.innerHeight,
      });
    }
    window.addEventListener("resize", handleResize);
    return () => window.removeEventListener("resize", handleResize);
  }, []);

  return (
    <span>
      {dimensions.width} x {dimensions.height}
    </span>
  );
}
```

► Using Class Component

[↑ Back to Top](#)

66. How to pretty print JSON with React?

We can use `<pre>` tag so that the formatting of the `JSON.stringify()` is retained:

```
const data = { name: "John", age: 42 };

function User {
  return <pre>{JSON.stringify(data, null, 2)}</pre>;
}

const container = createRoot(document.getElementById("container"));

container.render(<User />);
```

► See Class

[↑ Back to Top](#)

67. Why can't you update props in React?

The React philosophy is that props should be *immutable* (read only) and *top-down*. This means that a parent can send any prop values to a child, but the child can't modify received props.

[↑ Back to Top](#)

68. How to focus an input element on page load?

You need to use `useEffect` hook to set focus on input field during page load time for functional component.

```
import React, { useEffect, useRef } from "react";

const App = () => {
  const inputElRef = useRef(null);

  useEffect(() => {
    inputElRef.current.focus();
  }, []);

  return (
    <div>
      <input defaultValue={"Won't focus"} />
      <input ref={inputElRef} defaultValue={"Will focus"} />
    </div>
  );
};

ReactDOM.render(<App />, document.getElementById("app"));
```

► See Class

[↑ Back to Top](#)

69. How can we find the version of React at runtime in the browser?

You can use `React.version` to get the version.

```
const REACT_VERSION = React.version;

ReactDOM.render(
  <div>{`React version: ${REACT_VERSION}`}</div>,
  document.getElementById("app")
);
```

[↑ Back to Top](#)

70. How to add Google Analytics for React Router?

Add a listener on the `history` object to record each page view:

```
history.listen(function (location) {
  window.ga("set", "page", location.pathname + location.search);
  window.ga("send", "pageview", location.pathname + location.search);
});
```

[↑ Back to Top](#)

71. How do you apply vendor prefixes to inline styles in React?

React *does not* apply *vendor prefixes* automatically. You need to add vendor prefixes manually.

```
<div
  style={{
    transform: "rotate(90deg)",
    WebkitTransform: "rotate(90deg)", // note the capital 'W' here
    msTransform: "rotate(90deg)", // 'ms' is the only lowercase vendor prefix
  }}
/>
```

[↑ Back to Top](#)

72. How to import and export components using React and ES6?

You should use default for exporting the components

```
import User from "user";

export default function MyProfile {
  return <User type="customer">{...}</User>;
}
```

[► See Class](#)[↑ Back to Top](#)

73. What are the exceptions on React component naming?

The component names should start with an uppercase letter but there are few exceptions to this convention. The lowercase tag names with a dot (property accessors) are still considered as valid component names. For example, the below tag can be compiled to a valid component,

```
render() {  
  return (  
    <obj.component/> // `React.createElement(obj.component)`  
  )  
}
```

[↑ Back to Top](#)

74. Is it possible to use async/await in plain React?

Yes, you can use `async/await` in plain React, as long as your JavaScript environment supports ES2017+. Nowadays most modern browsers and build tools support ES2017+ version. If you're using **Create React App**, **Next.js**, **Remix**, or any modern React setup, `async/await` is supported out of the box through **Babel**.

Example Usage

```
import { useEffect, useState } from 'react';  
  
function UserProfile() {  
  const [user, setUser] = useState(null);  
  
  useEffect(() => {  
    const fetchUser = async () => {  
      const response = await fetch('/api/user');  
      const data = await response.json();  
      setUser(data);  
    };  
  
    fetchUser();  
  }, []);  
  
  return user ? <div>Hello, {user.name}</div> : <div>Loading...</div>;  
}
```

But If you're not using a bundler like **Webpack** or **Babel**, you will need *Babel* and [transform-async-to-generator](#) plugin. However, React Native ships with Babel and a set of transforms.

[↑ Back to Top](#)

75. What are the common folder structures for React?

There are two common practices for React project file structure.

i. Grouping by features or routes:

One common way to structure projects is locate CSS, JS, and tests together, grouped by feature or route.

```
common/  
├─ Avatar.js  
├─ Avatar.css  
├─ APIUtils.js  
└─ APIUtils.test.js  
feed/  
├─ index.js  
├─ Feed.js  
├─ Feed.css  
├─ FeedStory.js  
├─ FeedStory.test.js  
└─ FeedAPI.js  
profile/  
├─ index.js  
└─ Profile.js
```

```
├─ ProfileHeader.js
├─ ProfileHeader.css
└─ ProfileAPI.js
```

ii. Grouping by file type:

Another popular way to structure projects is to group similar files together.

```
api/
├─ APIUtils.js
├─ APIUtils.test.js
├─ ProfileAPI.js
└─ UserAPI.js
components/
├─ Avatar.js
├─ Avatar.css
├─ Feed.js
├─ Feed.css
├─ FeedStory.js
├─ FeedStory.test.js
├─ Profile.js
├─ ProfileHeader.js
└─ ProfileHeader.css
```

[↑ Back to Top](#)

76. What are the popular packages for animation?

React Transition Group and *React Motion* are popular animation packages in React ecosystem.

[↑ Back to Top](#)

77. What is the benefit of styles modules?

It is recommended to avoid hard coding style values in components. Any values that are likely to be used across different UI components should be extracted into their own modules.

For example, these styles could be extracted into a separate component:

```
export const colors = {
  white,
  black,
  blue,
};

export const space = [0, 8, 16, 32, 64];
```

And then imported individually in other components:

```
import { space, colors } from "../styles";
```

[↑ Back to Top](#)

78. What are the popular React-specific linters?

ESLint is a popular JavaScript linter. There are plugins available that analyse specific code styles. One of the most common for React is an npm package called `eslint-plugin-react`. By default, it will check a number of best practices, with rules checking things from keys in iterators to a complete set of prop types.

Another popular plugin is `eslint-plugin-jsx-a11y`, which will help fix common issues with accessibility. As JSX offers slightly different syntax to regular HTML, issues with `alt` text and `tabindex`, for example, will not be picked up by regular plugins.

[↑ Back to Top](#)

React Router

[↑ Back to Top](#)

79. What is React Router?

React Router is a powerful routing library built on top of React that helps you add new screens and flows to your application incredibly quickly, all while keeping the URL in sync with what's being displayed on the page.

[↑ Back to Top](#)

80. How React Router is different from history library?

React Router is a wrapper around the `history` library which handles interaction with the browser's `window.history` with its browser and hash histories. It also provides memory history which is useful for environments that don't have global history, such as mobile app development (React Native) and unit testing with Node.

[↑ Back to Top](#)

81. What are the `<Router>` components of React Router v6?

React Router v6 provides below 4 `<Router>` components:

- `<BrowserRouter>` :Uses the HTML5 history API for standard web apps.
- `<HashRouter>` :Uses hash-based routing for static servers.
- `<MemoryRouter>` :Uses in-memory routing for testing and non-browser environments.
- `<StaticRouter>` :Provides static routing for server-side rendering (SSR).

The above components will create *browser*, *hash*, *memory* and *static* history instances. React Router v6 makes the properties and methods of the `history` instance associated with your router available through the context in the `router` object.

[↑ Back to Top](#)

82. What is the purpose of `push()` and `replace()` methods of `history`?

A history instance has two methods for navigation purpose.

- `push()`
- `replace()`

If you think of the history as an array of visited locations, `push()` will add a new location to the array and `replace()` will replace the current location in the array with the new one.

[↑ Back to Top](#)

83. How do you programmatically navigate using React Router v4?

There are three different ways to achieve programmatic routing/navigation within components.

i. Using the `withRouter()` higher-order function:

The `withRouter()` higher-order function will inject the history object as a prop of the component. This object provides `push()` and `replace()` methods to avoid the usage of context.

```
import { withRouter } from "react-router-dom"; // this also works with 'react-router-native'

const Button = withRouter(({ history }) => (
  <button
    type="button"
    onClick={() => {
      history.push("/new-location");
    }}
  >
    {"Click Me!"}
  </button>
));
```

ii. Using `<Route>` component and render props pattern:

The `<Route>` component passes the same props as `withRouter()`, so you will be able to access the history methods through the history prop.

```
import { Route } from "react-router-dom";

const Button = () => (
  <Route
    render={({ history }) => (
```

```

    <button
      type="button"
      onClick={() => {
        history.push("/new-location");
      }}
    >
      {"Click Me!"}
    </button>
  )}
/>
);

```

iii. Using context:

This option is not recommended and treated as unstable API.

```

const Button = (props, context) => (
  <button
    type="button"
    onClick={() => {
      context.history.push("/new-location");
    }}
  >
    {"Click Me!"}
  </button>
);

Button.contextTypes = {
  history: React.PropTypes.shape({
    push: React.PropTypes.func.isRequired,
  }),
};

```

[↑ Back to Top](#)

84. How to get query parameters in React Router v4?

The ability to parse query strings was taken out of React Router v4 because there have been user requests over the years to support different implementation. So the decision has been given to users to choose the implementation they like. The recommended approach is to use query strings library.

```

const queryString = require("query-string");
const parsed = queryString.parse(props.location.search);

```

You can also use `URLSearchParams` if you want something native:

```

const params = new URLSearchParams(props.location.search);
const foo = params.get("name");

```

You should use a *polyfill* for IE11.

[↑ Back to Top](#)

85. Why you get "Router may have only one child element" warning?

You have to wrap your Route's in a `<Switch>` block because `<Switch>` is unique in that it renders a route exclusively.

At first you need to add `Switch` to your imports:

```

import { Switch, Router, Route } from "react-router";

```

Then define the routes within `<Switch>` block:

```

<Router>
  <Switch>
    <Route { /* ... */ } />
    <Route { /* ... */ } />
  </Switch>
</Router>

```

[↑ Back to Top](#)

86. How to pass params to `history.push` method in React Router v4?

While navigating you can pass props to the `history` object:

```
this.props.history.push({
  pathname: "/template",
  search: "?name=sudheer",
  state: { detail: response.data },
});
```

The `search` property is used to pass query params in `push()` method.

[↑ Back to Top](#)

87. How to implement *default* or *NotFound* page?

A `<Switch>` renders the first child `<Route>` that matches. A `<Route>` with no path always matches. So you just need to simply drop path attribute as below

```
<Switch>
  <Route exact path="/" component={Home} />
  <Route path="/user" component={User} />
  <Route component={NotFound} />
</Switch>
```

[↑ Back to Top](#)

88. How to get history on React Router v4?

Below are the list of steps to get history object on React Router v4,

- i. Create a module that exports a `history` object and import this module across the project.

For example, create `history.js` file:

```
import { createBrowserHistory } from "history";

export default createBrowserHistory({
  /* pass a configuration object here if needed */
});
```

- ii. You should use the `<Router>` component instead of built-in routers. Import the above `history.js` inside `index.js` file:

```
import { Router } from "react-router-dom";
import history from "./history";
import App from "./App";

ReactDOM.render(
  <Router history={history}>
    <App />
  </Router>,
  holder
);
```

- iii. You can also use push method of `history` object similar to built-in history object:

```
// some-other-file.js
import history from "./history";

history.push("/go-here");
```

[↑ Back to Top](#)

89. How to perform automatic redirect after login?

The `react-router` package provides `<Redirect>` component in React Router. Rendering a `<Redirect>` will navigate to a new location. Like server-side redirects, the new location will override the current location in the history stack.


```
import { Redirect } from "react-router";

export default function Login {
  if (this.state.isLoggedIn === true) {
    return <Redirect to="/your/redirect/page" />;
  } else {
    return <div>{"Login Please"}</div>;
  }
}
```

► See Class

[↑ Back to Top](#)

React Internationalization

90. What is React Intl?

The *React Intl* library makes internationalization in React straightforward, with off-the-shelf components and an API that can handle everything from formatting strings, dates, and numbers, to pluralization. React Intl is part of *FormatJS* which provides bindings to React via its components and API.

[↑ Back to Top](#)

91. What are the main features of React Intl?

Below are the main features of React Intl,

- i. Display numbers with separators.
- ii. Display dates and times correctly.
- iii. Display dates relative to "now".
- iv. Pluralize labels in strings.
- v. Support for 150+ languages.
- vi. Runs in the browser and Node.
- vii. Built on standards.

[↑ Back to Top](#)

92. What are the two ways of formatting in React Intl?

The library provides two ways to format strings, numbers, and dates:

i. Using react components:

```
<FormattedMessage
  id={"account"}
  defaultMessage={"The amount is less than minimum balance."}
/>
```

ii. Using an API:

```
const messages = defineMessages({
  accountMessage: {
    id: "account",
    defaultMessage: "The amount is less than minimum balance.",
  },
});

formatMessage(messages.accountMessage);
```

[↑ Back to Top](#)

93. How to use `<FormattedMessage>` as placeholder using React Intl?

The `<Formatted... />` components from `react-intl` return elements, not plain text, so they can't be used for placeholders, alt text, etc. In that case, you should use lower level API `formatMessage()`. You can inject the `intl` object into your component using `injectIntl()` higher-order component and then format the message using `formatMessage()` available on that object.

```
import React from "react";
import { injectIntl, intlShape } from "react-intl";

const MyComponent = ({ intl }) => {
  const placeholder = intl.formatMessage({ id: "messageId" });
  return <input placeholder={placeholder} />;
};

MyComponent.propTypes = {
  intl: intlShape.isRequired,
};

export default injectIntl(MyComponent);
```

[↑ Back to Top](#)

94. How to access current locale with React Intl?

You can get the current locale in any component of your application using `injectIntl()` :

```
import { injectIntl, intlShape } from "react-intl";

const MyComponent = ({ intl }) => (
  <div>`The current locale is ${intl.locale}`</div>
);

MyComponent.propTypes = {
  intl: intlShape.isRequired,
};

export default injectIntl(MyComponent);
```

[↑ Back to Top](#)

95. How to format date using React Intl?

The `injectIntl()` higher-order component will give you access to the `formatDate()` method via the props in your component. The method is used internally by instances of `FormattedDate` and it returns the string representation of the formatted date.

```
import { injectIntl, intlShape } from "react-intl";

const stringDate = this.props.intl.formatDate(date, {
  year: "numeric",
  month: "numeric",
  day: "numeric",
});

const MyComponent = ({ intl }) => (
  <div>`The formatted date is ${stringDate}`</div>
);

MyComponent.propTypes = {
  intl: intlShape.isRequired,
};

export default injectIntl(MyComponent);
```

[↑ Back to Top](#)

React Testing

96. What is Shallow Renderer in React testing?

Shallow rendering is useful for writing unit test cases in React. It lets you render a component *one level deep* and assert facts about what its render method returns, without worrying about the behavior of child components, which are not instantiated or rendered.

For example, if you have the following component:

```
function MyComponent() {
  return (
    <div>
```

```

    <span className={"heading"}>{"Title"}</span>
    <span className={"description"}>{"Description"}</span>
  </div>
);
}

```

Then you can assert as follows:

```

import ShallowRenderer from "react-test-renderer/shallow";

// in your test
const renderer = new ShallowRenderer();
renderer.render(<MyComponent />);

const result = renderer.getRenderOutput();

expect(result.type).toBe("div");
expect(result.props.children).toEqual([
  <span className={"heading"}>{"Title"}</span>,
  <span className={"description"}>{"Description"}</span>,
]);

```

[↑ Back to Top](#)

97. What is `TestRenderer` package in React?

This package provides a renderer that can be used to render components to pure JavaScript objects, without depending on the DOM or a native mobile environment. This package makes it easy to grab a snapshot of the platform view hierarchy (similar to a DOM tree) rendered by a ReactDOM or React Native without using a browser or `jsdom`.

```

import TestRenderer from "react-test-renderer";

const Link = ({ page, children }) => <a href={page}>{children}</a>;

const testRenderer = TestRenderer.create(
  <Link page={"https://www.facebook.com/"}>{"Facebook"}</Link>
);

console.log(testRenderer.toJSON());
// {
//   type: 'a',
//   props: { href: 'https://www.facebook.com/' },
//   children: [ 'Facebook' ]
// }

```

[↑ Back to Top](#)

98. What is the purpose of `ReactTestUtils` package?

`ReactTestUtils` are provided in the `with-addons` package and allow you to perform actions against a simulated DOM for the purpose of unit testing.

[↑ Back to Top](#)

99. What is Jest?

`Jest` is a JavaScript unit testing framework created by Facebook based on Jasmine and provides automated mock creation and a `jsdom` environment. It's often used for testing components.

[↑ Back to Top](#)

100. What are the advantages of Jest over Jasmine?

There are couple of advantages compared to Jasmine:

- Automatically finds tests to execute in your source code.
- Automatically mocks dependencies when running your tests.
- Allows you to test asynchronous code synchronously.
- Runs your tests with a fake DOM implementation (via `jsdom`) so that your tests can be run on the command line.
- Runs tests in parallel processes so that they finish sooner.

[↑ Back to Top](#)

101. Give a simple example of Jest test case

Let's write a test for a function that adds two numbers in `sum.js` file:

```
const sum = (a, b) => a + b;

export default sum;
```

Create a file named `sum.test.js` which contains actual test:

```
import sum from './sum';

test('adds 1 + 2 to equal 3', () => {
  expect(sum(1, 2)).toBe(3);
});
```

And then add the following section to your `package.json` :

```
{
  "scripts": {
    "test": "jest"
  }
}
```

Finally, run `yarn test` or `npm test` and Jest will print a result:

```
$ yarn test
PASS ./sum.test.js
✓ adds 1 + 2 to equal 3 (2ms)
```

React Redux

[↑ Back to Top](#)

102. What is flux?

Flux is an **application architecture** (not a framework or library) designed by Facebook to manage **data flow** in React applications. It was created as an alternative to the traditional **MVC (Model-View-Controller)** pattern, and it emphasizes a **unidirectional data flow** to make state changes more predictable and easier to debug.

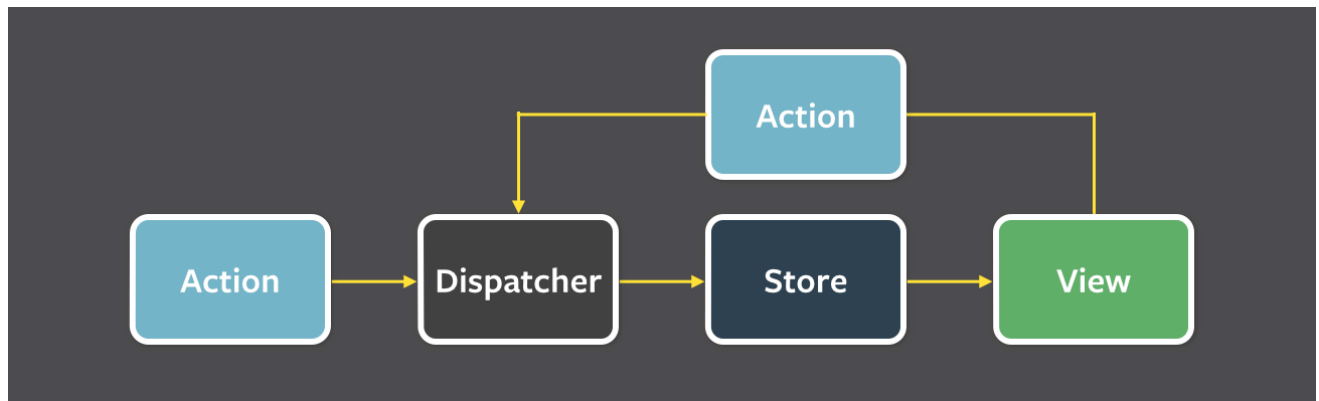
Flux complements React by organizing the way data moves through your application, especially in large-scale or complex projects.

Core Concepts of Flux

Flux operates using **four key components**, each with a specific responsibility:

- **Actions**
 - Plain JavaScript objects or functions that describe *what happened* (e.g., user interactions or API responses).
 - Example: `{ type: 'ADD_TODO', payload: 'Buy milk' }`
- **Dispatcher**
 - A central hub that receives actions and **dispatches** them to the appropriate stores.
 - There is **only one dispatcher** in a Flux application.
- **Stores**
 - Hold the **application state** and business logic.
 - Respond to actions from the dispatcher and update themselves accordingly.
 - They **emit change events** that views can listen to.
- **Views (React Components)**
 - Subscribe to stores and **re-render** when the data changes.
 - They can also trigger new actions (e.g., on user input).

The workflow between dispatcher, stores and views components with distinct inputs and outputs as follows:



[↑ Back to Top](#)

103. What is Redux?

Redux is a predictable state container for JavaScript applications, most commonly used with React. It helps you manage and centralize your application's state in a single source of truth, enabling easier debugging, testing, and maintenance—especially in large or complex applications. Redux core is tiny library (about 2.5kB gzipped) and has no dependencies.

[↑ Back to Top](#)

104. What are the core principles of Redux?

Redux follows three fundamental principles:

- i. **Single source of truth:** The state of your whole application is stored in an object tree within a single store. The single state tree makes it easier to keep track of changes over time and debug or inspect the application.

```
const store = createStore(reducer);
```

- ii. **State is read-only:** The only way to change the state is to emit an action, an object describing what happened. This ensures that neither the views nor the network callbacks will ever write directly to the state.

```
const action = { type: 'INCREMENT' };
store.dispatch(action);
```

- iii. **Changes are made with pure functions (Reducers):** To specify how the state tree is transformed by actions, you write reducers. Reducers are just pure functions that take the previous state and an action as parameters, and return the next state.

```
function counter(state = 0, action) {
  switch (action.type) {
    case 'INCREMENT':
      return state + 1;
    case 'DECREMENT':
      return state - 1;
    default:
      return state;
  }
}
```

[↑ Back to Top](#)

105. What are the downsides of Redux compared to Flux?

While Redux offers a powerful and predictable state management solution, it comes with a few trade-offs when compared to Flux. These include:

- i. **Immutability is essential**

Redux enforces a strict immutability model for state updates, which differs from Flux's more relaxed approach. This means you must avoid mutating state directly. Many Redux-related libraries assume immutability, so your team must be disciplined in writing pure update logic. You can use tools like `redux-immutable-state-invariant`, `Immer`, or `Immutable.js` to help enforce this practice, especially during development.

- ii. **Careful selection of complementary packages**

Redux is more minimal by design and provides extension points such as middleware and store enhancers. This has led to a large ecosystem, but it also means you must thoughtfully choose and configure additional packages for features like undo/redo, persistence, or form handling—something Flux explicitly leaves out but may be simpler to manage in smaller setups.

iii. Limited static type integration

While Flux has mature support for static type checking with tools like Flow, Redux's type integration is less seamless. Although TypeScript is commonly used with Redux now, early Flow support was limited, and more boilerplate was required for static type safety. This may affect teams that rely heavily on type systems for large codebases.

[↑ Back to Top](#)

106. What is the difference between `mapStateToProps()` and `mapDispatchToProps()` ?

`mapStateToProps()` is a utility which helps your component get updated state (which is updated by some other components):

```
const mapStateToProps = (state) => {  
  return {  
    todos: getVisibleTodos(state.todos, state.visibilityFilter),  
  };  
};
```

`mapDispatchToProps()` is a utility which will help your component to fire an action event (dispatching action which may cause change of application state):

```
const mapDispatchToProps = (dispatch) => {  
  return {  
    onTodoClick: (id) => {  
      dispatch(toggleTodo(id));  
    },  
  };  
};
```

It is recommended to always use the "object shorthand" form for the `mapDispatchToProps` .

Redux wraps it in another function that looks like `(...args) => dispatch(onTodoClick(...args))`, and pass that wrapper function as a prop to your component.

```
const mapDispatchToProps = {  
  onTodoClick,  
};
```

[↑ Back to Top](#)

107. Can I dispatch an action in reducer?

Dispatching an action within a reducer is an **anti-pattern**. Your reducer should be *without side effects*, simply digesting the action payload and returning a new state object. Adding listeners and dispatching actions within the reducer can lead to chained actions and other side effects.

[↑ Back to Top](#)

108. How to access Redux store outside a component?

You just need to export the store from the module where it created with `createStore()` . Also, it shouldn't pollute the global window object.

```
store = createStore(myReducer);  
  
export default store;
```

[↑ Back to Top](#)

109. What are the drawbacks of MVW pattern?

- i. DOM manipulation is very expensive which causes applications to behave slow and inefficient.
- ii. Due to circular dependencies, a complicated model was created around models and views.
- iii. Lot of data changes happens for collaborative applications(like Google Docs).
- iv. No way to do undo (travel back in time) easily without adding so much extra code.

[↑ Back to Top](#)

110. Are there any similarities between Redux and RxJS?

These libraries are very different for very different purposes, but there are some vague similarities.

Redux is a tool for managing state throughout the application. It is usually used as an architecture for UIs. Think of it as an alternative to (half of) Angular. RxJS is a reactive programming library. It is usually used as a tool to accomplish asynchronous tasks in JavaScript. Think of it as an alternative to Promises. Redux uses the Reactive paradigm because the Store is reactive. The Store observes actions from a distance, and changes itself. RxJS also uses the Reactive paradigm, but instead of being an architecture, it gives you basic building blocks, Observables, to accomplish this pattern.

[↑ Back to Top](#)

111. How to reset state in Redux?

You need to write a *root reducer* in your application which delegate handling the action to the reducer generated by `combineReducers()`.

For example, let us take `rootReducer()` to return the initial state after `USER_LOGOUT` action. As we know, reducers are supposed to return the initial state when they are called with `undefined` as the first argument, no matter the action.

```
const appReducer = combineReducers({
  /* your app's top-level reducers */
});

const rootReducer = (state, action) => {
  if (action.type === "USER_LOGOUT") {
    state = undefined;
  }

  return appReducer(state, action);
};
```

In case of using `redux-persist`, you may also need to clean your storage. `redux-persist` keeps a copy of your state in a storage engine. First, you need to import the appropriate storage engine and then, to parse the state before setting it to undefined and clean each storage state key.

```
const appReducer = combineReducers({
  /* your app's top-level reducers */
});

const rootReducer = (state, action) => {
  if (action.type === "USER_LOGOUT") {
    Object.keys(state).forEach((key) => {
      storage.removeItem(`persist:${key}`);
    });

    state = undefined;
  }

  return appReducer(state, action);
};
```

[↑ Back to Top](#)

112. What is the difference between React context and React Redux?

You can use **Context** in your application directly and is going to be great for passing down data to deeply nested components which what it was designed for.

Whereas **Redux** is much more powerful and provides a large number of features that the Context API doesn't provide. Also, React Redux uses context internally but it doesn't expose this fact in the public API.

[↑ Back to Top](#)

113. Why are Redux state functions called reducers?

Reducers always return the accumulation of the state (based on all previous and current actions). Therefore, they act as a reducer of state. Each time a Redux reducer is called, the state and action are passed as parameters. This state is then reduced (or accumulated) based on the action, and then the next state is returned. You could *reduce* a collection of actions and an initial state (of the store) on which to perform these actions to get the resulting final state.

[↑ Back to Top](#)

114. How to make AJAX request in Redux?

You can use `redux-thunk` middleware which allows you to define async actions.

Let's take an example of fetching specific account as an AJAX call using *fetch API*:

```
export function fetchAccount(id) {
  return (dispatch) => {
    dispatch(setLoadingAccountState()); // Show a loading spinner
    fetch(`/account/${id}`, (response) => {
      dispatch(doneFetchingAccount()); // Hide loading spinner
      if (response.status === 200) {
        dispatch(setAccount(response.json)); // Use a normal function to set the received state
      } else {
        dispatch(someError);
      }
    });
  };
}

function setAccount(data) {
  return { type: "SET_Account", data: data };
}
```

[↑ Back to Top](#)

115. Should I keep all component's state in Redux store?

Keep your data in the Redux store, and the UI related state internally in the component.

[↑ Back to Top](#)

116. What is the proper way to access Redux store?

The best way to access your store in a component is to use the `connect()` function, that creates a new component that wraps around your existing one. This pattern is called *Higher-Order Components*, and is generally the preferred way of extending a component's functionality in React. This allows you to map state and action creators to your component, and have them passed in automatically as your store updates.

Let's take an example of `<FilterLink>` component using `connect`:

```
import { connect } from "react-redux";
import { setVisibilityFilter } from "../actions";
import Link from "../components/Link";

const mapStateToProps = (state, ownProps) => ({
  active: ownProps.filter === state.visibilityFilter,
});

const mapDispatchToProps = (dispatch, ownProps) => ({
  onClick: () => dispatch(setVisibilityFilter(ownProps.filter)),
});

const FilterLink = connect(mapStateToProps, mapDispatchToProps)(Link);

export default FilterLink;
```

Due to it having quite a few performance optimizations and generally being less likely to cause bugs, the Redux developers almost always recommend using `connect()` over accessing the store directly (using context API).

```
function MyComponent {
  someMethod() {
    doSomethingWith(this.context.store);
  }
}
```

[↑ Back to Top](#)

117. What is the difference between component and container in React Redux?

Component is a class or function component that describes the presentational part of your application.

Container is an informal term for a component that is connected to a Redux store. Containers *subscribe* to Redux state updates and *dispatch* actions, and they usually don't render DOM elements; they delegate rendering to presentational child components.

[↑ Back to Top](#)

118. What is the purpose of the constants in Redux?

Constants allows you to easily find all usages of that specific functionality across the project when you use an IDE. It also prevents you from introducing silly bugs caused by typos – in which case, you will get a `ReferenceError` immediately.

Normally we will save them in a single file (`constants.js` or `actionTypes.js`).

```
export const ADD_TODO = "ADD_TODO";
export const DELETE_TODO = "DELETE_TODO";
export const EDIT_TODO = "EDIT_TODO";
export const COMPLETE_TODO = "COMPLETE_TODO";
export const COMPLETE_ALL = "COMPLETE_ALL";
export const CLEAR_COMPLETED = "CLEAR_COMPLETED";
```

In Redux, you use them in two places:

i. During action creation:

Let's take `actions.js` :

```
import { ADD_TODO } from "../actionTypes";

export function addTodo(text) {
  return { type: ADD_TODO, text };
}
```

ii. In reducers:

Let's create `reducer.js` :

```
import { ADD_TODO } from "../actionTypes";

export default (state = [], action) => {
  switch (action.type) {
    case ADD_TODO:
      return [
        ...state,
        {
          text: action.text,
          completed: false,
        },
      ];
    default:
      return state;
  }
};
```

[↑ Back to Top](#)

119. What are the different ways to write `mapDispatchToProps()` ?

There are a few ways of binding *action creators* to `dispatch()` in `mapDispatchToProps()` .

Below are the possible options:

```
const mapDispatchToProps = (dispatch) => ({
  action: () => dispatch(action()),
});
```

```
const mapDispatchToProps = (dispatch) => ({
  action: bindActionCreators(action, dispatch),
});
```

```
const mapDispatchToProps = { action };
```

The third option is just a shorthand for the first one.

[↑ Back to Top](#)

120. What is the use of the `ownProps` parameter in `mapStateToProps()` and `mapDispatchToProps()`?

If the `ownProps` parameter is specified, React Redux will pass the props that were passed to the component into your *connect* functions. So, if you use a connected component:

```
import ConnectedComponent from "../containers/ConnectedComponent";  
  
<ConnectedComponent user={"john"} />;
```

The `ownProps` inside your `mapStateToProps()` and `mapDispatchToProps()` functions will be an object:

```
{  
  user: "john";  
}
```

You can use this object to decide what to return from those functions.

[↑ Back to Top](#)

121. How to structure Redux top level directories?

Most of the applications has several top-level directories as below:

- i. **Components:** Used for *dumb* components unaware of Redux.
- ii. **Containers:** Used for *smart* components connected to Redux.
- iii. **Actions:** Used for all action creators, where file names correspond to part of the app.
- iv. **Reducers:** Used for all reducers, where files name correspond to state key.
- v. **Store:** Used for store initialization.

This structure works well for small and medium size apps.

[↑ Back to Top](#)

122. What is redux-saga?

`redux-saga` is a library that aims to make side effects (asynchronous things like data fetching and impure things like accessing the browser cache) in React/Redux applications easier and better.

It is available in NPM:

```
$ npm install --save redux-saga
```

[↑ Back to Top](#)

123. What is the mental model of redux-saga?

Saga is like a separate thread in your application, that's solely responsible for side effects. `redux-saga` is a *redux middleware*, which means this thread can be started, paused and cancelled from the main application with normal Redux actions, it has access to the full Redux application state and it can dispatch Redux actions as well.

[↑ Back to Top](#)

124. What are the differences between `call()` and `put()` in redux-saga?

Both `call()` and `put()` are effect creator functions. `call()` function is used to create effect description, which instructs middleware to call the promise. `put()` function creates an effect, which instructs middleware to dispatch an action to the store.

Let's take example of how these effects work for fetching particular user data.

```
function* fetchUserSaga(action) {  
  // `call` function accepts rest arguments, which will be passed to `api.fetchUser` function.  
  // Instructing middleware to call promise, it resolved value will be assigned to `userData` variable  
  const userData = yield call(api.fetchUser, action.userId);  
}
```

```
// Instructing middleware to dispatch corresponding action.  
yield put({  
  type: "FETCH_USER_SUCCESS",  
  userData,  
});  
}
```

[↑ Back to Top](#)

125. What is Redux Thunk?

Redux Thunk middleware allows you to write action creators that return a function instead of an action. The thunk can be used to delay the dispatch of an action, or to dispatch only if a certain condition is met. The inner function receives the store methods `dispatch()` and `getState()` as parameters.

[↑ Back to Top](#)

126. What are the differences between `redux-saga` and `redux-thunk`?

Both *Redux Thunk* and *Redux Saga* take care of dealing with side effects. In most of the scenarios, Thunk uses *Promises* to deal with them, whereas Saga uses *Generators*. Thunk is simple to use and Promises are familiar to many developers, Sagas/Generators are more powerful but you will need to learn them. But both middleware can coexist, so you can start with Thunks and introduce Sagas when/if you need them.

[↑ Back to Top](#)

127. What is Redux DevTools?

Redux DevTools is a live-editing time travel environment for Redux with hot reloading, action replay, and customizable UI. If you don't want to bother with installing Redux DevTools and integrating it into your project, consider using Redux DevTools Extension for Chrome and Firefox.

[↑ Back to Top](#)

128. What are the features of Redux DevTools?

Some of the main features of Redux DevTools are below,

- i. Lets you inspect every state and action payload.
- ii. Lets you go back in time by *cancelling* actions.
- iii. If you change the reducer code, each *staged* action will be re-evaluated.
- iv. If the reducers throw, you will see during which action this happened, and what the error was.
- v. With `persistState()` store enhancer, you can persist debug sessions across page reloads.

[↑ Back to Top](#)

129. What are Redux selectors and why use them?

Selectors are functions that take Redux state as an argument and return some data to pass to the component.

For example, to get user details from the state:

```
const getUserData = (state) => state.user.data;
```



These selectors have two main benefits,

- i. The selector can compute derived data, allowing Redux to store the minimal possible state
- ii. The selector is not recomputed unless one of its arguments changes

[↑ Back to Top](#)

130. What is Redux Form?

Redux Form works with React and Redux to enable a form in React to use Redux to store all of its state. Redux Form can be used with raw HTML5 inputs, but it also works very well with common UI frameworks like Material UI, React Widgets and React Bootstrap.

[↑ Back to Top](#)

131. What are the main features of Redux Form?

Some of the main features of Redux Form are:

- i. Field values persistence via Redux store.
- ii. Validation (sync/async) and submission.
- iii. Formatting, parsing and normalization of field values.

[↑ Back to Top](#)

132. How to add multiple middlewares to Redux?

You can use `applyMiddleware()` .

For example, you can add `redux-thunk` and `logger` passing them as arguments to `applyMiddleware()` :

```
import { createStore, applyMiddleware } from "redux";
const createStoreWithMiddleware = applyMiddleware(
  ReduxThunk,
  logger
)(createStore);
```

[↑ Back to Top](#)

133. How to set initial state in Redux?

You need to pass initial state as second argument to `createStore`:

```
const rootReducer = combineReducers({
  todos: todos,
  visibilityFilter: visibilityFilter,
});

const initialState = {
  todos: [{ id: 123, name: "example", completed: false }],
};

const store = createStore(rootReducer, initialState);
```

[↑ Back to Top](#)

134. How Relay is different from Redux?

Relay is similar to Redux in that they both use a single store. The main difference is that relay only manages state originated from the server, and all access to the state is used via *GraphQL* queries (for reading data) and mutations (for changing data). Relay caches the data for you and optimizes data fetching for you, by fetching only changed data and nothing more.

135. What is an action in Redux?

Actions are plain JavaScript objects or payloads of information that send data from your application to your store. They are the only source of information for the store. Actions must have a `type` property that indicates the type of action being performed.

For example, let's take an action which represents adding a new todo item:

```
{
  type: ADD_TODO,
  text: 'Add todo item'
}
```

[↑ Back to Top](#)

React Native

[↑ Back to Top](#)

136. What is the difference between React Native and React?

React is a JavaScript library, supporting both front end web and being run on the server, for building user interfaces and web applications.

React Native is a mobile framework that compiles to native app components, allowing you to build native mobile applications (iOS, Android, and Windows) in JavaScript that allows you to use React to build your components, and implements React under the hood.

[↑ Back to Top](#)

137. How to test React Native apps?

React Native can be tested only in mobile simulators like iOS and Android. You can run the app in your mobile using expo app (<https://expo.io>) Where it syncs using QR code, your mobile and computer should be in same wireless network.

[↑ Back to Top](#)

138. How to do logging in React Native?

You can use `console.log`, `console.warn`, etc. As of React Native v0.29 you can simply run the following to see logs in the console:

```
$ react-native log-ios  
$ react-native log-android
```

[↑ Back to Top](#)

139. How to debug your React Native?

Follow the below steps to debug React Native app:

- Run your application in the iOS simulator.
- Press `Command + D` and a webpage should open up at `http://localhost:8081/debugger-ui`.
- Enable *Pause On Caught Exceptions* for a better debugging experience.
- Press `Command + Option + I` to open the Chrome Developer tools, or open it via `View -> Developer -> Developer Tools`.
- You should now be able to debug as you normally would.

React supported libraries & Integration

[↑ Back to Top](#)

140. What is reselect and how it works?

Reselect is a **selector library** (for Redux) which uses *memoization* concept. It was originally written to compute derived data from Redux-like applications state, but it can't be tied to any architecture or library.

Reselect keeps a copy of the last inputs/outputs of the last call, and recomputes the result only if one of the inputs changes. If the same inputs are provided twice in a row, Reselect returns the cached output. It's memoization and cache are fully customizable.

[↑ Back to Top](#)

141. What is Flow?

Flow is a *static type checker* designed to find type errors in JavaScript. Flow types can express much more fine-grained distinctions than traditional type systems. For example, Flow helps you catch errors involving `null`, unlike most type systems.

[↑ Back to Top](#)

142. What is the difference between Flow and PropTypes?

Flow is a *static analysis tool* (static checker) which uses a superset of the language, allowing you to add type annotations to all of your code and catch an entire class of bugs at compile time.

PropTypes is a *basic type checker* (runtime checker) which has been patched onto React. It can't check anything other than the types of the props being passed to a given component. If you want more flexible typechecking for your entire project Flow/TypeScript are appropriate choices.

[↑ Back to Top](#)

143. How to use Font Awesome icons in React?

The below steps followed to include Font Awesome in React:

- Install `font-awesome`:

```
$ npm install --save font-awesome
```



- ii. Import `font-awesome` in your `index.js` file:

```
import "font-awesome/css/font-awesome.min.css";
```

- iii. Add Font Awesome classes in `className` :

```
function MyComponent {  
  return <div><i className={'fa fa-spinner'} /></div>  
}
```

[↑ Back to Top](#)

144. What is React Dev Tools?

React Developer Tools let you inspect the component hierarchy, including component props and state. It exists both as a browser extension (for Chrome and Firefox), and as a standalone app (works with other environments including Safari, IE, and React Native).

The official extensions available for different browsers or environments.

- i. Chrome extension
- ii. Firefox extension
- iii. Standalone app (Safari, React Native, etc)

[↑ Back to Top](#)

145. Why is DevTools not loading in Chrome for local files?

If you opened a local HTML file in your browser (`file:///...`) then you must first open *Chrome Extensions* and check `Allow access to file URLs` .

[↑ Back to Top](#)

146. How to use Polymer in React?

You need to follow below steps to use Polymer in React,

- i. Create a Polymer element:

```
<link  
  rel="import"  
  href="../../bower_components/polymer/polymer.html"  
>;  
Polymer({  
  is: "calendar-element",  
  ready: function () {  
    this.textContent = "I am a calendar";  
  },  
});
```

- ii. Create the Polymer component HTML tag by importing it in a HTML document, e.g. import it in the `index.html` of your React application:

```
<link  
  rel="import"  
  href="../src/polymer-components/calendar-element.html"  
>
```

- iii. Use that element in the JSX file:

```
export default function MyComponent {  
  return <calendar-element />;  
}
```

[↑ Back to Top](#)

147. What are the advantages of React over Vue.js?

React has the following advantages over Vue.js:

- i. Gives more flexibility in large apps developing.
- ii. Easier to test.
- iii. Suitable for mobile apps creating.
- iv. More information and solutions available.

Note: The above list of advantages are purely opinionated and it vary based on the professional experience. But they are helpful as base parameters.

[↑ Back to Top](#)

148. What is the difference between React and Angular?

Let's see the difference between React and Angular in a table format.

React	Angular
React is a library and has only the View layer	Angular is a framework and has complete MVC functionality
React handles rendering on the server side	AngularJS renders only on the client side but Angular 2 and above renders on the server side
React uses JSX that looks like HTML in JS which can be confusing	Angular follows the template approach for HTML, which makes code shorter and easy to understand
React Native, which is a React type to build mobile applications are faster and more stable	Ionic, Angular's mobile native app is relatively less stable and slower
In React, data flows only in one way and hence debugging is easy	In Angular, data flows both way i.e it has two-way data binding between children and parent and hence debugging is often difficult

Note: The above list of differences are purely opinionated and it vary based on the professional experience. But they are helpful as base parameters.

[↑ Back to Top](#)

149. Why React tab is not showing up in DevTools?

When the page loads, *React DevTools* sets a global named `__REACT_DEVTOOLS_GLOBAL_HOOK__`, then React communicates with that hook during initialization. If the website is not using React or if React fails to communicate with DevTools then it won't show up the tab.

[↑ Back to Top](#)

150. What are Styled Components?

`styled-components` is a JavaScript library for styling React applications. It removes the mapping between styles and components, and lets you write actual CSS augmented with JavaScript.

[↑ Back to Top](#)

151. Give an example of Styled Components?

Lets create `<Title>` and `<Wrapper>` components with specific styles for each.

```
import React from "react";
import styled from "styled-components";

// Create a <Title> component that renders an <h1> which is centered, red and sized at 1.5em
const Title = styled.h1`
  font-size: 1.5em;
  text-align: center;
  color: palevioletred;
`;

// Create a <Wrapper> component that renders a <section> with some padding and a papayawhip background
const Wrapper = styled.section`
  padding: 4em;
  background: papayawhip;
`;
```

These two variables, `Title` and `Wrapper`, are now components that you can render just like any other react component.

```
<Wrapper>
  <Title>{"Lets start first styled component!"}</Title>
</Wrapper>
```

[↑ Back to Top](#)

152. What is Relay?

Relay is a JavaScript framework for providing a data layer and client-server communication to web applications using the React view layer.

[↑ Back to Top](#)

Miscellaneous

153. What are the main features of Reselect library?

Let's see the main features of Reselect library,

- i. Selectors can compute derived data, allowing Redux to store the minimal possible state.
- ii. Selectors are efficient. A selector is not recomputed unless one of its arguments changes.
- iii. Selectors are composable. They can be used as input to other selectors.

154. Give an example of Reselect usage?

Let's take calculations and different amounts of a shipment order with the simplified usage of Reselect:

```
import { createSelector } from "reselect";

const shopItemsSelector = (state) => state.shop.items;
const taxPercentSelector = (state) => state.shop.taxPercent;

const subtotalSelector = createSelector(shopItemsSelector, (items) =>
  items.reduce((acc, item) => acc + item.value, 0)
);

const taxSelector = createSelector(
  subtotalSelector,
  taxPercentSelector,
  (subtotal, taxPercent) => subtotal * (taxPercent / 100)
);

export const totalSelector = createSelector(
  subtotalSelector,
  taxSelector,
  (subtotal, tax) => ({ total: subtotal + tax })
);

let exampleState = {
  shop: {
    taxPercent: 8,
    items: [
      { name: "apple", value: 1.2 },
      { name: "orange", value: 0.95 },
    ],
  },
};

console.log(subtotalSelector(exampleState)); // 2.15
console.log(taxSelector(exampleState)); // 0.172
console.log(totalSelector(exampleState)); // { total: 2.322 }
```

[↑ Back to Top](#)

155. Can Redux only be used with React?

Redux can be used as a data store for any UI layer. The most common usage is with React and React Native, but there are bindings available for Angular, Angular 2, Vue, Mithril, and more. Redux simply provides a subscription mechanism which can be used by any other code.

[↑ Back to Top](#)

156. Do you need to have a particular build tool to use Redux?

Redux is originally written in ES6 and transpiled for production into ES5 with Webpack and Babel. You should be able to use it regardless of your JavaScript build process. Redux also offers a UMD build that can be used directly without any build process at all.

[↑ Back to Top](#)

157. How Redux Form `initialValues` get updated from state?

You need to add `enableReinitialize : true` setting.

```
const InitializeFromStateForm = reduxForm({
  form: "initializeFromState",
  enableReinitialize: true,
})(UserEdit);
```

If your `initialValues` prop gets updated, your form will update too.

[↑ Back to Top](#)

158. How React PropTypes allow different types for one prop?

You can use `oneOfType()` method of `PropTypes`.

For example, the height property can be defined with either `string` or `number` type as below:

```
Component.propTypes = {
  size: PropTypes.oneOfType([PropTypes.string, PropTypes.number]),
};
```

[↑ Back to Top](#)

159. Can I import an SVG file as react component?

You can import SVG directly as component instead of loading it as a file. This feature is available with `react-scripts@2.0.0` and higher.

```
import { ReactComponent as Logo } from "../logo.svg";

const App = () => (
  <div>
    {/* Logo is an actual react component */}
    <Logo />
  </div>
);
```

Note: Don't forget about the curly braces in the import.

[↑ Back to Top](#)

160. What is render hijacking in react?

The concept of render hijacking is the ability to control what a component will output from another component. It means that you decorate your component by wrapping it into a Higher-Order component. By wrapping, you can inject additional props or make other changes, which can cause changing logic of rendering. It does not actually enable hijacking, but by using HOC you make your component behave differently.

[↑ Back to Top](#)

161. How to pass numbers to React component?

We can pass `numbers` as `props` to React component using curly braces `{}` where as `strings` in double quotes `""` or single quotes `' '`

```
import React from "react";

const ChildComponent = ({ name, age }) => {
  return (
    <>
      My Name is {name} and Age is {age}
    </>
  );
};
```

```
};

const ParentComponent = () => {
  return (
    <>
      <ChildComponent name="Chetan" age={30} />
    </>
  );
};

export default ParentComponent;
```

[↑ Back to Top](#)

162. Do I need to keep all my state into Redux? Should I ever use react internal state?

It is up to the developer's decision, i.e., it is developer's job to determine what kinds of state make up your application, and where each piece of state should live. Some users prefer to keep every single piece of data in Redux, to maintain a fully serializable and controlled version of their application at all times. Others prefer to keep non-critical or UI state, such as "is this dropdown currently open", inside a component's internal state.

Below are the rules of thumb to determine what kind of data should be put into Redux

- Do other parts of the application care about this data?
- Do you need to be able to create further derived data based on this original data?
- Is the same data being used to drive multiple components?
- Is there value to you in being able to restore this state to a given point in time (ie, time travel debugging)?
- Do you want to cache the data (i.e, use what's in state if it's already there instead of re-requesting it)?

[↑ Back to Top](#)

163. What is the purpose of registerServiceWorker in React?

React creates a service worker for you without any configuration by default. The service worker is a web API that helps you cache your assets and other files so that when the user is offline or on a slow network, he/she can still see results on the screen, as such, it helps you build a better user experience, that's what you should know about service worker for now. It's all about adding offline capabilities to your site.

```
import React from "react";
import ReactDOM from "react-dom";
import App from "./App";
import registerServiceWorker from "./registerServiceWorker";

ReactDOM.render(<App />, document.getElementById("root"));
registerServiceWorker();
```

[↑ Back to Top](#)

164. What is React memo function?

Class components can be restricted from re-rendering when their input props are the same using **PureComponent** or **shouldComponentUpdate**. Now you can do the same with function components by wrapping them in **React.memo**.

```
const MyComponent = React.memo(function MyComponent(props) {
  /* only rerenders if props change */
});
```

[↑ Back to Top](#)

165. What is React lazy function?

The **React.lazy** function lets you render a dynamic import as a regular component. It will automatically load the bundle containing the **OtherComponent** when the component gets rendered. This must return a Promise which resolves to a module with a default export containing a React component.

```
const OtherComponent = React.lazy(() => import("./OtherComponent"));

function MyComponent() {
  return (
```

```

    <div>
      <OtherComponent />
    </div>
  );
}

```

Note: `React.lazy` and `Suspense` is not yet available for server-side rendering. If you want to do code-splitting in a server rendered app, we still recommend `React Loadable`.

[↑ Back to Top](#)

166. How to prevent unnecessary updates using `setState`?

You can compare the current value of the state with an existing state value and decide whether to rerender the page or not. If the values are the same then you need to return `null` to stop re-rendering otherwise return the latest state value.

For example, the user profile information is conditionally rendered as follows,

```

getUserProfile = (user) => {
  const latestAddress = user.address;
  this.setState((state) => {
    if (state.address === latestAddress) {
      return null;
    } else {
      return { title: latestAddress };
    }
  });
};

```

[↑ Back to Top](#)

167. How do you render Array, Strings and Numbers in React 16 Version?

Arrays: Unlike older releases, you don't need to make sure `render` method return a single element in React16. You are able to return multiple sibling elements without a wrapping element by returning an array.

For example, let us take the below list of developers,

```

const ReactJSDevs = () => {
  return [
    <li key="1">John</li>,
    <li key="2">Jackie</li>,
    <li key="3">Jordan</li>,
  ];
};

```

You can also merge this array of items in another array component.

```

const JSDevs = () => {
  return (
    <ul>
      <li>Brad</li>
      <li>Brodge</li>
      <ReactJSDevs />
      <li>Brandon</li>
    </ul>
  );
};

```

Strings and Numbers: You can also return string and number type from the render method.

```

render() {
  return 'Welcome to ReactJS questions';
}
// Number
render() {
  return 2018;
}

```

[↑ Back to Top](#)

168. What are hooks?

Hooks is a special JavaScript function that allows you use state and other React features without writing a class. This pattern has been introduced as a new feature in React 16.8 and helped to isolate the stateful logic from the components.

Let's see an example of useState hook:

```
import { useState } from "react";

function Example() {
  // Declare a new state variable, which we'll call "count"
  const [count, setCount] = useState(0);

  return (
    <>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>Click me</button>
    </>
  );
}
```

Note: Hooks can be used inside an existing function component without rewriting the component.

[↑ Back to Top](#)

169. What rules need to be followed for hooks?

You need to follow two rules in order to use hooks,

- i. **Call Hooks only at the top level of your react functions:** You should always use hooks at the top level of react function before any early returns. i.e, You shouldn't call Hooks inside loops, conditions, or nested functions. This will ensure that Hooks are called in the same order each time a component renders and it preserves the state of Hooks between multiple re-renders due to `useState` and `useEffect` calls.

Let's see the difference using an example, **Correct usage::**

```
function UserProfile() {
  // Correct: Hooks called at the top level
  const [name, setName] = useState('John');
  const [country, setCountry] = useState('US');

  return (
    <div>
      <h1>Name: {name}</h1>
      <p>Country: {country}</p>
    </div>
  );
}
```

Incorrect usage::

```
function UserProfile() {
  const [name, setName] = useState('John');

  if (name === 'John') {
    // Incorrect: useState is called inside a conditional
    const [country, setCountry] = useState('US');
  }

  return (
    <div>
      <h1>Name: {name}</h1>
      <p>Country: {country}</p> {/* This will throw an error if the name condition isn't met */}
    </div>
  );
}
```

The `useState` hook for the country field is being called conditionally within an `if` block. This can lead to inconsistent state behavior and may cause hooks to be called in a different order on each re-render.

- ii. **Call Hooks from React Functions only:** You shouldn't call Hooks from regular JavaScript functions or class components. Instead, you should call them from either function components or custom hooks.

Let's find the difference of correct and incorrect usage with below examples,

Correct usage::

```
//Example1:
function Counter() {
  // Correct: useState is used inside a functional component
  const [count, setCount] = useState(0);

  return <div>Counter: {count}</div>;
}

//Example2:
function useFetchData(url) {
  const [data, setData] = useState(null);

  useEffect(() => {
    fetch(url)
      .then((response) => response.json())
      .then((data) => setData(data));
  }, [url]);

  return data;
}

function UserProfile() {
  // Correct: Using a custom hook here
  const user = useFetchData('https://some-api.com/user');

  return (
    <div>
      <h1>{user ? user.name : 'Loading profile...'}</h1>
    </div>
  );
}
```

Incorrect usage::

```
//Example1
function normalFunction() {
  // Incorrect: Can't call hooks in normal functions
  const [count, setCount] = useState(0);
}

//Example2
function fetchData(url) {
  // Incorrect: Hooks can't be used in non-React functions
  const [data, setData] = useState(null);

  useEffect(() => {
    fetch(url)
      .then((response) => response.json())
      .then((data) => setData(data));
  }, [url]);

  return data;
}
```

In the above incorrect usage example, both `useState` and `useEffect` are used in non-React functions(`normalFunction` and `fetchData`), which is not allowed.

[↑ Back to Top](#)

170. How to ensure hooks followed the rules in your project?

React team released an ESLint plugin called `eslint-plugin-react-hooks` that enforces Hook's two rules. It is part of Hooks API. You can add this plugin to your project using the below command,

```
npm install eslint-plugin-react-hooks --save-dev
```

And apply the below config in your ESLint config file,

```
// Your ESLint configuration
{
```

```

"plugins": [
  // ...
  "react-hooks"
],
"rules": {
  // ...
  "react-hooks/rules-of-hooks": "error"
}
}

```

This plugin also provide another important rule through `react-hooks/exhaustive-deps`. It ensures that the dependencies of `useEffect`, `useCallback`, and `useMemo` hooks are correctly listed to avoid potential bugs.

```

useEffect(() => {
  // Forgetting `message` will result in incorrect behavior
  console.log(message);
}, []); // Here `message` should be a dependency

```

The recommended `eslint-config-react-app` preset already includes the hooks rules of this plugin. For example, the linter enforce proper naming convention for hooks. If you rename your custom hooks which as prefix "use" to something else then linter won't allow you to call built-in hooks such as `useState`, `useEffect` etc inside of your custom hook anymore.

Note: This plugin is intended to use in Create React App by default.

[↑ Back to Top](#)

171. What are the differences between Flux and Redux?

Below are the major differences between Flux and Redux

Flux	Redux
State is mutable	State is immutable
The Store contains both state and change logic	The Store and change logic are separate
There are multiple stores exist	There is only one store exist
All the stores are disconnected and flat	Single store with hierarchical reducers
It has a singleton dispatcher	There is no concept of dispatcher
React components subscribe to the store	Container components uses connect function

[↑ Back to Top](#)

172. What are the benefits of React Router V4?

Below are the main benefits of React Router V4 module,

- In React Router v4(version 4), the API is completely about components. A router can be visualized as a single component(`<BrowserRouter>`) which wraps specific child router components(`<Route>`).
- You don't need to manually set history. The router module will take care history by wrapping routes with `<BrowserRouter>` component.
- The application size is reduced by adding only the specific router module(Web, core, or native)

[↑ Back to Top](#)

173. Can you describe about componentDidCatch lifecycle method signature?

The `componentDidCatch` lifecycle method is invoked after an error has been thrown by a descendant component. The method receives two parameters,

- error: - The error object which was thrown
- info: - An object with a `componentStack` key contains the information about which component threw the error.

The method structure would be as follows

```

componentDidCatch(error, info);

```

[↑ Back to Top](#)

174. In which scenarios do error boundaries not catch errors?

Below are the cases in which error boundaries don't work,

- i. Inside Event handlers
- ii. Asynchronous code using `setTimeout` or `requestAnimationFrame` callbacks
- iii. During Server side rendering
- iv. When errors thrown in the error boundary code itself

[↑ Back to Top](#)

175. What is the behavior of uncaught errors in react 16?

In React 16, errors that were not caught by any error boundary will result in unmounting of the whole React component tree. The reason behind this decision is that it is worse to leave corrupted UI in place than to completely remove it. For example, it is worse for a payments app to display a wrong amount than to render nothing.

[↑ Back to Top](#)

176. What is the proper placement for error boundaries?

The granularity of error boundaries usage is up to the developer based on project needs. You can follow either of these approaches,

- i. You can wrap top-level route components to display a generic error message for the entire application.
- ii. You can also wrap individual components in an error boundary to protect them from crashing the rest of the application.

[↑ Back to Top](#)

177. What is the benefit of component stack trace from error boundary?

Apart from error messages and javascript stack, React16 will display the component stack trace with file names and line numbers using error boundary concept.

For example, BuggyCounter component displays the component stack trace as below,

```
► React caught an error thrown by BuggyCounter. You should fix this error in your code. react-dom.development.js:7708
React will try to recreate this component tree from scratch using the error boundary you provided, ErrorBoundary.
Error: I crashed!
The error is located at:
  in BuggyCounter (at App.js:26)
  in ErrorBoundary (at App.js:21)
  in div (at App.js:8)
  in App (at index.js:5)
```

[↑ Back to Top](#)

178. What are default props?

The `defaultProps` can be defined as a property on the component to set the default values for the props. These default props are used when props not supplied (i.e., undefined props), but not for `null` or `0` as props. That means, If you provide null value then it remains null value. It's the same behavior with 0 as well.

For example, let us create color default prop for the button component,

```
function MyButton {
  // ...
}

MyButton.defaultProps = {
  color: "red",
};
```

If `props.color` is not provided then it will set the default value to 'red'. i.e, Whenever you try to access the color prop it uses the default value

```
function MyButton() {
  return <MyButton />; // props.color will contain red value
}
```

[↑ Back to Top](#)

179. What is the purpose of displayName class property?

The displayName string is used in debugging messages. Usually, you don't need to set it explicitly because it's inferred from the name of the function or class that defines the component. You might want to set it explicitly if you want to display a different name for debugging purposes or when you create a higher-order component.

For example, To ease debugging, choose a display name that communicates that it's the result of a withSubscription HOC.

```
function withSubscription(WrappedComponent) {  
  class WithSubscription extends React.Component {  
    /* ... */  
  }  
  WithSubscription.displayName = `WithSubscription(${getDisplayName(  
    WrappedComponent  
  )})`;   
  return WithSubscription;  
}  
function getDisplayName(WrappedComponent) {  
  return (  
    WrappedComponent.displayName || WrappedComponent.name || "Component"  
  );  
}
```

[↑ Back to Top](#)

180. What is the browser support for react applications?

React supports all popular browsers, including Internet Explorer 9 and above, although some polyfills are required for older browsers such as IE 9 and IE 10. If you use es5-shim and es5-sham polyfill then it even support old browsers that doesn't support ES5 methods.

[↑ Back to Top](#)

181. What is code-splitting?

Code-Splitting is a feature supported by bundlers like Webpack and Browserify which can create multiple bundles that can be dynamically loaded at runtime. The react project supports code splitting via dynamic import() feature.

For example, in the below code snippets, it will make moduleA.js and all its unique dependencies as a separate chunk that only loads after the user clicks the 'Load' button.

moduleA.js

```
const moduleA = "Hello";  
  
export { moduleA };
```

App.js

```
export default function App {  
  function handleClick() {  
    import("./moduleA")  
      .then(({ moduleA }) => {  
        // Use moduleA  
      })  
      .catch((err) => {  
        // Handle failure  
      });  
  }  
  
  return (  
    <div>  
      <button onClick={this.handleClick}>Load</button>  
    </div>  
  );  
}
```

► See Class

[↑ Back to Top](#)

182. What are Keyed Fragments?

The Fragments declared with the explicit `<React.Fragment>` syntax may have keys. The general use case is mapping a collection to an array of fragments as below,

```
function Glossary(props) {
  return (
    <dl>
      {props.items.map((item) => (
        // Without the `key`, React will fire a key warning
        <React.Fragment key={item.id}>
          <dt>{item.term}</dt>
          <dd>{item.description}</dd>
        </React.Fragment>
      ))}
    </dl>
  );
}
```

Note: key is the only attribute that can be passed to Fragment. In the future, there might be a support for additional attributes, such as event handlers.

[↑ Back to Top](#)

183. Does React support all HTML attributes?

As of React 16, both standard or custom DOM attributes are fully supported. Since React components often take both custom and DOM-related props, React uses the camelCase convention just like the DOM APIs.

Let us take few props with respect to standard HTML attributes,

```
<div tabIndex="-1" /> // Just like node.tabIndex DOM API
<div className="Button" /> // Just like node.className DOM API
<input readOnly={true} /> // Just like node.readOnly DOM API
```

These props work similarly to the corresponding HTML attributes, with the exception of the special cases. It also support all SVG attributes.

[↑ Back to Top](#)

184. When component props defaults to true?

If you pass no value for a prop, it defaults to true. This behavior is available so that it matches the behavior of HTML.

For example, below expressions are equivalent,

```
<MyInput autocomplete />
<MyInput autocomplete={true} />
```

Note: It is not recommended to use this approach because it can be confused with the ES6 object shorthand (example, `{name}` which is short for `{name: name}`)

[↑ Back to Top](#)

185. What is NextJS and major features of it?

Next.js is a popular and lightweight framework for static and server-rendered applications built with React. It also provides styling and routing solutions. Below are the major features provided by NextJS,

- i. Server-rendered by default
- ii. Automatic code splitting for faster page loads
- iii. Simple client-side routing (page based)
- iv. Webpack-based dev environment which supports (HMR)
- v. Able to implement with Express or any other Node.js HTTP server
- vi. Customizable with your own Babel and Webpack configurations

[↑ Back to Top](#)

186. How do you pass an event handler to a component?

You can pass event handlers and other functions as props to child components. The functions can be passed to child component as below,

```
function Button({ onClick }) {
  return <button onClick={onClick}>Download</button>;
}

export default function downloadExcel() {
  function handleClick() {
    alert("Downloaded");
  }

  return <Button onClick={handleClick}></Button>;
}
```

[↑ Back to Top](#)

187. How to prevent a function from being called multiple times?

If you use an event handler such as `onClick` or `onScroll` and want to prevent the callback from being fired too quickly, then you can limit the rate at which callback is executed. This can be achieved in the below possible ways,

- Throttling:** Changes based on a time based frequency. For example, it can be used using `_throttle` lodash function
- Debouncing:** Publish changes after a period of inactivity. For example, it can be used using `_debounce` lodash function
- RequestAnimationFrame throttling:** Changes based on requestAnimationFrame. For example, it can be used using `raf-schd` lodash function

[↑ Back to Top](#)

188. How JSX prevents Injection Attacks?

React DOM escapes any values embedded in JSX before rendering them. Thus it ensures that you can never inject anything that's not explicitly written in your application. Everything is converted to a string before being rendered.

For example, you can embed user input as below,

```
const name = response.potentiallyMaliciousInput;
const element = <h1>{name}</h1>;
```

This way you can prevent XSS(Cross-site-scripting) attacks in the application.

[↑ Back to Top](#)

189. How do you update rendered elements?

You can update UI(represented by rendered element) by passing the newly created element to ReactDOM's render method.

For example, lets take a ticking clock example, where it updates the time by calling render method multiple times,

```
function tick() {
  const element = (
    <div>
      <h1>Hello, world!</h1>
      <h2>It is {new Date().toLocaleTimeString()}.</h2>
    </div>
  );
  ReactDOM.render(element, document.getElementById("root"));
}

setInterval(tick, 1000);
```

[↑ Back to Top](#)

190. How do you say that props are readonly?

When you declare a component as a function or a class, it must never modify its own props.

Let us take a below capital function,

```
function capital(amount, interest) {
  return amount + interest;
}
```

The above function is called “pure” because it does not attempt to change their inputs, and always return the same result for the same inputs. Hence, React has a single rule saying “All React components must act like pure functions with respect to their props.”

[↑ Back to Top](#)

191. What are the conditions to safely use the index as a key?

There are three conditions to make sure, it is safe use the index as a key.

- The list and items are static– they are not computed and do not change
- The items in the list have no ids
- The list is never reordered or filtered.

[↑ Back to Top](#)

192. Should keys be globally unique?

The keys used within arrays should be unique among their siblings but they don’t need to be globally unique. i.e, You can use the same keys with two different arrays.

For example, the below `Book` component uses two arrays with different arrays,

```
function Book(props) {  
  const index = (  
    <ul>  
      {props.pages.map((page) => (  
        <li key={page.id}>{page.title}</li>  
      ))}  
    </ul>  
  );  
  const content = props.pages.map((page) => (  
    <div key={page.id}>  
      <h3>{page.title}</h3>  
      <p>{page.content}</p>  
      <p>{page.pageNumber}</p>  
    </div>  
  ));  
  return (  
    <div>  
      {index}  
      <hr />  
      {content}  
    </div>  
  );  
}
```

[↑ Back to Top](#)

193. What is the popular choice for form handling?

`Formik` is a form library for react which provides solutions such as validation, keeping track of the visited fields, and handling form submission.

In detail, You can categorize them as follows,

- Getting values in and out of form state
- Validation and error messages
- Handling form submission

It is used to create a scalable, performant, form helper with a minimal API to solve annoying stuff.

[↑ Back to Top](#)

194. What are the advantages of formik over redux form library?

Below are the main reasons to recommend formik over redux form library,

- The form state is inherently short-term and local, so tracking it in Redux (or any kind of Flux library) is unnecessary.
- Redux-Form calls your entire top-level Redux reducer multiple times ON EVERY SINGLE KEYSTROKE. This way it increases input latency for large apps.
- Redux-Form is 22.5 kB minified gzipped whereas Formik is 12.7 kB

[↑ Back to Top](#)

195. Why are you not required to use inheritance?

In React, it is recommended to use composition over inheritance to reuse code between components. Both Props and composition give you all the flexibility you need to customize a component's look and behavior explicitly and safely. Whereas, If you want to reuse non-UI functionality between components, it is suggested to extract it into a separate JavaScript module. Later components import it and use that function, object, or class, without extending it.

[↑ Back to Top](#)

196. Can I use web components in react application?

Yes, you can use web components in a react application. Even though many developers won't use this combination, it may require especially if you are using third-party UI components that are written using Web Components.

For example, let us use `Vaadin` date picker web component as below,

```
import './App.css';
import '@vaadin/vaadin-date-picker';
export default function App() {
  return (
    <div className="App">
      <vaadin-date-picker label="When were you born?"></vaadin-date-picker>
    </div>
  );
}
```

[↑ Back to Top](#)

197. What is dynamic import?

You can achieve code-splitting in your app using dynamic import.

Let's take an example of addition,

i. Normal Import

```
import { add } from './math';
console.log(add(10, 20));
```

ii. Dynamic Import

```
import('./math').then((math) => {
  console.log(math.add(10, 20));
});
```

[↑ Back to Top](#)

198. What are loadable components?

With the release of React 18, `React.lazy` and `Suspense` are now available for server-side rendering. However, prior to React 18, it was recommended to use Loadable Components for code-splitting in a server-side rendered app because `React.lazy` and `Suspense` were not available for server-side rendering. Loadable Components lets you render a dynamic import as a regular component. For example, you can use Loadable Components to load the `OtherComponent` in a separate bundle like this:

```
import loadable from '@loadable/component';

const OtherComponent = loadable(() => import('./OtherComponent'));

function MyComponent() {
  return (
    <div>
      <OtherComponent />
    </div>
  );
}
```

Now `OtherComponent` will be loaded in a separated bundle Loadable Components provides additional benefits beyond just code-splitting, such as automatic code reloading, error handling, and preloading. By using Loadable Components, you can ensure that your application loads quickly and efficiently, providing a better user experience for your users.

[↑ Back to Top](#)

199. What is suspense component?

React Suspense is a built-in feature that lets you defer rendering part of your component tree until some condition(asynchronous operation) is met—usually, data or code has finished loading. While waiting, Suspense lets you display a fallback UI like a spinner or placeholder.

i. Lazy loading components uses suspense feature,

If the module containing the dynamic import is not yet loaded by the time parent component renders, you must show some fallback content while you're waiting for it to load using a loading indicator. This can be done using **Suspense** component.

```
const OtherComponent = React.lazy(() => import("./OtherComponent"));

function MyComponent() {
  return (
    <div>
      <Suspense fallback={<div>Loading...</div>}>
        <OtherComponent />
      </Suspense>
    </div>
  );
}
```

The above component shows fallback UI instead real component until `OtherComponent` is fully loaded.

ii. As an another example, suspend until async data(data fetching) is ready

```
function UserProfile() {
  const user = use(fetchUser()); // throws a promise internally
  return <div>{user.name}</div>;
}

function App() {
  return (
    <Suspense fallback={<div>Loading user...</div>}>
      <UserProfile />
    </Suspense>
  );
}
```

...

[↑ Back to Top](#)

200. What is route based code splitting?

One of the best place to do code splitting is with routes. The entire page is going to re-render at once so users are unlikely to interact with other elements in the page at the same time. Due to this, the user experience won't be disturbed.

Let us take an example of route based website using libraries like React Router with React.lazy,

```
import { BrowserRouter as Router, Route, Switch } from "react-router-dom";
import React, { Suspense, lazy } from "react";

const Home = lazy(() => import("./routes/Home"));
const About = lazy(() => import("./routes/About"));

const App = () => (
  <Router>
    <Suspense fallback={<div>Loading...</div>}>
      <Switch>
        <Route exact path="/" component={Home} />
        <Route path="/about" component={About} />
      </Switch>
    </Suspense>
  </Router>
);
```

In the above code, the code splitting will happen at each route level.

[↑ Back to Top](#)

201. What is the purpose of default value in context?

The `defaultValue` argument is only used when a component does not have a matching Provider above it in the tree. This can be helpful for testing components in isolation without wrapping them.

Below code snippet provides default theme value as Luna.

```
const MyContext = React.createContext(defaultValue);
```

[↑ Back to Top](#)

202. What is diffing algorithm?

React needs to use algorithms to find out how to efficiently update the UI to match the most recent tree. The diffing algorithms is generating the minimum number of operations to transform one tree into another. However, the algorithms have a complexity in the order of $O(n^3)$ where n is the number of elements in the tree.

In this case, displaying 1000 elements would require in the order of one billion comparisons. This is far too expensive. Instead, React implements a heuristic $O(n)$ algorithm based on two assumptions:

- Two elements of different types will produce different trees.
- The developer can hint at which child elements may be stable across different renders with a key prop.

[↑ Back to Top](#)

203. What are the rules covered by diffing algorithm?

When diffing two trees, React first compares the two root elements. The behavior is different depending on the types of the root elements. It covers the below rules during reconciliation algorithm,

- Elements Of Different Types:** Whenever the root elements have different types, React will tear down the old tree and build the new tree from scratch. For example, elements to , or from
to of different types lead a full rebuild.
- DOM Elements Of The Same Type:** When comparing two React DOM elements of the same type, React looks at the attributes of both, keeps the same underlying DOM node, and only updates the changed attributes. Lets take an example with same DOM elements except `className` attribute,

```
<div className="show" title="ReactJS" />  
  
<div className="hide" title="ReactJS" />
```



- Component Elements Of The Same Type:** When a component updates, the instance stays the same, so that state is maintained across renders. React updates the props of the underlying component instance to match the new element, and calls `componentWillReceiveProps()` and `componentWillUpdate()` on the underlying instance. After that, the `render()` method is called and the diff algorithm recurses on the previous result and the new result.
- Recurring On Children:** when recursing on the children of a DOM node, React just iterates over both lists of children at the same time and generates a mutation whenever there's a difference. For example, when adding an element at the end of the children, converting between these two trees works well.

```
<ul>  
  <li>first</li>  
  <li>second</li>  
</ul>  
  
<ul>  
  <li>first</li>  
  <li>second</li>  
  <li>third</li>  
</ul>
```



- Handling keys:** React supports a key attribute. When children have keys, React uses the key to match children in the original tree with children in the subsequent tree. For example, adding a key can make the tree conversion efficient,

```

<ul>
  <li key="2015">Duke</li>
  <li key="2016">Villanova</li>
</ul>

<ul>
  <li key="2014">Connecticut</li>
  <li key="2015">Duke</li>
  <li key="2016">Villanova</li>
</ul>

```

[↑ Back to Top](#)

204. When do you need to use refs?

There are few use cases to go for refs,

- i. Managing focus, text selection, or media playback.
- ii. Triggering imperative animations.
- iii. Integrating with third-party DOM libraries.

[↑ Back to Top](#)

205. Must prop be named as render for render props?

Even though the pattern named render props, you don't have to use a prop named render to use this pattern. i.e, Any prop that is a function that a component uses to know what to render is technically a "render prop". Lets take an example with the children prop for render props,

```

<Mouse
  children={(mouse) => (
    <p>
      The mouse position is {mouse.x}, {mouse.y}
    </p>
  )}
/>

```

Actually children prop doesn't need to be named in the list of "attributes" in JSX element. Instead, you can keep it directly inside element,

```

<Mouse>
  {(mouse) => (
    <p>
      The mouse position is {mouse.x}, {mouse.y}
    </p>
  )}
</Mouse>

```

While using this above technique(without any name), explicitly state that children should be a function in your propTypes.

```

Mouse.propTypes = {
  children: PropTypes.func.isRequired,
};

```

[↑ Back to Top](#)

206. What are the problems of using render props with pure components?

If you create a function inside a render method, it negates the purpose of pure component. Because the shallow prop comparison will always return false for new props, and each render in this case will generate a new value for the render prop. You can solve this issue by defining the render function as instance method.

[↑ Back to Top](#)

207. What is windowing technique?

Windowing is a technique that only renders a small subset of your rows at any given time, and can dramatically reduce the time it takes to re-render the components as well as the number of DOM nodes created. If your application renders long lists of data then this technique is recommended. Both react-window and react-virtualized are popular windowing libraries which provides several reusable components for displaying lists, grids, and tabular data.

[↑ Back to Top](#)

208. How do you print falsy values in JSX?

The falsy values such as false, null, undefined, and true are valid children but they don't render anything. If you still want to display them then you need to convert it to string. Let's take an example on how to convert to a string,

```
<div>My JavaScript variable is {String(myVariable)}.</div>
```

[↑ Back to Top](#)

209. What is the typical use case of portals?

React Portals are primarily used to render UI components such as **modals, tooltips, dropdowns, hovercards, and notifications** outside of their parent component's DOM tree. This helps avoid common CSS issues caused by parent elements, such as:

- **overflow: hidden** on parent elements clipping or hiding child elements like modals or tooltips,
- **stacking context and z-index conflicts** created by parent containers that prevent child elements from appearing above other content.

That means, you need to visually "break out" of its container. By rendering these UI elements into a separate DOM node (often directly under `<body>`), portals ensure they appear above all other content and are not restricted by the parent's CSS or layout constraints, resulting in correct positioning and visibility regardless of the parent's styling.

[↑ Back to Top](#)

210. How do you set default value for uncontrolled component?

In React, the value attribute on form elements will override the value in the DOM. With an uncontrolled component, you might want React to specify the initial value, but leave subsequent updates uncontrolled. To handle this case, you can specify a **defaultValue** attribute instead of **value**.



```
````javascript
render() {
 return (
 <form onSubmit={this.handleSubmit}>
 <label>
 User Name:
 <input
 defaultValue="John"
 type="text"
 ref={this.input} />
 </label>
 <input type="submit" value="Submit" />
 </form>
);
}
```

The same applies for `select` and `textArea` inputs. But you need to use **defaultChecked** for `checkbox` and `radio` inputs.

[↑ Back to Top](#)

## 211. What is your favorite React stack?

Even though the tech stack varies from developer to developer, the most popular stack is used in react boilerplate project code. It mainly uses Redux and redux-saga for state management and asynchronous side-effects, react-router for routing purpose, styled-components for styling react components, axios for invoking REST api, and other supported stack such as webpack, reselect, ESNEXT, Babel. You can clone the project <https://github.com/react-boilerplate/react-boilerplate> and start working on any new react project.

[↑ Back to Top](#)

## 212. What is the difference between Real DOM and Virtual DOM?

Below are the main differences between Real DOM and Virtual DOM,

Real DOM	Virtual DOM
Updates are slow	Updates are fast



Real DOM	Virtual DOM
DOM manipulation is very expensive.	DOM manipulation is very easy
You can update HTML directly.	You Can't directly update HTML
It causes too much of memory wastage	There is no memory wastage
Creates a new DOM if element updates	It updates the JSX if element update

[↑ Back to Top](#)

### 213. How to add Bootstrap to a react application?

Bootstrap can be added to your React app in a three possible ways,

- Using the Bootstrap CDN: This is the easiest way to add bootstrap. Add both bootstrap CSS and JS resources in a head tag.
- Bootstrap as Dependency: If you are using a build tool or a module bundler such as Webpack, then this is the preferred option for adding Bootstrap to your React application

```
npm install bootstrap
```



- React Bootstrap Package: In this case, you can add Bootstrap to our React app is by using a package that has rebuilt Bootstrap components to work particularly as React components. Below packages are popular in this category,
  - react-bootstrap
  - reactstrap

[↑ Back to Top](#)

### 214. Can you list down top websites or applications using react as front end framework?

Below are the `top 10 websites` using React as their front-end framework,

- Facebook
- Uber
- Instagram
- WhatsApp
- Khan Academy
- Airbnb
- Dropbox
- Flipboard
- Netflix
- PayPal

[↑ Back to Top](#)

### 215. Is it recommended to use CSS In JS technique in React?

React does not have any opinion about how styles are defined but if you are a beginner then good starting point is to define your styles in a separate \*.css file as usual and refer to them using className. This functionality is not part of React but came from third-party libraries. But If you want to try a different approach(CSS-In-JS) then styled-components library is a good option.

[↑ Back to Top](#)

### 216. Do I need to rewrite all my class components with hooks?

No. But you can try Hooks in a few components(or new components) without rewriting any existing code. Because there are no plans to remove classes in ReactJS.

[↑ Back to Top](#)

### 217. What is useEffect hook? How to fetch data with React Hooks?

The `useEffect` hook is a React Hook that lets you perform **side effects** in function components. Side effects are operations that interact with the outside world or system and aren't directly related to rendering UI — such as fetching data, setting up subscriptions, timers, manually manipulating the DOM, etc.

In function components, useEffect replaces the class component lifecycle methods( `componentDidMount` , `componentDidUpdate` and `componentWillUnmount` ) with a single, unified API.

## Syntax

```
useEffect(() => {
 // Side effect logic here

 return () => {
 // Cleanup logic (optional)
 };
}, [dependencies]);
```

This effect hook can be used to fetch data from an API and to set the data in the local state of the component with the `useState` hook's update function.

Here is an example of fetching a list of ReactJS articles from an API using `fetch`.

```
import React from "react";

function App() {
 const [data, setData] = React.useState({ hits: [] });

 React.useEffect(() => {
 fetch("http://hn.algolia.com/api/v1/search?query=react")
 .then((response) => response.json())
 .then((data) => setData(data));
 }, []);

 return (

 {data.hits.map((item) => (
 <li key={item.objectID}>
 {item.title}

))}

);
}

export default App;
```

A popular way to simplify this is by using the library `axios`.

We provided an empty array as second argument to the `useEffect` hook to avoid activating it on component updates. This way, it only fetches on component mount.

[↑ Back to Top](#)

## 218. Is Hooks cover all use cases for classes?

Hooks doesn't cover all use cases of classes but there is a plan to add them soon. Currently there are no Hook equivalents to the uncommon `getSnapshotBeforeUpdate` and `componentDidCatch` lifecycles yet.

[↑ Back to Top](#)

## 219. What is the stable release for hooks support?

React includes a stable implementation of React Hooks in 16.8 release for below packages

- i. React DOM
- ii. React DOM Server
- iii. React Test Renderer
- iv. React Shallow Renderer

[↑ Back to Top](#)

## 220. Why do we use array destructuring (square brackets notation) in `useState` ?

When we declare a state variable with `useState`, it returns a pair — an array with two items. The first item is the current value, and the second is a function that updates the value. Using `[0]` and `[1]` to access them is a bit confusing because they have a specific meaning. This is why we use array destructuring instead.

For example, the array index access would look as follows:

```
var userStateVariable = useState("userProfile"); // Returns an array pair
var user = userStateVariable[0]; // Access first item
var setUser = userStateVariable[1]; // Access second item
```

Whereas with array destructuring the variables can be accessed as follows:

```
const [user, setUser] = useState("userProfile");
```

[↑ Back to Top](#)

## 221. What are the sources used for introducing hooks?

Hooks got the ideas from several different sources. Below are some of them,

- i. Previous experiments with functional APIs in the react-future repository
- ii. Community experiments with render prop APIs such as Reactions Component
- iii. State variables and state cells in DisplayScript.
- iv. Subscriptions in Rx.
- v. Reducer components in ReasonReact.

[↑ Back to Top](#)

## 222. How do you access imperative API of web components?

Web Components often expose an imperative API to implement its functions. You will need to use a **ref** to interact with the DOM node directly if you want to access imperative API of a web component. But if you are using third-party Web Components, the best solution is to write a React component that behaves as a **wrapper** for your Web Component.

[↑ Back to Top](#)

## 223. What is formik?

Formik is a small react form library that helps you with the three major problems,

- i. Getting values in and out of form state
- ii. Validation and error messages
- iii. Handling form submission

[↑ Back to Top](#)

## 224. What are typical middleware choices for handling asynchronous calls in Redux?

Some of the popular middleware choices for handling asynchronous calls in Redux ecosystem are `Redux Thunk`, `Redux Promise`, `Redux Saga`.

[↑ Back to Top](#)

## 225. Do browsers understand JSX code?

No, browsers can't understand JSX code. You need a transpiler to convert your JSX to regular Javascript that browsers can understand. The most widely used transpiler right now is Babel.

[↑ Back to Top](#)

## 226. Describe about data flow in react?

React implements one-way reactive data flow using props which reduce boilerplate and is easier to understand than traditional two-way data binding.

[↑ Back to Top](#)

## 227. What is MobX?

MobX is a simple, scalable and battle tested state management solution for applying functional reactive programming (TFRP). For ReactJS application, you need to install below packages,

```
npm install mobx --save
npm install mobx-react --save
```

[↑ Back to Top](#)

## 228. What are the differences between Redux and MobX?

Below are the main differences between Redux and MobX,

Topic	Redux	MobX
Definition	It is a javascript library for managing the application state	It is a library for reactively managing the state of your applications
Programming	It is mainly written in ES6	It is written in JavaScript(ES5)
Data Store	There is only one large store exist for data storage	There is more than one store for storage
Usage	Mainly used for large and complex applications	Used for simple applications
Performance	Need to be improved	Provides better performance
How it stores	Uses JS Object to store	Uses observable to store the data

[↑ Back to Top](#)

## 229. Should I learn ES6 before learning ReactJS?

No, you don't have to learn es2015/es6 to learn react. But you may find many resources or React ecosystem uses ES6 extensively. Let's see some of the frequently used ES6 features,

i. **Destructuring**: To get props and use them in a component

```
// in es 5
var someData = this.props.someData;
var dispatch = this.props.dispatch;

// in es6
const { someData, dispatch } = this.props;
```



ii. **Spread operator**: Helps in passing props down into a component

```
// in es 5
<SomeComponent someData={this.props.someData} dispatch={this.props.dispatch} />

// in es6
<SomeComponent {...this.props} />
```



iii. **Arrow functions**: Makes compact syntax

```
// es 5
var users = usersList.map(function (user) {
 return {user.name};
});

// es 6
const users = usersList.map((user) => {user.name});
```

[↑ Back to Top](#)

## 230. What is Concurrent Rendering?

The Concurrent rendering makes React apps to be more responsive by rendering component trees without blocking the main UI thread. It allows React to interrupt a long-running render to handle a high-priority event. i.e, When you enabled concurrent Mode, React will keep an eye on other tasks that need to be done, and if there's something with a higher priority it will pause what it is currently rendering and let the other task finish first. You can enable this in two ways,

```
// 1. Part of an app by wrapping with ConcurrentMode
<React.unstable_ConcurrentMode>
 <Something />
</React.unstable_ConcurrentMode>;

// 2. Whole app using createRoot
ReactDOM.unstable_createRoot(domNode).render(<App />);
```



[↑ Back to Top](#)

### 231. What is the difference between async mode and concurrent mode?

Both refers the same thing. Previously concurrent Mode being referred to as "Async Mode" by React team. The name has been changed to highlight React's ability to perform work on different priority levels. So it avoids the confusion from other approaches to Async Rendering.

[↑ Back to Top](#)

### 232. Can I use javascript urls in react16.9?

Yes, you can use javascript: URLs but it will log a warning in the console. Because URLs starting with javascript: are dangerous by including unsanitized output in a tag like `<a href>` and create a security hole.

```
const companyProfile = {
 website: "javascript: alert('Your website is hacked')",
};
// It will log a warning
More details;
```

Remember that the future versions will throw an error for javascript URLs.

[↑ Back to Top](#)

### 233. What is the purpose of eslint plugin for hooks?

The ESLint plugin enforces rules of Hooks to avoid bugs. It assumes that any function starting with "use" and a capital letter right after it is a Hook. In particular, the rule enforces that,

- Calls to Hooks are either inside a PascalCase function (assumed to be a component) or another useSomething function (assumed to be a custom Hook).
- Hooks are called in the same order on every render.

[↑ Back to Top](#)

### 234. What is the difference between Imperative and Declarative in React?

Imagine a simple UI component, such as a "Like" button. When you tap it, it turns blue if it was previously grey, and grey if it was previously blue.

The imperative way of doing this would be:

```
if (user.likes()) {
 if (hasBlue()) {
 removeBlue();
 addGrey();
 } else {
 removeGrey();
 addBlue();
 }
}
```

Basically, you have to check what is currently on the screen and handle all the changes necessary to redraw it with the current state, including undoing the changes from the previous state. You can imagine how complex this could be in a real-world scenario.

In contrast, the declarative approach would be:

```
if (this.state.liked) {
 return <blueLike />;
} else {
 return <greyLike />;
}
```

Because the declarative approach separates concerns, this part of it only needs to handle how the UI should look in a specific state, and is therefore much simpler to understand.

[↑ Back to Top](#)

### 235. What are the benefits of using TypeScript with ReactJS?

Below are some of the benefits of using TypeScript with ReactJS,

- i. It is possible to use latest JavaScript features
- ii. Use of interfaces for complex type definitions
- iii. IDEs such as VS Code was made for TypeScript
- iv. Avoid bugs with the ease of readability and Validation

[↑ Back to Top](#)

## 236. How do you make sure that user remains authenticated on page refresh while using Context API State Management?

When a user logs in and reload, to persist the state generally we add the load user action in the useEffect hooks in the main App.js. While using Redux, loadUser action can be easily accessed.

App.js

```
import { loadUser } from "../actions/auth";
store.dispatch(loadUser());
```

- o But while using **Context API**, to access context in App.js, wrap the AuthState in index.js so that App.js can access the auth context. Now whenever the page reloads, no matter what route you are on, the user will be authenticated as **loadUser** action will be triggered on each re-render.

index.js

```
import React from "react";
import ReactDOM from "react-dom";
import App from "./App";
import AuthState from "../context/auth/AuthState";

ReactDOM.render(
 <React.StrictMode>
 <AuthState>
 <App />
 </AuthState>
 </React.StrictMode>,
 document.getElementById("root")
);
```

App.js

```
const authContext = useContext(AuthContext);

const { loadUser } = authContext;

useEffect(() => {
 loadUser();
}, []);
```

loadUser

```
const loadUser = async () => {
 const token = sessionStorage.getItem("token");

 if (!token) {
 dispatch({
 type: ERROR,
 });
 }
 setAuthToken(token);

 try {
 const res = await axios("/api/auth");

 dispatch({
 type: USER_LOADED,
 payload: res.data.data,
 });
 } catch (err) {
 console.error(err);
 }
};
```

```
}
};
```

[↑ Back to Top](#)

### 237. What are the benefits of new JSX transform?

There are three major benefits of new JSX transform,

- It is possible to use JSX without importing React packages
- The compiled output might improve the bundle size in a small amount
- The future improvements provides the flexibility to reduce the number of concepts to learn React.

[↑ Back to Top](#)

### 238. How is the new JSX transform different from old transform??

The new JSX transform doesn't require React to be in scope. i.e, You don't need to import React package for simple scenarios.

Let's take an example to look at the main differences between the old and the new transform,

**Old Transform:**

```
import React from "react";

function App() {
 return <h1>Good morning!!</h1>;
}
```



Now JSX transform convert the above code into regular JavaScript as below,

```
import React from "react";

function App() {
 return React.createElement("h1", null, "Good morning!!");
}
```



**New Transform:**

The new JSX transform doesn't require any React imports

```
function App() {
 return <h1>Good morning!!</h1>;
}
```



Under the hood JSX transform compiles to below code

```
import { jsx as _jsx } from "react/jsx-runtime";

function App() {
 return _jsx("h1", { children: "Good morning!!" });
}
```



**Note:** You still need to import React to use Hooks.

[↑ Back to Top](#)

### 239. What are React Server components?

React Server Component is a way to write React component that gets rendered in the server-side with the purpose of improving React app performance. These components allow us to load components from the backend.

**Note:** React Server Components is still under development and not recommended for production yet.

[↑ Back to Top](#)

### 240. What is prop drilling?

Prop Drilling is the process by which you pass data from one component of the React Component tree to another by going through other components that do not need the data but only help in passing it around.

[↑ Back to Top](#)

#### 241. What is the difference between useState and useRef hook?

- useState causes components to re-render after state updates whereas useRef doesn't cause a component to re-render when the value or state changes. Essentially, useRef is like a "box" that can hold a mutable value in its ( `.current` ) property.
- useState allows us to update the state inside components. While useRef allows referencing DOM elements and tracking values.

[↑ Back to Top](#)

#### 242. What is a wrapper component?

A wrapper in React is a component that wraps or surrounds another component or group of components. It can be used for a variety of purposes such as adding additional functionality, styling, or layout to the wrapped components.

For example, consider a simple component that displays a message:

```
const Message = ({ text }) => {
 return <p>{text}</p>;
};
```

We can create a wrapper component that will add a border to the message component:

```
const MessageWrapper = (props) => {
 return (
 <div style={{ border: "1px solid black" }}>
 <Message {...props} />
 </div>
);
};
```

Now we can use the MessageWrapper component instead of the Message component and the message will be displayed with a border:

```
<MessageWrapper text="Hello World" />
```

Wrapper component can also accept its own props and pass them down to the wrapped component, for example, we can create a wrapper component that will add a title to the message component:

```
const MessageWrapperWithTitle = ({ title, ...props }) => {
 return (
 <div>
 <h3>{title}</h3>
 <Message {...props} />
 </div>
);
};
```

Now we can use the MessageWrapperWithTitle component and pass title props:

```
<MessageWrapperWithTitle title="My Message" text="Hello World" />
```

This way, the wrapper component can add additional functionality, styling, or layout to the wrapped component while keeping the wrapped component simple and reusable.

[↑ Back to Top](#)

#### 243. What are the differences between useEffect and useLayoutEffect hooks?

useEffect and useLayoutEffect are both React hooks that can be used to synchronize a component with an external system, such as a browser API or a third-party library. However, there are some key differences between the two:

- Timing: useEffect runs after the browser has finished painting, while useLayoutEffect runs synchronously before the browser paints. This means that useLayoutEffect can be used to measure and update layout in a way that feels more synchronous to the user.



- Browser Paint: useEffect allows browser to paint the changes before running the effect, hence it may cause some visual flicker. useEffectLayoutEffect synchronously runs the effect before browser paints and hence it will avoid visual flicker.
- Execution Order: The order in which multiple useEffect hooks are executed is determined by React and may not be predictable. However, the order in which multiple useEffectLayoutEffect hooks are executed is determined by the order in which they were called.
- Error handling: useEffect has a built-in mechanism for handling errors that occur during the execution of the effect, so that it does not crash the entire application. useEffectLayoutEffect does not have this mechanism, and errors that occur during the execution of the effect will crash the entire application.

In general, it's recommended to use useEffect as much as possible, because it is more performant and less prone to errors. useEffectLayoutEffect should only be used when you need to measure or update layout, and you can't achieve the same result using useEffect.

[📄 Back to Top](#)

## 244. What are the differences between Functional and Class Components?

There are two different ways to create components in ReactJS. The main differences are listed down as below,

### 1. Syntax:

The class components uses ES6 classes to create the components. It uses `render` function to display the HTML content in the webpage.

The syntax for class component looks like as below.

```
class App extends React.Component {
 render() {
 return <h1>This is a class component</h1>;
 }
}
```

**Note:** The **Pascal Case** is the recommended approach to provide naming to a component.

Functional component has been improved over the years with some added features like Hooks. Here is a syntax for functional component.

```
function App() {
 return (
 <div className="App">
 <h1>Hello, I'm a function component</h1>
 </div>
);
}
```

### 2. State:

State contains information or data about a component which may change over time.

In class component, you can update the state when a user interacts with it or server updates the data using the `setState()` method. The initial state is going to be assigned in the `constructor()` method using the `this.state` object and it is possible to assign different data types such as string, boolean, numbers, etc.

A simple example showing how we use the `setState()` and `constructor()`:

```
class App extends Component {
 constructor() {
 super();
 this.state = {
 message: "This is a class component",
 };
 }
 updateMessage() {
 this.setState({
 message: "Updating the class component",
 });
 }
 render() {
 return (
 <>
 <h1>{this.state.message}</h1>
 <button
 onClick={() => {

```

```

 this.updateMessage();
 }>
 Click!!
</button>
</>
);
}
}

```

You didn't use state in functional components because it was only supported in class components. But over the years hooks have been implemented in functional components which enables to use state too.

The `useState()` hook can be used to implement state in functional components. It returns an array with two items: the first item is current state and the next one is a function (`setState`) that updates the value of the current state.

Let's see an example to demonstrate the state in functional components,

```

function App() {
 const [message, setMessage] = useState("This is a functional component");
 const updateMessage = () => {
 setMessage("Updating the functional component");
 };
 return (
 <div className="App">
 <h1>{message}</h1>
 <button onClick={updateMessage}>Click me!!</button>
 </div>
);
}

```

### 3. Props:

Props are referred to as "properties". The props are passed into React component just like arguments passed to a function. In other words, they are similar to HTML attributes.

The props are accessible in child class component using `this.props` as shown in below example,

```

class Child extends React.Component {
 render() {
 return (
 <h1>
 {" "}
 This is a functional component and component name is {
 this.props.name
 }{" "}
 </h1>
);
 }
}

class Parent extends React.Component {
 render() {
 return (
 <div className="Parent">
 <Child name="First child component" />
 <Child name="Second child component" />
 </div>
);
 }
}

```

Props in functional components are similar to that of the class components but the difference is the absence of 'this' keyword.

```

function Child(props) {
 return (
 <h1>
 This is a child component and the component name is{props.name}
 </h1>
);
}

function Parent() {
 return (

```

```

<div className="Parent">
 <Child name="First child component" />
 <Child name="Second child component" />
</div>
);
}

```

[↑ Back to Top](#)

## 245. What is strict mode in React?

`React.StrictMode` is a useful component for highlighting potential problems in an application. Just like `<Fragment>`, `<StrictMode>` does not render any extra DOM elements. It activates additional checks and warnings for its descendants. These checks apply for *development mode* only.

```

import { StrictMode } from "react";

function App() {
 return (
 <div>
 <Header />
 <StrictMode>
 <div>
 <ComponentOne />
 <ComponentTwo />
 </div>
 </StrictMode>
 <Header />
 </div>
);
}

```

In the example above, the *strict mode* checks apply to `<ComponentOne>` and `<ComponentTwo>` components only. i.e., Part of the application only.

**Note:** Frameworks such as NextJS has this flag enabled by default.

[↑ Back to Top](#)

## 246. What is the benefit of strict mode?

It will be helpful in the below cases,

- i. To find the bugs caused by impure rendering where the components will re-render twice.
- ii. To find the bugs caused by missing cleanup of effects where the components will re-run effects one more extra time.
- iii. Identifying components with **unsafe lifecycle methods**.
- iv. Warning about **legacy string ref** API usage.
- v. Detecting unexpected **side effects**.
- vi. Detecting **legacy context** API.
- vii. Warning about deprecated **findDOMNode** usage

[↑ Back to Top](#)

## 247. Why does strict mode render twice in React?

`StrictMode` renders components twice in development mode(not production) in order to detect any problems with your code and warn you about those problems. This is used to detect accidental side effects in the render phase. If you used `create-react-app` development tool then it automatically enables `StrictMode` by default.

```

const root = createRoot(document.getElementById("root"));
root.render(
 <StrictMode>
 <App />
 </StrictMode>
);

```

If you want to disable this behavior then you can simply remove `strict` mode.

```

const root = createRoot(document.getElementById("root"));

```

```
root.render(<App />);
```

To detect side effects the following functions are invoked twice:

- Function component bodies, excluding the code inside event handlers.
- Functions passed to `useState`, `useMemo`, or `useReducer` (any other Hook)
- Class component's `constructor`, `render`, and `shouldComponentUpdate` methods
- Class component static `getDerivedStateFromProps` method
- State updater functions

[↑ Back to Top](#)

## 248. What are the rules of JSX?

The below 3 rules needs to be followed while using JSX in a react application.

- Return a single root element:** If you are returning multiple elements from a component, wrap them in a single parent element. Otherwise you will receive the below error in your browser console.

```
html Adjacent JSX elements must be wrapped in an enclosing tag.
```

- All the tags needs to be closed:** Unlike HTML, all tags needs to closed explicitly with in JSX. This rule applies for self-closing tags(like `hr`, `br` and `img` tags) as well.
  - Use camelCase naming:** It is suggested to use camelCase naming for attributes in JSX. For example, the common attributes of HTML elements such as `class`, `tabindex` will be used as `className` and `tabIndex`.
- Note:** There is an exception for `aria-*` and `data-*` attributes which should be lower cased all the time.

[↑ Back to Top](#)

## 249. What is the reason behind multiple JSX tags to be wrapped?

Behind the scenes, JSX is transformed into plain javascript objects. It is not possible to return two or more objects from a function without wrapping into an array. This is the reason you can't simply return two or more JSX tags from a function without wrapping them into a single parent tag or a Fragment.

[↑ Back to Top](#)

## 250. How do you prevent mutating array variables?

The preexisting variables outside of the function scope including state, props and context leads to a mutation and they result in unpredictable bugs during the rendering stage. The below points should be taken care while working with arrays variables.

- You need to take copy of the original array and perform array operations on it for the rendering purpose. This is called local mutation.
- Avoid triggering mutation methods such as `push`, `pop`, `sort` and `reverse` methods on original array. It is safe to use `filter`, `map` and `slice` method because they create a new array.

[↑ Back to Top](#)

## 251. What are capture phase events?

The `onClickCapture` React event is helpful to catch all the events of child elements irrespective of event propagation logic or even if the events propagation stopped. This is useful if you need to log every click events for analytics purpose.

For example, the below code triggers the click event of parent first followed by second level child event though leaf child button elements stops the propagation.

```
<div onClickCapture={() => alert("parent")}>
 <div onClickCapture={() => alert("child")}>
 <button onClick={(e) => e.stopPropagation()} />
 <button onClick={(e) => e.stopPropagation()} />
 </div>
</div>
```

The event propagation for the above code snippet happens in the following order:

- It travels downwards in the DOM tree by calling all `onClickCapture` event handlers.
- It executes `onClick` event handler on the target element.
- It travels upwards in the DOM tree by call all `onClick` event handlers above to it.

## 252. How does React updates screen in an application?

React updates UI in three steps,

- i. **Triggering or initiating a render:** The component is going to triggered for render in two ways.
  - a. **Initial render:** When the app starts, you can trigger the initial render by calling `createRoot` with the target DOM node followed by invoking component's `render` method. For example, the following code snippet renders `App` component on root DOM node.

```
import { createRoot } from "react-dom/client";

const root = createRoot(document.getElementById("root"));
root.render(<App />);
```
  - b. **Re-render when the state updated:** When you update the component state using the state setter function, the componen't state automatically queues for a render.
- ii. **Rendering components:** After triggering a render, React will call your components to display them on the screen. React will call the root component for initial render and call the function component whose state update triggered the render. This is a recursive process for all nested components of the target component.
- iii. **Commit changes to DOM:** After calling components, React will modify the DOM for initial render using `appendChild()` DOM API and apply minimal necessary DOM updates for re-renders based on differences between rerenders.

[🔙 Back to Top](#)

## 253. How does React batch multiple state updates?

React prevents component from re-rendering for each and every state update by grouping multiple state updates within an event handler. This strategy improves the application performance and this process known as **batching**. The older version of React only supported batching for browser events whereas React18 supported for asynchronous actions, timeouts and intervals along with native events. This improved version of batching is called **automatic batching**.

Let's demonstrate this automatic batching feature with a below example.

```
import { useState } from "react";

export default function BatchingState() {
 const [count, setCount] = useState(0);
 const [message, setMessage] = useState("batching");

 console.log("Application Rendered");

 const handleUsers = () => {
 fetch("https://jsonplaceholder.typicode.com/users/1").then(() => {
 // Automatic Batching re-render only once
 setCount(count + 1);
 setMessage("users fetched");
 });
 };

 return (
 <>
 <h1>{count}</h1>
 <button onClick={handleAsyncFetch}>Click Me!</button>
 </>
);
}
```

The preceding code updated two state variables with in an event handler. However, React will perform automatic batching feature and the component will be re-rendered only once for better performance.

[🔙 Back to Top](#)

## 254. Is it possible to prevent automatic batching?

Yes, it is possible to prevent automatic batching default behavior. There might be cases where you need to re-render your component after each state update or updating one state depends on another state variable. Considering this situation, React introduced `flushSync` method from `react-dom` API for the usecases where you need to flush state updates to DOM immediately.

The usage of `flushSync` method within an `onClick` event handler will be looking like as below,

```
import { flushSync } from "react-dom";

const handleClick = () => {
 flushSync(() => {
 setClicked(!clicked); //Component will create a re-render here
 });

 setCount(count + 1); // Component will create a re-render again here
};
```

In the above click handler, React will update DOM at first using flushSync and second time updates DOM because of the counter setter function by avoiding automatic batching.

[↑ Back to Top](#)

## 255. What is React hydration?

React hydration is used to add client-side JavaScript interactivity to pre-rendered static HTML generated by the server. It is used only for server-side rendering(SSR) to enhance the initial rendering time and make it SEO friendly application. This hydration acts as a bridge to reduce the gap between server side and client-side rendering.

After the page loaded with generated static HTML, React will add application state and interactivity by attaching all event handlers for the respective elements. Let's demonstrate this with an example.

Consider that React DOM API(using `renderToString`) generated HTML for `<App>` component which contains `<button>` element to increment the counter.

```
import {useState} from 'react';
import { renderToString } from 'react-dom/server';

export default function App() {
 const [count, setCount] = React.useState(0);

 return (
 <h1>Counter</h1>
 <button onClick={() => setCount(prevCount => prevCount + 1)}>
 {count} times
 </button>
);
}

const html = renderToString(<App />);
```

The above code generates the below HTML with a header text and button component without any interactivity.

```
<h1>Counter</h1>
<button>
 <!-- -->0<!-- -->
 times
</button>
```

At this stage `hydrateRoot` API can be used to perform hydration by attaching `onClick` event handler.

```
import { hydrateRoot } from "react-dom/client";
import App from "./App.js";

hydrateRoot(document.getElementById("root"), <App />);
```

After this step, you are able to run React application on server-side and hydrating the javascript bundle on client-side for smooth user experience and SEO purposes.

[↑ Back to Top](#)

## 256. How do you update objects inside state?

You cannot update the objects which exists in the state directly. Instead, you should create a fresh new object (or copy from the existing object) and update the latest state using the newly created object. Eventhough JavaScript objects are mutable, you need to treat objects inside state as read-only while updating the state.

Let's see this comparison with an example. The issue with regular object mutation approach can be described by updating the user details fields of `Profile` component. The properties of `Profile` component such as `firstName`, `lastName` and `age` details mutated in an event handler as shown below.

```
import { useState } from "react";

export default function Profile() {
 const [user, setUser] = useState({
 firstName: "John",
 lastName: "Abraham",
 age: 30,
 });

 function handleFirstNameChange(e) {
 user.firstName = e.target.value;
 }

 function handleLastNameChange(e) {
 user.lastName = e.target.value;
 }

 function handleAgeChange(e) {
 user.age = e.target.value;
 }

 return (
 <>
 <label>
 First name:
 <input value={user.firstName} onChange={handleFirstNameChange} />
 </label>
 <label>
 Last name:
 <input value={user.lastName} onChange={handleLastNameChange} />
 </label>
 <label>
 Age:
 <input value={user.age} onChange={handleAgeChange} />
 </label>
 <p>
 Profile:
 {person.firstName} {person.lastName} ({person.age})
 </p>
 </>
);
}
```

Once you run the application with above user profile component, you can observe that user profile details won't be update upon entering the input fields. This issue can be fixed by creating a new copy of object which includes existing properties through spread syntax(...obj) and add changed values in a single event handler itself as shown below.

```
handleProfileChange(e) {
 setUser({
 ...user,
 [e.target.name]: e.target.value
 });
}
```

The above event handler is concise instead of maintaining separate event handler for each field. Now, UI displays the updated field values as expected without an issue.

[🔼 Back to Top](#)

## 257. How do you update nested objects inside state?

You cannot simply use spread syntax for all kinds of objects inside state. Because spread syntax is shallow and it copies properties for one level deep only. If the object has nested object structure, UI doesn't work as expected with regular JavaScript nested property mutation. Let's demonstrate this behavior with an example of User object which has address nested object inside of it.

```
const user = {
 name: "John",
 age: 32,
 address: {
```

```
country: "Singapore",
postalCode: 440004,
},
};
```

If you try to update the country nested field in a regular javascript fashion(as shown below) then user profile screen won't be updated with latest value.

```
user.address.country = "Germany";
```

This issue can be fixed by flattening all the fields into a top-level object or create a new object for each nested object and point it to its parent object. In this example, first you need to create copy of address object and update it with the latest value. Later, the address object should be linked to parent user object something like below.

```
setUser({
 ...user,
 address: {
 ...user.address,
 country: "Germany",
 },
});
```

This approach is bit verbose and not easy for deep hierarchical state updates. But this workaround can be used for few levels of nested objects without much hassle.

[↑ Back to Top](#)

## 258. How do you update arrays inside state?

Eventhough arrays in JavaScript are mutable in nature, you need to treat them as immutable while storing them in a state. That means, similar to objects, the arrays cannot be updated directly inside state. Instead, you need to create a copy of the existing array and then set the state to use newly copied array.

To ensure that arrays are not mutated, the mutation operations like direct assignment(`arr[1]='one'`), `push`, `pop`, `shift`, `unshift`, `splice` etc methods should be avoided on original array. Instead, you can create a copy of existing array with help of array operations such as `filter`, `map`, `slice`, `spread syntax` etc.

For example, the below push operation doesn't add the new todo to the total todo's list in an event handler.

```
onClick = {
 todos.push({
 id: id+1,
 name: name
 })
}
```

This issue is fixed by replacing push operation with spread syntax where it will create a new array and the UI updated with new todo.

```
onClick = {
 [
 ...todos,
 { id: id+1, name: name }
]
}
```

[↑ Back to Top](#)

## 259. How do you use immer library for state updates?

Immer library enforces the immutability of state based on **copy-on-write** mechanism. It uses JavaScript proxy to keep track of updates to immutable states. Immer has 3 main states as below,

- i. **Current state:** It refers to actual state
- ii. **Draft state:** All new changes will be applied to this state. In this state, draft is just a proxy of the current state.
- iii. **Next state:** It is formed after all mutations applied to the draft state

Immer can be used by following below instructions,



- i. Install the dependency using `npm install use-immer` command
- ii. Replace `useState` hook with `useImmer` hook by importing at the top
- iii. The setter function of `useImmer` hook can be used to update the state.

For example, the mutation syntax of immer library simplifies the nested address object of user state as follows,

```
import { useImmer } from "use-immer";
const [user, setUser] = useImmer({
 name: "John",
 age: 32,
 address: {
 country: "Singapore",
 postalCode: 440004,
 },
});

//Update user details upon any event
setUser((draft) => {
 draft.address.country = "Germany";
});
```



The preceding code enables you to update nested objects with a concise mutation syntax.

[↑ Back to Top](#)

## 260. What are the benefits of preventing the direct state mutations?

[↑ Back to Top](#)

## 261. What are the preferred and non-preferred array operations for updating the state?

The below table represent preferred and non-preferred array operations for updating the component state.

Action	Preferred	Non-preferred
Adding	concat, [...arr]	push, unshift
Removing	filter, slice	pop, shift, splice
Replacing	map	splice, arr[i] = someValue
sorting	copying to new array	reverse, sort

If you use Immer library then you can able to use all array methods without any problem.

[↑ Back to Top](#)

## 262. What will happen by defining nested function components?

Technically it is possible to write nested function components but it is not suggested to write nested function definitions. Because it leads to unexpected bugs and performance issues.

[↑ Back to Top](#)

## 263. Can I use keys for non-list items?

Keys are primarily used for rendering list items but they are not just for list items. You can also use them React to distinguish components. By default, React uses order of the components in

[↑ Back to Top](#)

## 264. What are the guidelines to be followed for writing reducers?

There are two guidelines to be taken care while writing reducers in your code.

- i. Reducers must be pure without mutating the state. That means, same input always returns the same output. These reducers run during rendering time similar to state updater functions. So these functions should not send any requests, schedule time outs and any other side effects.

- ii. Each action should describe a single user interaction even though there are multiple changes applied to data. For example, if you "reset" registration form which has many user input fields managed by a reducer, it is suggested to send one "reset" action instead of creating separate action for each field. The proper ordering of actions should reflect the user interactions in the browser and it helps a lot for debugging purpose.

[📄 Back to Top](#)

## 265. How does ReactJS work behind the scenes?

ReactJS is a powerful JavaScript library for building user interfaces. While it appears simple on the surface, React performs a lot of complex operations behind the scenes to efficiently update the UI. Here's an overview of how it works internally:

### 1. Virtual DOM & Component Rendering

React doesn't manipulate the real DOM directly. Instead, it uses a **Virtual DOM** — a lightweight JavaScript representation of the UI.

When a component renders (e.g., `<App />`):

- React **executes the component function** (e.g., `App()` ).
- Hooks like `useState` are registered and tracked in order.
- React builds a **Virtual DOM tree** from the returned JSX.
- This virtual DOM is a **plain JS object** that describes the desired UI.

This process ensures fast and efficient rendering before React decides how to update the real DOM.

### 2. React Fiber Architecture

React's core engine is called **Fiber**, introduced in React 16. Fiber is a reimplementation of the React reconciliation algorithm with the following capabilities:

- Breaks rendering work into **units of work** (fiber nodes).
- Enables **interruptible rendering** (important for responsiveness).
- Supports **priority scheduling** and **concurrent rendering**.

Each Fiber node represents a component and stores:

- The component type (function/class).
- Props, state, and effects.
- Links to parent, child, and sibling fibers.

### 3. Reconciliation (Diffing Algorithm)

When state or props change:

- React re-executes the component to produce a new virtual DOM.
- It **compares the new virtual DOM to the previous one** using an efficient diffing algorithm.
- React determines the **minimal set of DOM changes** required.

This process is known as **reconciliation**.

### 4. Commit Phase (Real DOM Updates)

Once reconciliation is done:

- React enters the **commit phase**.
- It applies calculated changes to the **real DOM**.
- It also runs side effects like `useEffect` or `useLayoutEffect` .

This is the only time React interacts directly with the browser DOM.

### 5. Hooks and State Management

With Hooks (like `useState` , `useEffect` ):

- React keeps an **internal list of hooks per component**.
- Hooks are identified by their order in the function.
- When state updates occur, React re-renders the component and re-runs the hooks in the same order.

### 6. React Scheduler

React uses an internal **Scheduler** to control how updates are prioritized:

- Urgent tasks like clicks and inputs are processed immediately.
- Non-urgent tasks (like data fetching) can be delayed or paused.
- This improves responsiveness and allows for **time slicing** in Concurrent Mode.

[↑ Back to Top](#)

## 266. How is `useReducer` Different from `useState` ?

There are notable differences between `useState` and `useReducer` hooks.

Feature	<code>useState</code>	<code>useReducer</code>
State complexity	Simple (one variable or flat object)	Complex, multi-part or deeply nested
Update style	Direct (e.g. <code>setState(x)</code> )	Through actions (e.g. <code>dispatch({})</code> )
Update logic	In component	In reducer function
Reusability & testing	Less reusable	Highly reusable & testable

[↑ Back to Top](#)

## 267. What is `useContext`? What are the steps to follow for `useContext`?

The `useContext` hook is a built-in React Hook that lets you access the value of a context inside a functional component without needing to wrap it in a `<Context.Consumer>` component.

It helps you **avoid prop drilling** (passing props through multiple levels) by allowing components to access shared data like themes, authentication status, or user preferences.

The usage of `useContext` involves three main steps:

### Step 1 : Create the Context

Use `React.createContext()` to create a context object.

```
import React, { createContext } from 'react';

const ThemeContext = createContext(); // default value optional
```

You typically export this so other components can import it.

### Step 2: Provide the Context Value

Wrap your component tree (or a part of it) with the `Context.Provider` and pass a `value` prop.

```
function App() {
 return (
 <ThemeContext.Provider value="dark">
 <MyComponent />
 </ThemeContext.Provider>
);
}
```

Now any component inside `<ThemeContext.Provider>` can access the context value.

### Step 3: Consume the Context with `**useContext**`

In any functional component **inside the Provider**, use the `useContext` hook:

```
import { useContext } from 'react';
function MyComponent() {
 const theme = useContext(ThemeContext); // theme = "dark"
 return <p>Current Theme: {theme}</p>;
}
```

[↑ Back to Top](#)

## 268. What are the use cases of useContext hook?

The `useContext` hook in React is used to share data across components without having to pass props manually through each level. Here are some common and effective use cases:

### i. Theme Customization

`useContext` can be used to manage application-wide themes, such as light and dark modes, ensuring consistent styling and enabling user-driven customization.

### ii. Localization and Internationalization

It supports localization by providing translated strings or locale-specific content to components, adapting the application for users in different regions.

### iii. User Authentication and Session Management

`useContext` allows global access to authentication status and user data. This enables conditional rendering of components and helps manage protected routes or user-specific UI elements.

### iv. Shared Modal or Sidebar Visibility

It's ideal for managing the visibility of shared UI components like modals, drawers, or sidebars, especially when their state needs to be controlled from various parts of the app.

### v. Combining with `useReducer` for Global State Management

When combined with `useReducer`, `useContext` becomes a powerful tool for managing more complex global state logic. This pattern helps maintain cleaner, scalable state logic without introducing external libraries like Redux. Some of the common use cases of `useContext` are listed below,

[↑ Back to Top](#)

## 269. When to use client and server components?

You can efficiently build nextjs application if you are aware about which part of the application needs to use client components and which other parts needs to use server components. The common cases of both client and server components are listed below:

### Client components:

- Whenever your need to add interactivity and event listeners such as `onClick()`, `onChange()`, etc to the pages
- If you need to use State and Lifecycle Effects like `useState()`, `useReducer()`, `useEffect()` etc.
- If there is a requirement to use browser-only APIs.
- If you need to implement custom hooks that depend on state, effects, or browser-only APIs.
- There are React Class components in the pages.

### Server components:

- If the component logic is about data fetching.
- If you need to access backend resources directly.
- When you need to keep sensitive information((access tokens, API keys, etc) ) on the server.
- If you want reduce client-side JavaScript and placing large dependencies on the server.

[↑ Back to Top](#)

## 270. What are the differences between page router and app router in nextjs?

[↑ Back to Top](#)

## 271. Can you describe the useMemo() Hook?

The `useMemo()` Hook in React is used to **optimize performance** by **memoizing the result of expensive calculations**. It ensures that a function is **only re-executed when its dependencies change**, preventing unnecessary computations on every re-render.

### Syntax

```
const memoizedValue = useMemo(() => computeExpensiveValue(arg), [dependencies]);
```

- **computeExpensiveValue** :  
A function that returns the computed result.
- **dependencies** :  
An array of values that, when changed, will cause the memoized function to re-run.

If the dependencies haven't changed since the last render, React returns the **cached result** instead of re-running the function.

Let's explain the usage of `useMemo` hook with an example of user search and its respective filtered users list.

### Example: Memoizing a Filtered List

```
import React, { useState, useMemo } from 'react';

const users = [
 { id: 1, name: 'Sudheer' },
 { id: 2, name: 'Brendon' },
 { id: 3, name: 'Charlie' },
 { id: 4, name: 'Dary' },
 { id: 5, name: 'Eden' }
];

export default function UserSearch({ users }) {
 const [searchTerm, setSearchTerm] = useState('');
 const [counter, setCounter] = useState(0);

 // Memoize the filtered user list based on the search term
 const filteredUsers = useMemo(() => {
 console.log("Filtering users...");
 return users.filter(user =>
 user.name.toLowerCase().includes(searchTerm.toLowerCase())
);
 }, [searchTerm]);

 return (
 <div>
 <h2>Counter: {counter}</h2>
 <button onClick={() => setCounter(prev => prev + 1)}>Increment Counter</button>

 <h2>Search Users</h2>
 <input
 type="text"
 value={searchTerm}
 onChange={(e) => setSearchTerm(e.target.value)}
 placeholder="Enter name"
 />

 {filteredUsers.map(user => (
 <li key={user.id}>{user.name}
))}

 </div>
);
}
```

In the above example:

- The filteredUsers list is only recomputed when searchTerm changes.
- Pressing the "Increment Counter" button does not trigger the filtering logic again, as it's not a dependency.
- The console will only log "Filtering users..." when the search term updates.

[↑ Back to Top](#)

## 272. Can Hooks be used in class components?

No, Hooks cannot be used inside class components. They are specially designed for function components. This is because hooks depend on the sequence in which they are called during a component's render, something that's only guaranteed in functional components. However, both class and function components can coexist in the same application.

[↑ Back to Top](#)

## 273. What is an updater function? Should an updater function be used in all cases?

An **updater function** is a form of `setState` where you pass a **function** instead of a direct value. This function receives the **previous state** as an argument and returns the **next state**.

The updater function expression looks like below,

```
setCount(prevCount => prevCount + 1); // Safe and predictable
```

Here, `prevCount => prevCount + 1` is the updater function.

In the React community, there's often a recommendation to use updater functions when updating state that depends on its previous value. This helps prevent unexpected behaviors that can arise from working with outdated or "stale" state.

While using an updater function is a good habit, it's not always necessary. In most cases, React batches updates and ensures that the state is up-to-date at the beginning of the event handler, so you typically don't run into stale state issues during a single synchronous event. However, if you're doing multiple updates to the same state variable within a single handler, using the updater form ensures that each update correctly uses the latest state value, rather than a potentially outdated one.

#### Example: Multiple Updates in One Handler

```
function handleCount() {
 setCounter(a => a + 1);
 setCounter(a => a + 1);
 setCounter(a => a + 1);
}
```

In this example, `a => a + 1` is an **updater function**. React queues these updater functions and applies them sequentially, each using the most recent state value. As a result, the counter will correctly increment by 3.

In many cases, such as setting state based on user input or assigning static values, you don't need the updater function:

```
setName('Sudheer');
```

[↑ Back to Top](#)

### 274. Can `useState` take a function as an initial value?

Yes, `useState` can take a function as an initial value, and this is a useful feature in React called **lazy initialization**. This function is also known as **initializer function**.

When you call `useState(initialValue)`, you normally pass in a value directly:

```
const [count, setCount] = useState(0); // initial value is 0
```

But if calculating that initial value is expensive or involves logic, you can pass a function that returns the value:

```
const [count, setCount] = useState(() => {
 // This function only runs once - when the component first renders
 return expensiveComputation();
});
```

This function avoids doing heavy computation on every render. If you don't use this function form and invokes it directly, the function will run everytime the component renders and impact the performance. For example, the below usage is not recommended.

```
const [count, setCount] = useState(expensiveComputation());
```

[↑ Back to Top](#)

### 275. What types of values can `useState` hold?

The `useState` hook accepts different types of values.

- Primitives: `number`, `string`, `boolean`
- Arrays
- Objects
- Functions
- `null` OR `undefined`

But you need to be cautious with **reference types (objects/arrays)** because React compares old and new values **by reference**, so direct mutations won't trigger a re-render. For example, the correct and wrong ways of state updates as shown below,

```
user.name = "Sudheer"; //wrong way
setUser(prev => ({ ...prev, name: 'Sudheer' })); //correct way
```

[↑ Back to Top](#)

## 276. What happens if you call `useState` conditionally?

As per rules of React Hooks, hooks must be called unconditionally. For example, if you conditionally call it:

```
if (someCondition) {
 const [state, setState] = useState(0);
}
```



React will throw a runtime error because it **relies on the order of Hook calls**, and conditional logic breaks that order.

[↑ Back to Top](#)

## 277. Is `useState` Synchronous or Asynchronous?

The `useState` hook is synchronous, but state updates are asynchronous. When you call `useState()`, it runs synchronously and returns the state variable and setter function as tuple.

```
const [count, setCount] = useState(0);
```



This happens immediately during rendering. However, the state update function (`setState`) is asynchronous in the sense that it doesn't update the state immediately. React **batches** updates and applies them before the next render. You won't see the updated value immediately after calling `setState`.

Example:

```
const [count, setCount] = useState(0);

function handleClick() {
 setCount(count + 1);
 console.log(count); // ! Still logs the old value
}
```



The `> console.log(count)` prints the **old value**, because the update hasn't happened yet.

To see the updated state value, you can use `useEffect()` hook. It runs **after the component has re-rendered**. By the time `useEffect` runs:

- The component has been updated.
- The state contains the new value.

```
import React, { useState, useEffect } from 'react';

function Counter() {
 const [count, setCount] = useState(0);

 const handleClick = () => {
 setCount(count + 1);
 console.log('Clicked count (old):', count); // Old value
 };

 useEffect(() => {
 console.log('Updated count:', count); // New value
 }, [count]); // Only runs when `count` changes

 return <button onClick={handleClick}>Count: {count}</button>;
}
```

[↑ Back to Top](#)

## 278. Can you explain how `useState` works internally?

React's hooks, including `useState`, rely on some internal machinery that keeps track of state **per component** and **per hook call** during rendering. Here's a simplified explanation of the internal mechanics:

### 1. Hook List / Linked List

- React maintains a linked list or array of "hook states" for each component.
- When a component renders, React keeps track of which hook it is currently processing via a cursor/index.

- Each call to `useState()` corresponds to one "slot" in this list.

## 2. State Storage

- Each slot stores:
  - The current state value.
  - A queue of pending state updates.

## 3. Initial Render

- When the component first renders, React:
  - Creates a new slot for `useState` with the initial state (e.g., `0`).
  - Returns `[state, updaterFunction]`.

## 4. Updater Function

- The updater function ( `setCount` ) is a closure that, when called:
  - Enqueues a state update to React's internal queue.
  - Schedules a re-render of the component.

## 5. Re-render and State Update

- On the next render:
  - React processes all queued updates for each hook slot.
  - Updates the stored state value accordingly.
  - Returns the new state to the component.

## 6. Important: Hook Order

- Hooks must be called in the same order on every render so React can match hook calls to their internal slots.
- That's why you can't call hooks conditionally.

The pseudocode for internal implementation of `useState` looks like below,

```
let hookIndex = 0;
const hooks = [];

function useState(initialValue) {
 const currentIndex = hookIndex;

 if (!hooks[currentIndex]) {
 // First render: initialize state
 hooks[currentIndex] = {
 state: initialValue,
 queue: [],
 };
 }

 const hook = hooks[currentIndex];

 // Process queued updates
 hook.queue.forEach(update => {
 hook.state = update(hook.state);
 });
 hook.queue = [];

 // Define updater function
 function setState(action) {
 // action can be new state or function(state) => new state
 hook.queue.push(typeof action === 'function' ? action : () => action);
 scheduleRender(); // triggers React re-render
 }

 hookIndex++;
 return [hook.state, setState];
}
```

[📄 Back to Top](#)

### 279. What is `useReducer` ? Why do you use `useReducer`?



The `useReducer` hook is a React hook used to manage **complex state logic** inside **functional components**. It is conceptually similar to **Redux**. i.e, Instead of directly updating state like with `useState`, you **dispatch an action** to a **reducer function**, and the reducer returns the new state.

The `useReducer` hook takes three arguments:

```
const [state, dispatch] = useReducer(reducer, initialState, initFunction);
```

- o **\*\*reducer\*\***: A function `(state, action) => newState` that handles how state should change based on the action.
- o **\*\*initialState\*\***: The starting state.
- o **\*\*dispatch\*\***: A function you call to trigger an update by passing an action.

The `useReducer` hook is used when:

- o The **state is complex**, such as nested structures or multiple related values.
- o State updates depend on the **previous state** and **logic**.
- o You want to **separate state update logic** from UI code to make it cleaner and testable.
- o You're managing features like:
  - Forms
  - Wizards / Multi-step flows
  - Undo/Redo functionality
  - Shopping cart logic
  - Toggle & conditional UI logic

[↑ Back to Top](#)

## 280. How does `useReducer` works? Explain with an example

The `useReducer` hooks works similarly to Redux, where:

- o You define a **reducer function** to handle state transitions.
- o You dispatch actions to update the state.

**Counter Example with Increment, Decrement, and Reset:**

i. Reducer function:

Define a counter reducer function that takes the current state and an action object with a type, and returns a new state based on that type.

```
function counterReducer(state, action) {
 switch (action.type) {
 case 'increment':
 return { count: state.count + 1 };
 case 'decrement':
 return { count: state.count - 1 };
 case 'reset':
 return { count: 0 };
 default:
 return state;
 }
}
```

ii. Using `useReducer`: Invoke `useReducer` with above reducer function along with initial state. Thereafter, you can attach dispatch actions for respective button handlers.

```
import React, { useReducer } from 'react';

function Counter() {
 const initialState = { count: 0 };
 const [state, dispatch] = useReducer(counterReducer, initialState);

 return (
 <div style={{ textAlign: 'center' }}>
 <h2>Count: {state.count}</h2>
 <button onClick={() => dispatch({ type: 'increment' })}>Increment</button>
 <button onClick={() => dispatch({ type: 'decrement' })}>Decrement</button>
 <button onClick={() => dispatch({ type: 'reset' })}>Reset</button>
 </div>
);
}
```

```

 });
 }

 export default Counter;

```

Once the new state has been returned, React re-renders the component with the updated `state.count`.

[↑ Back to Top](#)

## 281. Can you combine `useReducer` with `useContext`?

Yes, it's common to combine `useReducer` with `useContext` to build a lightweight state management system similar to Redux:

```

const AppContext = React.createContext();

function AppProvider({ children }) {
 const [state, dispatch] = useReducer(reducer, initialState);
 return (
 <AppContext.Provider value={{ state, dispatch }}>
 {children}
 </AppContext.Provider>
);
}

```

[↑ Back to Top](#)

## 282. Can you dispatch multiple actions in a row with `useReducer`?

Yes, you can dispatch multiple actions in a row using `useReducer` but not directly in one call. You'd have to call dispatch multiple times or create a composite action in your reducer that performs multiple updates based on the action type.

**Example: Dispatching Multiple Actions** You can define a custom function with dispatching actions one by one.

```

function handleMultipleActions(dispatch) {
 dispatch({ type: 'increment' });
 dispatch({ type: 'increment' });
 dispatch({ type: 'reset' });
}

```

After that, you need to invoke it through event handler

```

<button onClick={() => handleMultipleActions(dispatch)}>
 Run Multiple Actions
</button>

```

**Note:** You can also define a custom action type If you want multiple state changes to be handled in one reducer call.

```

case 'increment_twice':
 return { count: state.count + 2 };

```

Then dispatch

```

dispatch({ type: 'increment_twice' });

```

[↑ Back to Top](#)

## 283. How does `useContext` works? Explain with an example

The `useContext` hook can be used for authentication state management across multiple components and pages in a React application.

Let's build a simple authentication flow with:

- Login and Logout buttons
- Global `AuthContext` to share state
- Components that can access and update auth status

### 1. Create the Auth Context:

You can define `AuthProvider` which holds and provides `user`, `login()`, and `logout()` via context.

```
// AuthContext.js
import React, { createContext, useContext, useState } from 'react';

const AuthContext = createContext();

export function AuthProvider({ children }) {
 const [user, setUser] = useState(null);

 const login = (username) => setUser({ name: username });
 const logout = () => setUser(null);

 return (
 <AuthContext.Provider value={{ user, login, logout }}>
 {children}
 </AuthContext.Provider>
);
}

// Custom hook for cleaner usage
export const useAuth = () => useContext(AuthContext);
```

## 2. Wrap Your App with the Provider:

Wrap the above created provider in main `App.js` file

```
// App.js
import React from 'react';
import { AuthProvider } from './AuthContext';
import HomePage from './HomePage';
import Dashboard from './Dashboard';

function App() {
 return (
 <AuthProvider>
 <HomePage />
 <Dashboard />
 </AuthProvider>
);
}

export default App;
```

## 3. Home page with login: Read or access user and login details through custom useAuth hook and use it inside home page.

```
// HomePage.js
import React from 'react';
import { useAuth } from './AuthContext';

function HomePage() {
 const { user, login } = useAuth();

 return (
 <div>
 <h1>Home</h1>
 {user ? (
 <p>Welcome back, {user.name}!</p>
) : (
 <button onClick={() => login('Alice')}>Login</button>
)}
 </div>
);
}

export default HomePage;
```

## 4. Dashboard with logout: Read or access user and logout details from `useAuth` custom hook and use it inside dashboard page.

```
// Dashboard.js
import React from 'react';
import { useAuth } from './AuthContext';

function Dashboard() {
```

```
const { user, logout } = useAuth();

if (!user) {
 return <p>Please login to view the dashboard.</p>;
}

return (
 <div>
 <h2>Dashboard</h2>
 <p>Logged in as: {user.name}</p>
 <button onClick={logout}>Logout</button>
 </div>
);
}

export default Dashboard;
```

[↑ Back to Top](#)

## 284. Can You Use Multiple Contexts in One Component?

Yes, it is possible. You can use multiple contexts inside the same component by calling `useContext` multiple times, once for each context.

It can be achieved with below steps,

- Create multiple contexts using `createContext()`.
- Wrap your component tree with multiple `<Provider>`s.
- Call `useContext()` separately for each context in the same component.

Example: Using `ThemeContext` and `UserContext` Together

```
import React, { createContext, useContext } from 'react';

// Step 1: Create two contexts
const ThemeContext = createContext();
const UserContext = createContext();

function Dashboard() {
 // Step 2: Use both contexts
 const theme = useContext(ThemeContext);
 const user = useContext(UserContext);

 return (
 <div style={{ background: theme === 'dark' ? '#333' : '#fff' }}>
 <h1>Welcome, {user.name}</h1>
 <p>Current theme: {theme}</p>
 </div>
);
}

// Step 3: Provide both contexts
function App() {
 return (
 <ThemeContext.Provider value="dark">
 <UserContext.Provider value={{ name: 'Sudheer' }}>
 <Dashboard />
 </UserContext.Provider>
 </ThemeContext.Provider>
);
}

export default App;
```

[↑ Back to Top](#)

## 285. What's a common pitfall when using `useContext` with objects?

A common pitfall when using `useContext` with objects is triggering unnecessary re-renders across all consuming components — even when only part of the context value changes.

When you provide an object as the context value, React compares the entire object reference. If the object changes (even slightly), React assumes the whole context has changed, and **all components using `useContext(MyContext)` will re-render**, regardless of whether they use the part that changed.

Example:

```
const MyContext = React.createContext();

function MyProvider({ children }) {
 const [user, setUser] = useState(null);
 const [theme, setTheme] = useState('light');

 // This causes all consumers to re-render on any state change
 const contextValue = { user, setUser, theme, setTheme };

 return (
 <MyContext.Provider value={contextValue}>
 {children}
 </MyContext.Provider>
);
}
```

In this case, a change in `theme` will also trigger a re-render in components that only care about `user`.

This issue can be fixed in two ways,

### 1. Split Contexts

Create separate contexts for unrelated pieces of state:

```
const UserContext = React.createContext();
const ThemeContext = React.createContext();
```

### 2. Memoize Context Value

Use `useMemo` to prevent unnecessary re-renders:

```
const contextValue = useMemo(() => ({ user, setUser, theme, setTheme }), [user, theme]);
```

However, this only helps if the object structure and dependencies are well controlled.

[↑ Back to Top](#)

## 286. What would the context value be for no matching provider?

When a component calls `useContext(SomeContext)` but **no matching** `<SomeContext.Provider>` is present higher up in the component tree, the default value passed to `React.createContext(defaultValue)` is returned.

```
const ThemeContext = React.createContext('light'); // 'light' is the default value

function ThemedComponent() {
 const theme = useContext(ThemeContext);
 return <div>Current theme: {theme}</div>;
}

// No ThemeContext.Provider anywhere in the tree
```

In this case, `theme` will be 'light'. It's the default value you provided when you created the context.

**Note:** If you don't specify a default value, the context value will be undefined when used without a provider:

```
const AuthContext = React.createContext(); // No default

function Profile() {
 const auth = useContext(AuthContext);
 // auth will be undefined if there's no AuthContext.Provider
}
```

[↑ Back to Top](#)

## 287. How do reactive dependencies in the useEffect dependency array affect its execution behavior?

The `useEffect` hook accepts an optional dependencies argument that accepts an array of reactive values. The **dependency array** determines **when** the effect runs. i.e, It makes `useEffect` *reactive* to changes in specified values.

## How Dependency Array Affects Behavior

### i. Empty Dependency Array: `[]`

```
useEffect(() => {
 // runs once after the initial render
}, []);
```

- Effect runs **only once** (like `componentDidMount`).
- Ignores all state/prop changes.

### ii. With Specific Dependencies: `[count, user]`

```
useEffect(() => {
 // runs after initial render
 // AND whenever `count` or `user` changes
}, [count, user]);
```

- Effect runs on **first render**, and
- Again **every time** any dependency value changes.

### iii. No Dependency Array (Omitted)

```
useEffect(() => {
 // runs after every render
});
```

- Effect runs after **every render**, regardless of what changed.
- Can lead to **performance issues** if not used carefully.

React uses shallow comparison of the dependencies. If any value has changed (`!==`), the effect will re-run.

**Note:** This hook works well when dependencies are primitives or memoized objects/functions.

[↑ Back to Top](#)

## 288. When and how often does React invoke the setup and cleanup functions inside a `useEffect` hook?

### i. Setup Function Execution ( `useEffect` )

The setup function (or the main function) you pass to `useEffect` runs at specific points:

- After the component is mounted** (if the dependency array is empty `[]`)
- After every render** (if no dependency array is provided)
- After a dependency value changes** (if the dependency array contains variables)

### ii. Cleanup Function Execution (Returned function from `useEffect` )

The cleanup function is called **before the effect is re-executed** and when the component **unmounts**.

[↑ Back to Top](#)

## 289. What happens if you return a Promise from `useEffect`??

You should NOT return a Promise from `useEffect`. React expects the function passed to `useEffect` to return either nothing (undefined) or a cleanup function (synchronous function). i.e, It does not expect or handle a returned Promise. If you still return a Promise, React will ignore it silently, and it may lead to bugs or warnings in strict mode.

**Incorrect:**

```
useEffect(async () => {
 await fetchData(); // ❌ useEffect shouldn't be async
}, []);
```

**Correct:**

```
useEffect(() => {
 const fetchData = async () => {
 const res = await fetch('/api');
 const data = await res.json();
 setData(data);
 };
});
```

```
};

fetchData();
}, []);
```

[↑ Back to Top](#)

## 289. Can you have multiple useEffect hooks in a single component?

Yes, multiple useEffect hooks are allowed and recommended when you want to separate concerns.

```
useEffect(() => {
 // Handles API fetch
}, []);

useEffect(() => {
 // Handles event listeners
}, []);
```



Each effect runs independently and helps make code modular and easier to debug.

[↑ Back to Top](#)

## 290. How to prevent infinite loops with useEffect?

Infinite loops happen when the effect updates state that's listed in its own dependency array, which causes the effect to re-run, updating state again and so on.

Infinite loop scenario:

```
useEffect(() => {
 setCount(count + 1);
}, [count]); // Triggers again every time count updates
```



You need to ensure that setState calls do not depend on values that cause the effect to rerun, or isolate them with a guard.

```
useEffect(() => {
 if (count < 5) {
 setCount(count + 1);
 }
}, [count]);
```

[↑ Back to Top](#)

## 291. What are the usecases of useLayoutEffect?

You need to use `useLayoutEffect` when your effect **must run before the browser paints**, such as:

- Reading layout measurements (e.g., element size, scroll position)
- Synchronously applying DOM styles to prevent visual flicker
- Animating layout or transitions
- Integrating with third-party libraries that require DOM manipulation

If there's no visual or layout dependency, prefer `useEffect` — it's more performance-friendly.

```
useLayoutEffect(() => {
 const width = divRef.current.offsetWidth;
 if (width < 400) {
 divRef.current.style.background = 'blue'; // prevents flicker
 }
}, []);
```

[↑ Back to Top](#)

## 292. How does useLayoutEffect work during server-side rendering (SSR)?

The `useLayoutEffect` hook does **not run on the server**, because there is no DOM. React issues a warning in server environments like Next.js if `useLayoutEffect` is used directly.

This can be mitigated using a conditional polyfill:

```
const useIsomorphicLayoutEffect =
 typeof window !== 'undefined' ? useLayoutEffect : useEffect;
```

i.e, Use `useIsomorphicLayoutEffect` in components that render both on client and server.

[↑ Back to Top](#)

### 293. What happens if you use `useLayoutEffect` for non-layout logic?

Using `useLayoutEffect` for logic **unrelated to layout or visual DOM changes** (such as logging, data fetching, or analytics) is **not recommended**. It can lead to **performance issues** or even unexpected behavior.

**Example: Anti-pattern**

```
useLayoutEffect(() => {
 console.log("Tracking analytics");
 fetch('/log-page-view');
}, []);
```

The above usage delays the paint of the UI just to send a network request, which could (and should) be done after paint using `useEffect`.

[↑ Back to Top](#)

### 294. How does `useLayoutEffect` cause layout thrashing?

The `useLayoutEffect` can **cause layout thrashing** when you **repeatedly read and write to the DOM** in ways that force the browser to recalculate layout multiple times per frame. This is because `useLayoutEffect` runs *before the browser paints*, these reflows happen *synchronously*, blocking rendering and degrading performance.

**Example:**

```
function ThrashingComponent() {
 const ref = useRef();

 useLayoutEffect(() => {
 const height = ref.current.offsetHeight; //Read
 ref.current.style.height = height + 20 + 'px'; //Write
 const newHeight = ref.current.offsetHeight; //Read again - forces reflow
 }, []);

 return <div ref={ref}>Hello</div>;
}
```

In the above code, each read/write cycle triggers synchronous reflows, blocking the main thread and delays UI rendering.

This issue can be avoided by batching your DOM reads and writes and prevent unnecessary reads after writes.

[↑ Back to Top](#)

### 295. How Do You Use `useRef` to Access a DOM Element in React? Give an example.

The `useRef` hook is commonly used in React to directly reference and interact with DOM elements — like focusing an input, scrolling to a section, or controlling media elements.

When you assign a ref to a DOM element using `useRef`, React gives you access to the underlying DOM node via the `.current` property of the ref object.

**Example: Focus an input**

```
import React, { useRef } from 'react';

function FocusInput() {
 const inputRef = useRef(null); // create the ref

 const handleFocus = () => {
 inputRef.current.focus(); // access DOM element and focus it
 };

 return (
```



```

 <div>
 <input type="text" ref={inputRef} />
 <button onClick={handleFocus}>Focus the input</button>
 </div>
);
}

```

**Note:** The DOM reference is only available **after the component has mounted** — typically accessed in `useEffect` or event handlers.

[↑ Back to Top](#)

## 296. Can you use useRef to persist values across renders??

Yes, you can use `useRef` to persist values across renders in React. Unlike `useState`, changing `.current` does not cause re-renders, but the value is preserved across renders.

**Example:**

```

function Timer() {
 const renderCount = useRef(0);
 useEffect(() => {
 renderCount.current++;
 console.log("Render count:", renderCount.current);
 });

 return <div>Check console for render count.</div>;
}

```

[↑ Back to Top](#)

## 297. Can useRef be used to store previous values?

Yes, `useRef` is a common pattern when you want to compare current and previous props or state without causing re-renders.

**Example: Storing previous state value**

```

import { useEffect, useRef, useState } from 'react';

function PreviousValueExample() {
 const [count, setCount] = useState(0);
 const prevCountRef = useRef();

 useEffect(() => {
 prevCountRef.current = count;
 }, [count]);

 const prevCount = prevCountRef.current;

 return (
 <div>
 <p>Current: {count}</p>
 <p>Previous: {prevCount}</p>
 <button onClick={() => setCount(c => c + 1)}>Increment</button>
 </div>
);
}

```

[↑ Back to Top](#)

## 298. Is it possible to access a ref in the render method?

Yes, you can access a ref in the render method, but what you get from it depends on how you're using the ref and when in the component lifecycle you're rendering.

For example, when using ref to access a DOM node (e.g., `divRef.current`), it's not immediately available on the first render.

```

const divRef = useRef(null);

console.log(divRef.current); // ❌ null on initial render
return <div ref={divRef}>Hello</div>;

```

[↑ Back to Top](#)

### 299. What are the common usecases of useRef hook?

Some of the common cases are:

- Automatically focus an input when a component mounts.
- Scroll to a specific element.
- Measure element dimensions ( `offsetWidth` , `clientHeight` ).
- Control video/audio playback.
- Integrate with non-React libraries (like D3 or jQuery).

[↑ Back to Top](#)

### 300. What is useImperativeHandle Hook? Give an example.

`useImperativeHandle` is a React Hook that allows a **child component** to expose **custom functions or properties** to its **parent component**, when using `ref` . It is typically used with `forwardRef` and is very useful in cases like **modals, dialogs, custom inputs**, etc., where the parent needs to **control behavior imperatively** (e.g., open, close, reset).

Example: Dialog component

```
import React, {
 useRef,
 useState,
 useImperativeHandle,
 forwardRef,
} from 'react';
import './Dialog.css';

const Dialog = forwardRef((props, ref) => {
 const [isOpen, setIsOpen] = useState(false);
 const [formData, setFormData] = useState('');

 useImperativeHandle(ref, () => ({
 open: () => setIsOpen(true),
 close: () => setIsOpen(false),
 reset: () => setFormData(''),
 }));

 if (!isOpen) return null;

 return (
 <div className="dialog">
 <h2>Dialog</h2>
 <input
 type="text"
 value={formData}
 placeholder="Type something..."
 onChange={(e) => setFormData(e.target.value)}
 />

 <button onClick={() => setIsOpen(false)}>Close</button>
 </div>
);
});

function Parent() {
 const dialogRef = useRef();

 return (
 <div>
 <h1>useImperativeHandle Dialog Example</h1>
 <button onClick={() => dialogRef.current.open()}>Open Dialog</button>
 <button onClick={() => dialogRef.current.reset()}>Reset Dialog</button>
 <button onClick={() => dialogRef.current.close()}>Close Dialog</button>

 <Dialog ref={dialogRef} />
 </div>
);
}

export default Parent;
```

[↑ Back to Top](#)

### 301. When should you use useImperativeHandle?

The `useImperativeHandler` hook will be used in below cases:

- You want to expose **imperative methods** from a child component
  - Custom input controls exposing `focus`, `clear`, or `validate` methods
  - Modal components exposing `open()` and `close()` methods
  - Scroll containers exposing `scrollToTop()` or `scrollToBottom()` methods
- You want to **hide internal implementation** but provide controlled external access.
- You're building **reusable component libraries** (e.g., inputs, modals, form controls).

[↑ Back to Top](#)

### 302. Is that possible to use `useImperativeHandle` without `forwardRef`?

No. `useImperativeHandle` only works when the component is wrapped in `forwardRef`. It's the combination that allows parent components to use a `ref` on a function component.

[↑ Back to Top](#)

### 303. How is `useMemo` different from `useCallback`?

The following table compares both `useMemo` and `useCallback`:

Feature	<code>useMemo</code>	<code>useCallback</code>
Purpose	Memoizes the <b>result of a computation</b>	Memoizes a <b>function reference</b>
Returns	A <b>value</b> (e.g., result of a function)	A <b>function</b>
Usage	<code>useMemo(() =&gt; computeValue(), [deps])</code>	<code>useCallback(() =&gt; doSomething(), [deps])</code>
Primary Use Case	Avoid expensive recalculations	Prevent unnecessary re-creations of functions
Common Scenario	Filtering, sorting, calculating derived data	Passing callbacks to child components
When It's Useful	When the value is expensive to compute	When referential equality matters (e.g., props)
Recomputed When	Dependencies change	Dependencies change
Returned Value Type	Any (number, object, array, etc.)	Always a function
Overhead	Slight (evaluates a function and caches result)	Slight (caches a function reference)

[↑ Back to Top](#)

### 304. Does `useMemo` prevent re-rendering of child components?

The `useMemo` hook **does not directly prevent re-rendering of child components**. Its main purpose is to memoize the result of an expensive computation so that it doesn't get recalculated unless its dependencies change. While this can improve performance, it doesn't inherently control whether a child component re-renders.

However, `useMemo` **can help prevent re-renders** when the memoized value is passed as a prop to a child component that is wrapped in `React.memo`. In that case, if the memoized value doesn't change between renders (i.e., it has the same reference), `React.memo` can skip re-rendering the child. So, while `useMemo` doesn't stop renders on its own, it **works in combination** with tools like `React.memo` to optimize rendering behavior.

[↑ Back to Top](#)

### 305. What is `useCallback` and why is it used?

The `useCallback` is a React Hook used to memoize **function definitions** between renders. It returns the same function reference unless its dependencies change. This is especially useful when passing callbacks to optimized child components (e.g. those wrapped in `React.memo`) to prevent unnecessary re-renders.

Example:

```
const handleClick = useCallback(() => {
 console.log('Button clicked');
}, []);
```



Without `useCallback`, a new function is created on every render, potentially causing child components to re-render unnecessarily.

[↑ Back to Top](#)

### 306. What are Custom React Hooks, and How Can You Develop One?

**Custom Hooks** in React are JavaScript functions that allow you to **extract and reuse component logic** using React's built-in Hooks like `useState`, `useEffect`, etc.

They start with the word **"use"** and let you encapsulate logic that multiple components might share—such as fetching data, handling forms, or managing timers—without repeating code.

Let's explain the custom hook usage with `useFetchData` example. The `useFetchData` custom Hook is a reusable function in React that simplifies the process of fetching data from an API. It encapsulates common logic such as initiating the fetch request, managing loading and error states, and storing the fetched data. By using built-in Hooks like `useState` and `useEffect`, `useFetchData` provides a clean interface that returns the `data`, `loading`, and `error` values, which can be directly used in components.

```
import { useState, useEffect } from 'react';

function useFetchData(url) {
 const [data, setData] = useState(null); // Holds the response
 const [loading, setLoading] = useState(true); // Loading state
 const [error, setError] = useState(null); // Error state

 useEffect(() => {
 let isMounted = true; // Prevent setting state on unmounted component
 setLoading(true);

 fetch(url)
 .then((response) => {
 if (!response.ok) throw new Error('Network response was not ok');
 return response.json();
 })
 .then((json) => {
 if (isMounted) {
 setData(json);
 setLoading(false);
 }
 })
 .catch((err) => {
 if (isMounted) {
 setError(err.message);
 setLoading(false);
 }
 });

 return () => {
 isMounted = false; // Clean-up function to avoid memory leaks
 };
 }, [url]);

 return { data, loading, error };
}
```

The above custom hook can be used to retrieve users data for `AuthorList`, `ReviewerList` components.

#### Example: AuthorList component

```
function AuthorList() {
 const { data, loading, error } = useFetchData('https://api.example.com/authors');

 if (loading) return <p>Loading authors...</p>;
 if (error) return <p>Error: {error}</p>;

 return (

 {data.map((author) => (
 <li key={author.id}>{author.name}
))}

);
}
```

Some of the benefits of custom hooks are:

- Promotes **code reuse**
- Keeps components **clean and focused**
- Makes complex logic **easier to test and maintain**

[↑ Back to Top](#)

### 307. How does React Fiber works? Explain in detail.

React Fiber is the **core engine** that enables advanced features like **concurrent rendering**, **prioritization**, and **interruptibility** in React. Here's how it works:

#### 1. Fiber Tree Structure

Each component in your app is represented by a **Fiber node** in a tree structure. A Fiber node contains:

- Component type
- Props & state
- Pointers to parent, child, and sibling nodes
- Effect tags to track changes (e.g., update, placement)
- This forms the **Fiber Tree**, a data structure React uses instead of the traditional call stack.

#### 2. Two Phases of Rendering

##### A. Render Phase (work-in-progress)

- React builds a **work-in-progress Fiber tree**.
- It walks through each component (begin phase), calculates what needs to change, and collects side effects (complete phase).
- This phase is **interruptible**—React can pause it and resume later.

##### B. Commit Phase

- React applies changes to the **Real DOM**.
- Runs lifecycle methods (e.g., `componentDidMount`, `useEffect`).
- This phase is **non-interruptible** but fast.

#### 3. Work Units and Scheduling

- React breaks rendering into **units of work** (small tasks).
- These units are scheduled based on **priority** using the **React Scheduler**.
- If time runs out (e.g., user starts typing), React can **pause and yield** control back to the browser.

#### 4. Double Buffering with Two Trees

- React maintains two trees:
- **Current Tree** – what's visible on the screen.
- **Work-In-Progress Tree** – the next version being built in memory.
- Only after the new tree is fully ready, React **commits** it, making it the new current tree.

#### 5. Concurrency and Prioritization

- React can prepare multiple versions of UI at once (e.g., during slow data loading).
- Updates can be **assigned priorities**, so urgent updates (like clicks) are handled faster than background work.

[↑ Back to Top](#)

## Old Q&A

### 1. Why should we not update the state directly?

If you try to update the state directly then it won't re-render the component.

```
//Wrong
this.state.message = "Hello world";
```



Instead use `setState()` method. It schedules an update to a component's state object. When state changes, the component responds by re-rendering.

```
//Correct
```



```
this.setState({ message: "Hello World" });
```

**Note:** You can directly assign to the state object either in *constructor* or using latest javascript's class field declaration syntax.

[↑ Back to Top](#)

## 2. What is the purpose of callback function as an argument of `setState()` ?

The callback function provided as the second argument to `setState` is executed after the state has been updated and the component has re-rendered. Because `setState()` is asynchronous, you cannot reliably perform actions that require the updated state immediately after calling `setState`. The callback ensures your code runs only after the update and re-render are complete.

### Example

```
this.setState({ name: "Sudheer" }, () => {
 console.log("The name has been updated and the component has re-rendered.");
});
```

### When to use the callback?

Use the `setState` callback when you need to perform an action immediately after the DOM has been updated in response to a state change. i.e, The callback is a reliable way to perform actions after a state update and re-render, especially when the timing is critical due to the asynchronous nature of state updates in React. For example, if you need to interact with the updated DOM, trigger analytics, or perform further computations that depend on the new state or rendered output.

### Note

- In modern React (with function components), you can achieve similar effects using the `useEffect` hook to respond to state changes.
- In class components, you can also use lifecycle methods like `componentDidUpdate` for broader post-update logic.
- The `setState` callback is still useful for one-off actions that directly follow a specific state change.

[↑ Back to Top](#)

## 3. How to bind methods or event handlers in JSX callbacks?

There are 3 possible ways to achieve this in class components:

- Binding in Constructor:** In JavaScript classes, the methods are not bound by default. The same rule applies for React event handlers defined as class methods. Normally we bind them in constructor.

```
class User extends Component {
 constructor(props) {
 super(props);
 this.handleClick = this.handleClick.bind(this);
 }
 handleClick() {
 console.log("SingOut triggered");
 }
 render() {
 return <button onClick={this.handleClick}>SingOut</button>;
 }
}
```

- Public class fields syntax:** If you don't like to use bind approach then *public class fields syntax* can be used to correctly bind callbacks. The Create React App enables this syntax by default.

```
handleClick = () => {
 console.log("SingOut triggered", this);
};
```

```
<button onClick={this.handleClick}>SingOut</button>
```

- Arrow functions in callbacks:** It is possible to use *arrow functions* directly in the callbacks.

```
handleClick() {
 console.log('SingOut triggered');
}
render() {
```

```
return <button onClick={() => this.handleClick()}>SignOut</button>;
}
```

**Note:** If the callback is passed as prop to child components, those components might do an extra re-rendering. In those cases, it is preferred to go with `.bind()` or *public class fields syntax* approach considering performance.

[↑ Back to Top](#)

#### 4. How to pass a parameter to an event handler or callback?

You can use an *arrow function* to wrap around an *event handler* and pass parameters:

```
<button onClick={() => this.handleClick(id)} />
```

This is an equivalent to calling `.bind()`:

```
<button onClick={this.handleClick.bind(this, id)} />
```

Apart from these two approaches, you can also pass arguments to a function which is defined as arrow function

```
<button onClick={this.handleClick(id)} />;
handleClick = (id) => () => {
 console.log("Hello, your ticket number is", id);
};
```

[↑ Back to Top](#)

#### 5. What is the use of refs?

The *ref* is used to return a reference to the element. They *should be avoided* in most cases, however, they can be useful when you need a direct access to the DOM element or an instance of a component.

[↑ Back to Top](#)

#### 6. How to create refs?

There are two approaches

- i. This is a recently added approach. *Refs* are created using `React.createRef()` method and attached to React elements via the `ref` attribute. In order to use *refs* throughout the component, just assign the *ref* to the instance property within constructor.

```
class MyComponent extends React.Component {
 constructor(props) {
 super(props);
 this.myRef = React.createRef();
 }
 render() {
 return <div ref={this.myRef} />;
 }
}
```

- ii. You can also use *ref* callbacks approach regardless of React version. For example, the search bar component's input element is accessed as follows,

```
class SearchBar extends Component {
 constructor(props) {
 super(props);
 this.txtSearch = null;
 this.state = { term: "" };
 this.setInputSearchRef = (e) => {
 this.txtSearch = e;
 };
 }
 onChange(event) {
 this.setState({ term: this.txtSearch.value });
 }
 render() {
 return (
 <input
```

```

 value={this.state.term}
 onChange={this.onInputChange.bind(this)}
 ref={this.setInputSearchRef}
 />
);
}
}

```

You can also use *refs* in function components using **closures**. **Note:** You can also use inline ref callbacks even though it is not a recommended approach.

[⬆ Back to Top](#)

## 7. What are forward refs?

*Ref forwarding* is a feature that lets some components take a *ref* they receive, and pass it further down to a child.

```

const ButtonElement = React.forwardRef((props, ref) => (
 <button ref={ref} className="CustomButton">
 {props.children}
 </button>
));

// Create ref to the DOM button:
const ref = React.createRef();
<ButtonElement ref={ref}>{"Forward Ref"}</ButtonElement>;

```

[⬆ Back to Top](#)

## 8. Which is preferred option with in callback refs and findDOMNode()?

It is preferred to use *callback refs* over `findDOMNode()` API. Because `findDOMNode()` prevents certain improvements in React in the future.

The **legacy** approach of using `findDOMNode` :

```

class MyComponent extends Component {
 componentDidMount() {
 findDOMNode(this).scrollIntoView();
 }

 render() {
 return <div />;
 }
}

```

The recommended approach is:

```

class MyComponent extends Component {
 constructor(props) {
 super(props);
 this.node = createRef();
 }
 componentDidMount() {
 this.node.current.scrollIntoView();
 }

 render() {
 return <div ref={this.node} />;
 }
}

```

[⬆ Back to Top](#)

## 9. Why are String Refs legacy?

If you worked with React before, you might be familiar with an older API where the `ref` attribute is a string, like `ref='textInput'`, and the DOM node is accessed as `this.refs.textInput`. We advise against it because *string refs have below issues*, and are considered legacy. String refs were **removed in React v16**.

- i. They *force React to keep track of currently executing component*. This is problematic because it makes react module stateful, and thus causes weird errors when react module is duplicated in the bundle.



- ii. They are *not composable* — if a library puts a ref on the passed child, the user can't put another ref on it. Callback refs are perfectly composable.
- iii. They *don't work with static analysis* like Flow. Flow can't guess the magic that framework does to make the string ref appear on `this.refs`, as well as its type (which could be different). Callback refs are friendlier to static analysis.
- iv. It doesn't work as most people would expect with the "render callback" pattern (e.g.)

```
class MyComponent extends Component {
 renderRow = (index) => {
 // This won't work. Ref will get attached to DataTable rather than MyComponent:
 return <input ref={"input-" + index} />;

 // This would work though! Callback refs are awesome.
 return <input ref={(input) => (this["input-" + index] = input)} />;
 };

 render() {
 return (
 <DataTable data={this.props.data} renderRow={this.renderRow} />
);
 }
}
```

[↑ Back to Top](#)

## 10. What are the different phases of component lifecycle?

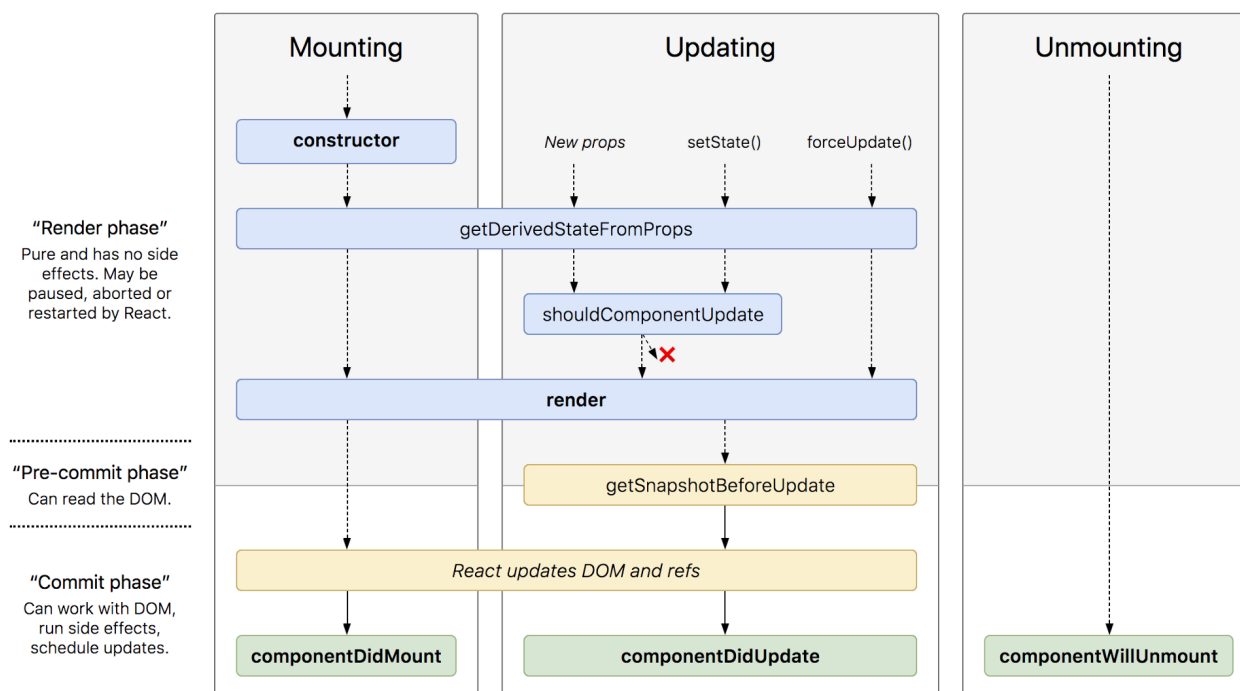
The component lifecycle has three distinct lifecycle phases:

- i. **Mounting:** The component is ready to mount in the browser DOM. This phase covers initialization from `constructor()`, `getDerivedStateFromProps()`, `render()`, and `componentDidMount()` lifecycle methods.
- ii. **Updating:** In this phase, the component gets updated in two ways, sending the new props and updating the state either from `setState()` or `forceUpdate()`. This phase covers `getDerivedStateFromProps()`, `shouldComponentUpdate()`, `render()`, `getSnapshotBeforeUpdate()` and `componentDidUpdate()` lifecycle methods.
- iii. **Unmounting:** In this last phase, the component is not needed and gets unmounted from the browser DOM. This phase includes `componentWillUnmount()` lifecycle method.

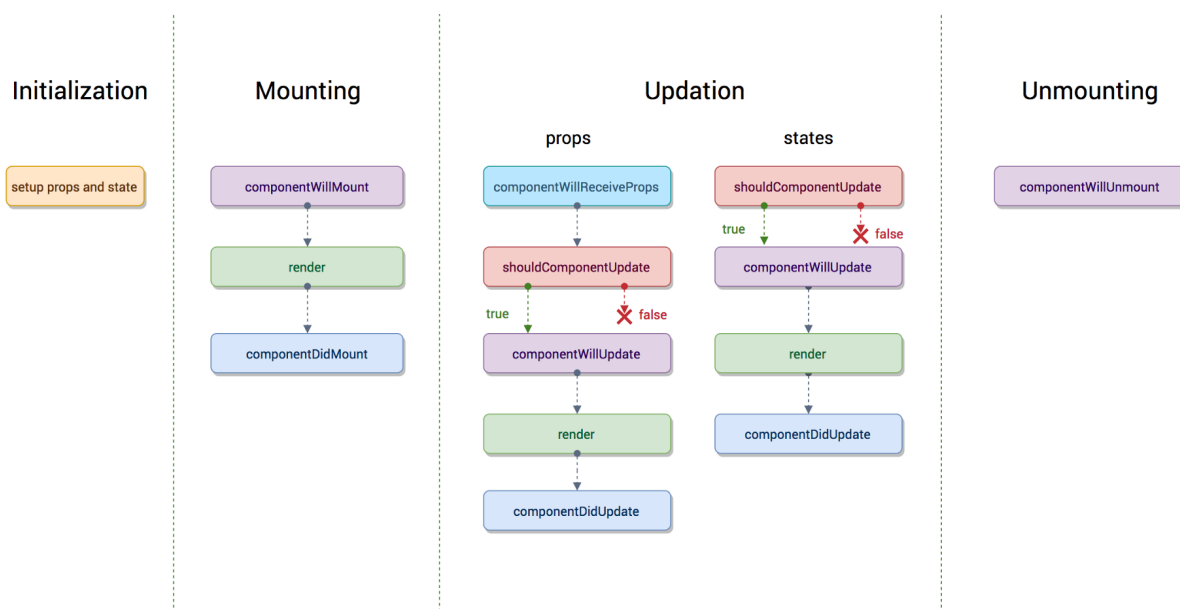
It's worth mentioning that React internally has a concept of phases when applying changes to the DOM. They are separated as follows

- iv. **Render** The component will render without any side effects. This applies to Pure components and in this phase, React can pause, abort, or restart the render.
- v. **Pre-commit** Before the component actually applies the changes to the DOM, there is a moment that allows React to read from the DOM through the `getSnapshotBeforeUpdate()`.
- vi. **Commit** React works with the DOM and executes the final lifecycles respectively `componentDidMount()` for mounting, `componentDidUpdate()` for updating, and `componentWillUnmount()` for unmounting.

React 16.3+ Phases (or an [interactive version](#))



Before React 16.3



[↑ Back to Top](#)

## 11. What are the lifecycle methods of React?

Before React 16.3

- **componentWillMount:** Executed before rendering and is used for App level configuration in your root component.
- **componentDidMount:** Executed after first rendering and here all AJAX requests, DOM or state updates, and set up event listeners should occur.
- **componentWillReceiveProps:** Executed when particular prop updates to trigger state transitions.
- **shouldComponentUpdate:** Determines if the component will be updated or not. By default it returns `true`. If you are sure that the component doesn't need to render after state or props are updated, you can return false value. It is a great place to improve performance as it allows you to prevent a re-render if component receives new prop.
- **componentWillUpdate:** Executed before re-rendering the component when there are props & state changes confirmed by `shouldComponentUpdate()` which returns `true`.
- **componentDidUpdate:** Mostly it is used to update the DOM in response to prop or state changes.
- **componentWillUnmount:** It will be used to cancel any outgoing network requests, or remove all event listeners associated with the component.

React 16.3+

- **getDerivedStateFromProps**: Invoked right before calling `render()` and is invoked on every render. This exists for rare use cases where you need a derived state. Worth reading [if you need derived state](#).
- **componentDidMount**: Executed after first rendering and where all AJAX requests, DOM or state updates, and set up event listeners should occur.
- **shouldComponentUpdate**: Determines if the component will be updated or not. By default, it returns `true`. If you are sure that the component doesn't need to render after the state or props are updated, you can return a false value. It is a great place to improve performance as it allows you to prevent a re-render if component receives a new prop.
- **getSnapshotBeforeUpdate**: Executed right before rendered output is committed to the DOM. Any value returned by this will be passed into `componentDidUpdate()`. This is useful to capture information from the DOM i.e. scroll position.
- **componentDidUpdate**: Mostly it is used to update the DOM in response to prop or state changes. This will not fire if `shouldComponentUpdate()` returns `false`.
- **componentWillUnmount** It will be used to cancel any outgoing network requests, or remove all event listeners associated with the component.

[↑ Back to Top](#)

## 12. How to create props proxy for HOC component?

You can add/edit props passed to the component using *props proxy* pattern like this:

```
function HOC(WrappedComponent) {
 return class Test extends Component {
 render() {
 const newProps = {
 title: "New Header",
 footer: false,
 showFeatureX: false,
 showFeatureY: true,
 };

 return <WrappedComponent {...this.props} {...newProps} />;
 }
 };
}
```

[↑ Back to Top](#)

## 13. What is context?

*Context* provides a way to pass data through the component tree without having to pass props down manually at every level.

For example, authenticated users, locale preferences, UI themes need to be accessed in the application by many components.

```
const { Provider, Consumer } = React.createContext(defaultValue);
```

[↑ Back to Top](#)

## 14. What is the purpose of using super constructor with props argument?

A child class constructor cannot make use of `this` reference until the `super()` method has been called. The same applies to ES6 sub-classes as well. The main reason for passing props parameter to `super()` call is to access `this.props` in your child constructors.

Passing props:

```
class MyComponent extends React.Component {
 constructor(props) {
 super(props);

 console.log(this.props); // prints { name: 'John', age: 42 }
 }
}
```

Not passing props:

```
class MyComponent extends React.Component {
 constructor(props) {
 super();
```

```

 console.log(this.props); // prints undefined

 // but props parameter is still available
 console.log(props); // prints { name: 'John', age: 42 }
 }

 render() {
 // no difference outside constructor
 console.log(this.props); // prints { name: 'John', age: 42 }
 }
}

```

The above code snippets reveals that `this.props` is different only within the constructor. It would be the same outside the constructor.

[↑ Back to Top](#)

## 15. How to set state with a dynamic key name?

If you are using ES6 or the Babel transpiler to transform your JSX code then you can accomplish this with *computed property names*.

```

handleInputChange(event) {
 this.setState({ [event.target.id]: event.target.value })
}

```

[↑ Back to Top](#)

## 16. What would be the common mistake of function being called every time the component renders?

You need to make sure that function is not being called while passing the function as a parameter.

```

render() {
 // Wrong: handleClick is called instead of passed as a reference!
 return <button onClick={this.handleClick()}>'Click Me'</button>
}

```

Instead, pass the function itself without parenthesis:

```

render() {
 // Correct: handleClick is passed as a reference!
 return <button onClick={this.handleClick}>'Click Me'</button>
}

```

[↑ Back to Top](#)

## 17. What are error boundaries in React v16?

*Error boundaries* are components that catch JavaScript errors anywhere in their child component tree, log those errors, and display a fallback UI instead of the component tree that crashed.

A class component becomes an error boundary if it defines a new lifecycle method called `componentDidCatch(error, info)` or `static getDerivedStateFromError()` :

```

class ErrorBoundary extends React.Component {
 constructor(props) {
 super(props);
 this.state = { hasError: false };
 }

 componentDidCatch(error, info) {
 // You can also log the error to an error reporting service
 logErrorToMyService(error, info);
 }

 static getDerivedStateFromError(error) {
 // Update state so the next render will show the fallback UI.
 return { hasError: true };
 }

 render() {
 if (this.state.hasError) {
 // You can render any custom fallback UI
 }
 }
}

```

```

 return <h1>{"Something went wrong."}</h1>;
 }
 return this.props.children;
}
}

```

After that use it as a regular component:

```

<ErrorBoundary>
 <MyWidget />
</ErrorBoundary>

```

[↑ Back to Top](#)

## 18. How are error boundaries handled in React v15?

React v15 provided very basic support for *error boundaries* using `unstable_handleError` method. It has been renamed to `componentDidCatch` in React v16.

[↑ Back to Top](#)

## 19. What is the purpose of render method of `react-dom`?

This method is used to render a React element into the DOM in the supplied container and return a reference to the component. If the React element was previously rendered into container, it will perform an update on it and only mutate the DOM as necessary to reflect the latest changes.

```
ReactDOM.render(element, container, [callback])
```

If the optional callback is provided, it will be executed after the component is rendered or updated.

[↑ Back to Top](#)

## 20. What will happen if you use `setState()` in constructor?

When you use `setState()`, then apart from assigning to the object state React also re-renders the component and all its children. You would get error like this: *Can only update a mounted or mounting component*. So we need to use `this.state` to initialize variables inside constructor.

[↑ Back to Top](#)

## 21. Is it good to use `setState()` in `componentWillMount()` method?

Yes, it is safe to use `setState()` inside `componentWillMount()` method. But at the same it is recommended to avoid async initialization in `componentWillMount()` lifecycle method. `componentWillMount()` is invoked immediately before mounting occurs. It is called before `render()`, therefore setting state in this method will not trigger a re-render. Avoid introducing any side-effects or subscriptions in this method. We need to make sure async calls for component initialization happened in `componentDidMount()` instead of `componentWillMount()`.

```

componentDidMount() {
 axios.get(`api/todos`)
 .then((result) => {
 this.setState({
 messages: [...result.data]
 })
 })
}

```

[↑ Back to Top](#)

## 22. What will happen if you use props in initial state?

If the props on the component are changed without the component being refreshed, the new prop value will never be displayed because the constructor function will never update the current state of the component. The initialization of state from props only runs when the component is first created.

The below component won't display the updated input value:

```
class MyComponent extends React.Component {
 constructor(props) {
 super(props);

 this.state = {
 records: [],
 inputValue: this.props.inputValue,
 };
 }

 render() {
 return <div>{this.state.inputValue}</div>;
 }
}
```

Using props inside render method will update the value:

```
class MyComponent extends React.Component {
 constructor(props) {
 super(props);

 this.state = {
 record: [],
 };
 }

 render() {
 return <div>{this.props.inputValue}</div>;
 }
}
```

[↑ Back to Top](#)

### 23. How you use decorators in React?

You can *decorate* your *class* components, which is the same as passing the component into a function. **Decorators** are flexible and readable way of modifying component functionality.

```
@setTitle("Profile")
class Profile extends React.Component {
 //....
}

/*
 title is a string that will be set as a document title
 WrappedComponent is what our decorator will receive when
 put directly above a component class as seen in the example above
*/
const setTitle = (title) => (WrappedComponent) => {
 return class extends React.Component {
 componentDidMount() {
 document.title = title;
 }

 render() {
 return <WrappedComponent {...this.props} />;
 }
 };
};
```

**Note:** Decorators are a feature that didn't make it into ES7, but are currently a *stage 2 proposal*.

[↑ Back to Top](#)

### 24. What is CRA and its benefits?

The `create-react-app` CLI tool allows you to quickly create & run React applications with no configuration step.

Let's create Todo App using *CRA*:

```
Installation
$ npm install -g create-react-app
```

```
Create new project
$ create-react-app todo-app
$ cd todo-app

Build, test and run
$ npm run build
$ npm run test
$ npm start
```

It includes everything we need to build a React app:

- i. React, JSX, ES6, and Flow syntax support.
- ii. Language extras beyond ES6 like the object spread operator.
- iii. Autoprefixed CSS, so you don't need -webkit- or other prefixes.
- iv. A fast interactive unit test runner with built-in support for coverage reporting.
- v. A live development server that warns about common mistakes.
- vi. A build script to bundle JS, CSS, and images for production, with hashes and sourcemaps.

[⬆️ Back to Top](#)

## 25. What is the lifecycle methods order in mounting?

The lifecycle methods are called in the following order when an instance of a component is being created and inserted into the DOM.

- i. `constructor()`
- ii. `static getDerivedStateFromProps()`
- iii. `render()`
- iv. `componentDidMount()`

[⬆️ Back to Top](#)

## 26. What are the lifecycle methods going to be deprecated in React v16?

The following lifecycle methods going to be unsafe coding practices and will be more problematic with async rendering.

- i. `componentWillMount()`
- ii. `componentWillReceiveProps()`
- iii. `componentWillUpdate()`

Starting with React v16.3 these methods are aliased with `UNSAFE_` prefix, and the unprefix version will be removed in React v17.

[⬆️ Back to Top](#)

## 27. What is the purpose of `getDerivedStateFromProps()` lifecycle method?

The new static `getDerivedStateFromProps()` lifecycle method is invoked after a component is instantiated as well as before it is re-rendered. It can return an object to update state, or `null` to indicate that the new props do not require any state updates.

```
class MyComponent extends React.Component {
 static getDerivedStateFromProps(props, state) {
 // ...
 }
}
```



This lifecycle method along with `componentDidUpdate()` covers all the use cases of `componentWillReceiveProps()`.

[⬆️ Back to Top](#)

## 28. What is the purpose of `getSnapshotBeforeUpdate()` lifecycle method?

The new `getSnapshotBeforeUpdate()` lifecycle method is called right before DOM updates. The return value from this method will be passed as the third parameter to `componentDidUpdate()`.

```
class MyComponent extends React.Component {
 getSnapshotBeforeUpdate(prevProps, prevState) {
 // ...
 }
}
```



This lifecycle method along with `componentDidUpdate()` covers all the use cases of `componentWillUpdate()`.

[↑ Back to Top](#)

## 29. What is the recommended way for naming components?

It is recommended to name the component by reference instead of using `displayName`.

Using `displayName` for naming component:

```
export default React.createClass({
 displayName: "TodoApp",
 // ...
});
```

The recommended approach:

```
export default class TodoApp extends React.Component {
 // ...
}
```

also

```
const TodoApp = () => {
 //...
};
export default TodoApp;
```

[↑ Back to Top](#)

## 30. What is the recommended ordering of methods in component class?

*Recommended ordering of methods from mounting to render stage:*

- i. `static` methods
- ii. `constructor()`
- iii. `getChildContext()`
- iv. `componentWillMount()`
- v. `componentDidMount()`
- vi. `componentWillReceiveProps()`
- vii. `shouldComponentUpdate()`
- viii. `componentWillUpdate()`
- ix. `componentDidUpdate()`
- x. `componentWillUnmount()`
- xi. click handlers or event handlers like `onClickSubmit()` or `onChangeDescription()`
- xii. getter methods for render like `getSelectReason()` or `getFooterContent()`
- xiii. optional render methods like `renderNavigation()` or `renderProfilePicture()`
- xiv. `render()`

[↑ Back to Top](#)

## 31. Why we need to pass a function to `setState()`?

The reason behind for this is that `setState()` is an asynchronous operation. React batches state changes for performance reasons, so the state may not change immediately after `setState()` is called. That means you should not rely on the current state when calling `setState()` since you can't be sure what that state will be. The solution is to pass a function to `setState()`, with the previous state as an argument. By doing this you can avoid issues with the user getting the old state value on access due to the asynchronous nature of `setState()`.

Let's say the initial count value is zero. After three consecutive increment operations, the value is going to be incremented only by one.

```
// assuming this.state.count === 0
this.setState({ count: this.state.count + 1 });
this.setState({ count: this.state.count + 1 });
```



```
this.setState({ count: this.state.count + 1 });
// this.state.count === 1, not 3
```

If we pass a function to `setState()`, the count gets incremented correctly.

```
this.setState((prevState, props) => ({
 count: prevState.count + props.increment,
}));
// this.state.count === 3 as expected
```

(OR)

### Why function is preferred over object for `setState()` ?

React may batch multiple `setState()` calls into a single update for performance. Because `this.props` and `this.state` may be updated asynchronously, you should not rely on their values for calculating the next state.

This counter example will fail to update as expected:

```
// Wrong
this.setState({
 counter: this.state.counter + this.props.increment,
});
```

The preferred approach is to call `setState()` with function rather than object. That function will receive the previous state as the first argument, and the props at the time the update is applied as the second argument.

```
// Correct
this.setState((prevState, props) => ({
 counter: prevState.counter + props.increment,
}));
```

[↑ Back to Top](#)

### 32. Why is `isMounted()` an anti-pattern and what is the proper solution?

The primary use case for `isMounted()` is to avoid calling `setState()` after a component has been unmounted, because it will emit a warning.

```
if (this.isMounted()) {
 this.setState({...})
}
```

Checking `isMounted()` before calling `setState()` does eliminate the warning, but it also defeats the purpose of the warning. Using `isMounted()` is a code smell because the only reason you would check is because you think you might be holding a reference after the component has unmounted.

An optimal solution would be to find places where `setState()` might be called after a component has unmounted, and fix them. Such situations most commonly occur due to callbacks, when a component is waiting for some data and gets unmounted before the data arrives. Ideally, any callbacks should be canceled in `componentWillUnmount()`, prior to unmounting.

[↑ Back to Top](#)

### 33. What is the difference between constructor and `getInitialState`?

You should initialize state in the constructor when using ES6 classes, and `getInitialState()` method when using `React.createClass()`.

Using ES6 classes:

```
class MyComponent extends React.Component {
 constructor(props) {
 super(props);
 this.state = {
 /* initial state */
 };
 }
}
```

Using `React.createClass()` :

```
const MyComponent = React.createClass({
 getInitialState() {
 return {
 /* initial state */
 };
 },
});
```

**Note:** `React.createClass()` is deprecated and removed in React v16. Use plain JavaScript classes instead.

[↑ Back to Top](#)

### 34. Can you force a component to re-render without calling `setState()`?

By default, when your component's state or props change, your component will re-render. If your `render()` method depends on some other data, you can tell React that the component needs re-rendering by calling `forceUpdate()`.

```
component.forceUpdate(callback);
```

It is recommended to avoid all uses of `forceUpdate()` and only read from `this.props` and `this.state` in `render()`.

[↑ Back to Top](#)

### 35. What is the difference between `super()` and `super(props)` in React using ES6 classes?

When you want to access `this.props` in `constructor()` then you should pass props to `super()` method.

Using `super(props)` :

```
class MyComponent extends React.Component {
 constructor(props) {
 super(props);
 console.log(this.props); // { name: 'John', ... }
 }
}
```

Using `super()` :

```
class MyComponent extends React.Component {
 constructor(props) {
 super();
 console.log(this.props); // undefined
 }
}
```

Outside `constructor()` both will display same value for `this.props`.

[↑ Back to Top](#)

### 36. What is the difference between `setState()` and `replaceState()` methods?

When you use `setState()` the current and previous states are merged. `replaceState()` throws out the current state, and replaces it with only what you provide. Usually `setState()` is used unless you really need to remove all previous keys for some reason. You can also set state to `false` / `null` in `setState()` instead of using `replaceState()`.

[↑ Back to Top](#)

### 37. How to listen to state changes?

The `componentDidUpdate` lifecycle method will be called when state changes. You can compare provided state and props values with current state and props to determine if something meaningful changed.

```
componentDidUpdate(object prevProps, object prevState)
```

**Note:** The previous releases of ReactJS also uses `componentWillUpdate(object nextProps, object nextState)` for state changes. It has been deprecated in latest releases.

[↑ Back to Top](#)

### 38. What is the recommended approach of removing an array element in React state?

The better approach is to use `Array.prototype.filter()` method.

For example, let's create a `removeItem()` method for updating the state.

```
removeItem(index) {
 this.setState({
 data: this.state.data.filter((item, i) => i !== index)
 })
}
```

[↑ Back to Top](#)

### 39. Is it possible to use React without rendering HTML?

It is possible. Below are the possible options:

```
render() {
 return false
}
```

```
render() {
 return true
}
```

```
render() {
 return null
}
```

React version  $\geq 16.0.0$ :

```
render() {
 return []
}
```

```
render() {
 return ""
}
```

React version  $\geq 16.2.0$ :

```
render() {
 return <React.Fragment></React.Fragment>
}
```

```
render() {
 return <></>
}
```

React version  $\geq 18.0.0$ :

```
render() {
 return undefined
}
```

[↑ Back to Top](#)

### 40. What are the possible ways of updating objects in state?

i. Calling `setState()` with an object to merge with state:

- Using `Object.assign()` to create a copy of the object:

```
const user = Object.assign({}, this.state.user, { age: 42 });
this.setState({ user });
```

- Using *spread operator*:

```
const user = { ...this.state.user, age: 42 };
this.setState({ user });
```

- ii. Calling `setState()` with a function:

```
this.setState((prevState) => ({
 user: {
 ...prevState.user,
 age: 42,
 },
}));
```

[↑ Back to Top](#)

#### 41. What are the approaches to include polyfills in your `create-react-app` ?

There are approaches to include polyfills in create-react-app,

- i. Manual import from `core-js` :

Create a file called (something like) `polyfills.js` and import it into root `index.js` file. Run `npm install core-js` or `yarn add core-js` and import your specific required features.

```
import "core-js/fn/array/find";
import "core-js/fn/array/includes";
import "core-js/fn/number/is-nan";
```

- ii. Using Polyfill service:

Use the polyfill.io CDN to retrieve custom, browser-specific polyfills by adding this line to `index.html` :

```
<script src="https://cdn.polyfill.io/v2/polyfill.min.js?features=default,Array.prototype.includes"></script>
```

In the above script we had to explicitly request the `Array.prototype.includes` feature as it is not included in the default feature set.

[↑ Back to Top](#)

#### 42. How to use https instead of http in create-react-app?

You just need to use `HTTPS=true` configuration. You can edit your `package.json` scripts section:

```
"scripts": {
 "start": "set HTTPS=true && react-scripts start"
}
```

or just run `set HTTPS=true && npm start`

[↑ Back to Top](#)

#### 43. How to avoid using relative path imports in create-react-app?

Create a file called `.env` in the project root and write the import path:

```
NODE_PATH=src/app
```

After that restart the development server. Now you should be able to import anything inside `src/app` without relative paths.

[↑ Back to Top](#)

#### 44. How to update a component every second?

You need to use `setInterval()` to trigger the change, but you also need to clear the timer when the component unmounts to prevent errors and memory leaks.

```
componentDidMount() {
 this.interval = setInterval(() => this.setState({ time: Date.now() }), 1000)
}

componentWillUnmount() {
 clearInterval(this.interval)
}
```

[↑ Back to Top](#)

#### 45. Why is a component constructor called only once?

React's *reconciliation* algorithm assumes that without any information to the contrary, if a custom component appears in the same place on subsequent renders, it's the same component as before, so reuses the previous instance rather than creating a new one.

[↑ Back to Top](#)

#### 46. How to define constants in React?

You can use ES7 `static` field to define constant.

```
class MyComponent extends React.Component {
 static DEFAULT_PAGINATION = 10;
}
```

[↑ Back to Top](#)

#### 47. How to programmatically trigger click event in React?

You could use the `ref` prop to acquire a reference to the underlying `HTMLInputElement` object through a callback, store the reference as a class property, then use that reference to later trigger a click from your event handlers using the `HTMLInputElement.click` method.

This can be done in two steps:

i. Create ref in render method:

```
<input ref={(input) => (this.inputElement = input)} />
```

ii. Apply click event in your event handler:

```
this.inputElement.click();
```

[↑ Back to Top](#)

#### 48. How to make AJAX call and in which component lifecycle methods should I make an AJAX call?

You can use AJAX libraries such as Axios, jQuery AJAX, and the browser built-in `fetch`. You should fetch data in the `componentDidMount()` lifecycle method. This is so you can use `setState()` to update your component when the data is retrieved.

For example, the employees list fetched from API and set local state:

```
class MyComponent extends React.Component {
 constructor(props) {
 super(props);
 this.state = {
 employees: [],
 error: null,
 };
 }

 componentDidMount() {
 fetch("https://api.example.com/items")
 .then((res) => res.json())
 .then(

```

```

 (result) => {
 this.setState({
 employees: result.employees,
 });
 },
 (error) => {
 this.setState({ error });
 }
);
}

render() {
 const { error, employees } = this.state;
 if (error) {
 return <div>Error: {error.message}</div>;
 } else {
 return (

 {employees.map((employee) => (
 <li key={employee.name}>
 {employee.name}-{employee.experience}

))}

);
 }
}
}

```

[↑ Back to Top](#)

#### 49. What are render props?

**Render Props** is a simple technique for sharing code between components using a prop whose value is a function. The below component uses render prop which returns a React element.

```
<DataProvider render={({data} => <h1>{`Hello ${data.target}`}</h1> } />
```



Libraries such as React Router and DownShift are using this pattern.

[↑ Back to Top](#)

#### 50. How to dispatch an action on load?

You can dispatch an action in `componentDidMount()` method and in `render()` method you can verify the data.

```

class App extends Component {
 componentDidMount() {
 this.props.fetchData();
 }

 render() {
 return this.props.isLoading ? (
 <div>{"Loaded"}</div>
) : (
 <div>{"Not Loaded"}</div>
);
 }
}

const mapStateToProps = (state) => ({
 isLoading: state.isLoading,
});

const mapDispatchToProps = { fetchData };

export default connect(mapStateToProps, mapDispatchToProps)(App);

```



[↑ Back to Top](#)

#### 51. How to use `connect()` from React Redux?

You need to follow two steps to use your store in your container:

- i. Use `mapStateToProps()` : It maps the state variables from your store to the props that you specify.
- ii. **Connect the above props to your container:** The object returned by the `mapStateToProps` function is connected to the container. You can import `connect()` from `react-redux`.

```
import React from "react";
import { connect } from "react-redux";

class App extends React.Component {
 render() {
 return <div>{this.props.containerData}</div>;
 }
}

function mapStateToProps(state) {
 return { containerData: state.data };
}

export default connect(mapStateToProps)(App);
```

[⬆ Back to Top](#)

## 52. Whats the purpose of `@` symbol in the Redux connect decorator?

The `@` symbol is in fact a JavaScript expression used to signify decorators. *Decorators* make it possible to annotate and modify classes and properties at design time.

Let's take an example setting up Redux without and with a decorator.

### o Without decorator:

```
import React from "react";
import * as actionCreators from "./actionCreators";
import { bindActionCreators } from "redux";
import { connect } from "react-redux";

function mapStateToProps(state) {
 return { todos: state.todos };
}

function mapDispatchToProps(dispatch) {
 return { actions: bindActionCreators(actionCreators, dispatch) };
}

class MyApp extends React.Component {
 // ...define your main app here
}

export default connect(mapStateToProps, mapDispatchToProps)(MyApp);
```

### o With decorator:

```
import React from "react";
import * as actionCreators from "./actionCreators";
import { bindActionCreators } from "redux";
import { connect } from "react-redux";

function mapStateToProps(state) {
 return { todos: state.todos };
}

function mapDispatchToProps(dispatch) {
 return { actions: bindActionCreators(actionCreators, dispatch) };
}

@connect(mapStateToProps, mapDispatchToProps)
export default class MyApp extends React.Component {
 // ...define your main app here
}
```

The above examples are almost similar except the usage of decorator. The decorator syntax isn't built into any JavaScript runtimes yet, and is still experimental and subject to change. You can use babel for the decorators support.

[↑ Back to Top](#)

### 53. How to use TypeScript in create-react-app application?

Starting from react-scripts@3.3.0+ releases onwards, you can now optionally start a new app from a template by appending `--template [template-name]` to the creation command. If you don't select a template, it will create your project with base template. Remember that templates are always named in the format `cra-template-[template-name]`, here you only need to fill the `[template-name]` section.

The typeScript can be used in your project by appending `--template typescript` to the creation command.

```
```bash
npx create-react-app my-app --template typescript
```
```

But if you are using React Scripting between react-scripts@2.1.0 and react-scripts@3.2.x , there is a built-in support for TypeScript. i.e, `create-react-app` now supports TypeScript natively. You can just pass `--typescript` option as below

```
```bash
npx create-react-app my-app --typescript

# or

yarn create react-app my-app --typescript
```
```

Whereas for lower versions of react scripts, just supply `--scripts-version` option as `react-scripts-ts` while you create a new project. `react-scripts-ts` is a set of adjustments to take the standard `create-react-app` project pipeline and bring TypeScript into the mix.

Now the project layout should look like the following:

```
...
my-app/
├ .gitignore
├ images.d.ts
├ node_modules/
├ public/
├ src/
├ └ ...
├ package.json
├ tsconfig.json
├ tsconfig.prod.json
├ tsconfig.test.json
└ tsconfig.json
...
└ tsconfig.json
└ tsconfig.test.json
└ tsconfig.prod.json
├ package.json
├ src/
├ └ ...
├ public/
├ node_modules/
├ images.d.ts
├ .gitignore
└ my-app/
...

```

[↑ Back to Top](#)

### 54. Does the statics object work with ES6 classes in React?

No, `statics` only works with `React.createClass()` :

```
someComponent = React.createClass({
 statics: {
 someMethod: function () {
 // ..
 },
 },
});
```

But you can write statics inside ES6+ classes as below,

```
class Component extends React.Component {
 static propTypes = {
 // ...
 };

 static someMethod() {
 // ...
 }
}
```



or writing them outside class as below,

```
class Component extends React.Component {

}

Component.propTypes = {...}
Component.someMethod = function(){...}
```

[↑ Back to Top](#)

## 55. Why are inline ref callbacks or functions not recommended?

If the ref callback is defined as an inline function, it will get called twice during updates, first with null and then again with the DOM element. This is because a new instance of the function is created with each render, so React needs to clear the old ref and set up the new one.

```
class UserForm extends Component {
 handleSubmit = () => {
 console.log("Input Value is: ", this.input.value);
 };

 render() {
 return (
 <form onSubmit={this.handleSubmit}>
 <input type="text" ref={(input) => (this.input = input)} /> //
 Access DOM input in handle submit
 <button type="submit">Submit</button>
 </form>
);
 }
}
```

But our expectation is for the ref callback to get called once, when the component mounts. One quick fix is to use the ES7 class property syntax to define the function

```
class UserForm extends Component {
 handleSubmit = () => {
 console.log("Input Value is: ", this.input.value);
 };

 setSearchInput = (input) => {
 this.input = input;
 };

 render() {
 return (
 <form onSubmit={this.handleSubmit}>
 <input type="text" ref={this.setSearchInput} /> // Access DOM input
 in handle submit
 <button type="submit">Submit</button>
 </form>
);
 }
}
```

[↑ Back to Top](#)

## 56. What are HOC factory implementations?

There are two main ways of implementing HOCs in React.

- i. Props Proxy (PP) and
- ii. Inheritance Inversion (II).

But they follow different approaches for manipulating the *WrappedComponent*.

### Props Proxy

In this approach, the render method of the HOC returns a React Element of the type of the *WrappedComponent*. We also pass through the props that the HOC receives, hence the name **Props Proxy**.

```
function ppHOC(WrappedComponent) {
 return class PP extends React.Component {
 render() {
 return <WrappedComponent {...this.props} />;
 }
 };
}
```

### Inheritance Inversion

In this approach, the returned HOC class (Enhancer) extends the WrappedComponent. It is called Inheritance Inversion because instead of the WrappedComponent extending some Enhancer class, it is passively extended by the Enhancer. In this way the relationship between them seems **inverse**.

```
function iiHOC(WrappedComponent) {
 return class Enhancer extends WrappedComponent {
 render() {
 return super.render();
 }
 };
}
```

[↑ Back to Top](#)

## 57. How to use class field declarations syntax in React classes?

React Class Components can be made much more concise using the class field declarations. You can initialize the local state without using the constructor and declare class methods by using arrow functions without the extra need to bind them.

Let's take a counter example to demonstrate class field declarations for state without using constructor and methods without binding,

```
class Counter extends Component {
 state = { value: 0 };

 handleIncrement = () => {
 this.setState((prevState) => ({
 value: prevState.value + 1,
 }));
 };

 handleDecrement = () => {
 this.setState((prevState) => ({
 value: prevState.value - 1,
 }));
 };

 render() {
 return (
 <div>
 {this.state.value}

 <button onClick={this.handleIncrement}></button>
 <button onClick={this.handleDecrement}></button>
 </div>
);
 }
}
```

[↑ Back to Top](#)

## 58. Why do you not need error boundaries for event handlers?

Error boundaries do not catch errors inside event handlers.

React doesn't need error boundaries to recover from errors in event handlers. Unlike the render method and lifecycle methods, the event handlers don't happen during rendering. So if they throw, React still knows what to display on the screen.

If you need to catch an error inside an event handler, use the regular JavaScript try / catch statement:

```
class MyComponent extends React.Component {
 constructor(props) {
 super(props);
 }
```

```

 this.state = { error: null };
 this.handleClick = this.handleClick.bind(this);
 }

 handleClick() {
 try {
 // Do something that could throw
 } catch (error) {
 this.setState({ error });
 }
 }

 render() {
 if (this.state.error) {
 return <h1>Caught an error.</h1>;
 }
 return <button onClick={this.handleClick}>Click Me</button>;
 }
}

```

Note that the above example is demonstrating regular JavaScript behavior and doesn't use error boundaries.

[↑ Back to Top](#)

## 59. What is the difference between try catch block and error boundaries?

Try catch block works with imperative code whereas error boundaries are meant for declarative code to render on the screen.

For example, the try catch block used for below imperative code

```

try {
 showButton();
} catch (error) {
 // ...
}

```

Whereas error boundaries wrap declarative code as below,

```

<ErrorBoundary>
 <MyComponent />
</ErrorBoundary>

```

So if an error occurs in a `componentDidUpdate` method caused by a `setState` somewhere deep in the tree, it will still correctly propagate to the closest error boundary.

[↑ Back to Top](#)

## 60. What is the required method to be defined for a class component?

The `render()` method is the only required method in a class component. i.e, All methods other than render method are optional for a class component.

[↑ Back to Top](#)

## 61. What are the possible return types of render method?

Below are the list of following types used and return from render method,

- i. **React elements:** Elements that instruct React to render a DOM node. It includes html elements such as `<div/>` and user defined elements.
- ii. **Arrays and fragments:** Return multiple elements to render as Arrays and Fragments to wrap multiple elements
- iii. **Portals:** Render children into a different DOM subtree.
- iv. **String and numbers:** Render both Strings and Numbers as text nodes in the DOM
- v. **Booleans or null:** Doesn't render anything but these types are used to conditionally render content.

[↑ Back to Top](#)

## 62. What is the main purpose of constructor?

The constructor is mainly used for two purposes,

i. To initialize local state by assigning object to this.state

ii. For binding event handler methods to the instance For example, the below code covers both the above cases,

```
constructor(props) {
 super(props);
 // Don't call this.setState() here!
 this.state = { counter: 0 };
 this.handleClick = this.handleClick.bind(this);
}
```



[↑ Back to Top](#)

### 63. Is it mandatory to define constructor for React component?

No, it is not mandatory. i.e, If you don't initialize state and you don't bind methods, you don't need to implement a constructor for your React component.

[↑ Back to Top](#)

### 64. Why should not call setState in componentWillUnmount?

You should not call `setState()` in `componentWillUnmount()` because once a component instance is unmounted, it will never be mounted again.

[↑ Back to Top](#)

### 65. What is the purpose of getDerivedStateFromError?

This lifecycle method is invoked after an error has been thrown by a descendant component. It receives the error that was thrown as a parameter and should return a value to update state.

The signature of the lifecycle method is as follows,

```
static getDerivedStateFromError(error)
```



Let us take error boundary use case with the above lifecycle method for demonstration purpose,

```
class ErrorBoundary extends React.Component {
 constructor(props) {
 super(props);
 this.state = { hasError: false };
 }

 static getDerivedStateFromError(error) {
 // Update state so the next render will show the fallback UI.
 return { hasError: true };
 }

 render() {
 if (this.state.hasError) {
 // You can render any custom fallback UI
 return <h1>Something went wrong.</h1>;
 }

 return this.props.children;
 }
}
```



[↑ Back to Top](#)

### 66. What is the methods order when component re-rendered?

An update can be caused by changes to props or state. The below methods are called in the following order when a component is being re-rendered.

- i. static `getDerivedStateFromProps()`
- ii. `shouldComponentUpdate()`
- iii. `render()`
- iv. `getSnapshotBeforeUpdate()`
- v. `componentDidUpdate()`

[↑ Back to Top](#)

## 67. What are the methods invoked during error handling?

Below methods are called when there is an error during rendering, in a lifecycle method, or in the constructor of any child component.

- i. static `getDerivedStateFromError()`
- ii. `componentDidCatch()`

[↑ Back to Top](#)

## 68. What is the purpose of `unmountComponentAtNode` method?

This method is available from `react-dom` package and it removes a mounted React component from the DOM and clean up its event handlers and state. If no component was mounted in the container, calling this function does nothing. Returns `true` if a component was unmounted and `false` if there was no component to unmount.

The method signature would be as follows,

```
ReactDOM.unmountComponentAtNode(container);
```

[↑ Back to Top](#)

## 69. What are the limitations with HOCs?

Higher-order components come with a few caveats apart from its benefits. Below are the few listed in an order,

- i. **Don't use HOCs inside the render method:** It is not recommended to apply a HOC to a component within the render method of a component.

```
render() {
 // A new version of EnhancedComponent is created on every render
 // EnhancedComponent1 !== EnhancedComponent2
 const EnhancedComponent = enhance(MyComponent);
 // That causes the entire subtree to unmount/remount each time!
 return <EnhancedComponent />;
}
```



The above code impacts on performance by remounting a component that causes the state of that component and all of its children to be lost. Instead, apply HOCs outside the component definition so that the resulting component is created only once.

- ii. **Static methods must be copied over:** When you apply a HOC to a component the new component does not have any of the static methods of the original component

```
// Define a static method
WrappedComponent.staticMethod = function () {
 /*...*/
};
// Now apply a HOC
const EnhancedComponent = enhance(WrappedComponent);

// The enhanced component has no static method
typeof EnhancedComponent.staticMethod === "undefined"; // true
```



You can overcome this by copying the methods onto the container before returning it,

```
function enhance(WrappedComponent) {
 class Enhance extends React.Component {
 /*...*/
 }
 // Must know exactly which method(s) to copy :(
 Enhance.staticMethod = WrappedComponent.staticMethod;
 return Enhance;
}
```



- iii. **Refs aren't passed through:** For HOCs you need to pass through all props to the wrapped component but this does not work for refs. This is because `ref` is not really a prop similar to `key`. In this case you need to use the `React.forwardRef` API

[↑ Back to Top](#)

## 70. How to debug forwardRefs in DevTools?

`React.forwardRef` accepts a render function as parameter and DevTools uses this function to determine what to display for the ref forwarding component.

For example, If you don't name the render function or not using `displayName` property then it will appear as "ForwardRef" in the DevTools,

```
const WrappedComponent = React.forwardRef((props, ref) => {
 return <LogProps {...props} forwardedRef={ref} />;
});
```

But If you name the render function then it will appear as "ForwardRef(myFunction)"

```
const WrappedComponent = React.forwardRef(function myFunction(props, ref) {
 return <LogProps {...props} forwardedRef={ref} />;
});
```

As an alternative, You can also set `displayName` property for `forwardRef` function,

```
function logProps(Component) {
 class LogProps extends React.Component {
 // ...
 }

 function forwardRef(props, ref) {
 return <LogProps {...props} forwardedRef={ref} />;
 }

 // Give this component a more helpful display name in DevTools.
 // e.g. "ForwardRef(logProps(MyComponent))"
 const name = Component.displayName || Component.name;
 forwardRef.displayName = `logProps(${name})`;

 return React.forwardRef(forwardRef);
}
```

[↑ Back to Top](#)

## 71. Is it good to use arrow functions in render methods?

Yes, You can use. It is often the easiest way to pass parameters to callback functions. But you need to optimize the performance while using it.

```
class Foo extends Component {
 handleClick() {
 console.log("Click happened");
 }
 render() {
 return <button onClick={() => this.handleClick()}>Click Me</button>;
 }
}
```

**Note:** Using an arrow function in render method creates a new function each time the component renders, which may have performance implications

[↑ Back to Top](#)

## 72. How do you say that state updates are merged?

When you call `setState()` in the component, React merges the object you provide into the current state.

For example, let us take a facebook user with posts and comments details as state variables,

```
constructor(props) {
 super(props);
 this.state = {
 posts: [],
 comments: []
 };
}
```

Now you can update them independently with separate `setState()` calls as below,

```
componentDidMount() {
 fetchPosts().then(response => {
 this.setState({
 posts: response.posts
 });
 });

 fetchComments().then(response => {
 this.setState({
 comments: response.comments
 });
 });
}
```

As mentioned in the above code snippets, `this.setState({comments})` updates only comments variable without modifying or replacing `posts` variable.

[↑ Back to Top](#)

### 73. How do you pass arguments to an event handler?

During iterations or loops, it is common to pass an extra parameter to an event handler. This can be achieved through arrow functions or `bind` method.

Let us take an example of user details updated in a grid,

```
<button onClick={(e) => this.updateUser(userId, e)}>Update User details</button>
<button onClick={this.updateUser.bind(this, userId)}>Update User details</button>
```

In the both approaches, the synthetic argument `e` is passed as a second argument. You need to pass it explicitly for arrow functions and it will be passed automatically for `bind` method.

[↑ Back to Top](#)

### 74. How to prevent component from rendering?

You can prevent component from rendering by returning null based on specific condition. This way it can conditionally render component.

```
function Greeting(props) {
 if (!props.loggedIn) {
 return null;
 }

 return <div className="greeting">welcome, {props.name}</div>;
}
```

```
class User extends React.Component {
 constructor(props) {
 super(props);
 this.state = {loggedIn: false, name: 'John'};
 }

 render() {
 return (
 <div>
 //Prevent component render if it is not loggedIn
 <Greeting loggedIn={this.state.loggedIn} />
 <UserDetails name={this.state.name}>
 </div>
);
 }
}
```

In the above example, the `greeting` component skips its rendering section by applying condition and returning null value.

[↑ Back to Top](#)

### 75. Give an example on How to use context?

`Context` is designed to share data that can be considered **global** for a tree of React components.

For example, in the code below lets manually thread through a "theme" prop in order to style the Button component.

```
// Lets create a context with a default theme value "luna"
const ThemeContext = React.createContext("luna");
// Create App component where it uses provider to pass theme value in the tree
class App extends React.Component {
 render() {
 return (
 <ThemeContext.Provider value="nova">
 <Toolbar />
 </ThemeContext.Provider>
);
 }
}
// A middle component where you don't need to pass theme prop anymore
function Toolbar(props) {
 return (
 <div>
 <ThemedButton />
 </div>
);
}
// Lets read theme value in the button component to use
class ThemedButton extends React.Component {
 static contextType = ThemeContext;
 render() {
 return <Button theme={this.context} />;
 }
}
```

[↑ Back to Top](#)

## 76. How do you use contextType?

ContextType is used to consume the context object. The contextType property can be used in two ways,

- i. **contextType as property of class:** The contextType property on a class can be assigned a Context object created by React.createContext(). After that, you can consume the nearest current value of that Context type using this.context in any of the lifecycle methods and render function.

Lets assign contextType property on MyClass as below,

```
class MyClass extends React.Component {
 componentDidMount() {
 let value = this.context;
 /* perform a side-effect at mount using the value of MyContext */
 }
 componentDidUpdate() {
 let value = this.context;
 /* ... */
 }
 componentWillUnmount() {
 let value = this.context;
 /* ... */
 }
 render() {
 let value = this.context;
 /* render something based on the value of MyContext */
 }
}
MyClass.contextType = MyContext;
```

- ii. **Static field** You can use a static class field to initialize your contextType using public class field syntax.

```
class MyClass extends React.Component {
 static contextType = MyContext;
 render() {
 let value = this.context;
 /* render something based on the value */
 }
}
```

[↑ Back to Top](#)



## 77. What is a consumer?

A Consumer is a React component that subscribes to context changes. It requires a function as a child which receives current context value as argument and returns a react node. The value argument passed to the function will be equal to the value prop of the closest Provider for this context above in the tree.

Lets take a simple example,

```
<MyContext.Consumer>
 {value => /* render something based on the context value */}
</MyContext.Consumer>
```

[↑ Back to Top](#)

## 78. How do you solve performance corner cases while using context?

The context uses reference identity to determine when to re-render, there are some gotchas that could trigger unintentional renders in consumers when a provider's parent re-renders.

For example, the code below will re-render all consumers every time the Provider re-renders because a new object is always created for value.

```
class App extends React.Component {
 render() {
 return (
 <Provider value={{ something: "something" }}>
 <Toolbar />
 </Provider>
);
 }
}
```

This can be solved by lifting up the value to parent state,

```
class App extends React.Component {
 constructor(props) {
 super(props);
 this.state = {
 value: { something: "something" },
 };
 }

 render() {
 return (
 <Provider value={this.state.value}>
 <Toolbar />
 </Provider>
);
 }
}
```

[↑ Back to Top](#)

## 79. What is the purpose of forward ref in HOCs?

Refs will not get passed through because ref is not a prop. It is handled differently by React just like **key**. If you add a ref to a HOC, the ref will refer to the outermost container component, not the wrapped component. In this case, you can use Forward Ref API. For example, we can explicitly forward refs to the inner FancyButton component using the React.forwardRef API.

The below HOC logs all props,

```
function logProps(Component) {
 class LogProps extends React.Component {
 componentDidUpdate(prevProps) {
 console.log("old props:", prevProps);
 console.log("new props:", this.props);
 }

 render() {
 const { forwardedRef, ...rest } = this.props;
```

```
// Assign the custom prop "forwardedRef" as a ref
return <Component ref={forwardedRef} {...rest} />;
}
}

return React.forwardRef((props, ref) => {
 return <LogProps {...props} forwardedRef={ref} />;
});
}
```

Let's use this HOC to log all props that get passed to our "fancy button" component,

```
class FancyButton extends React.Component {
 focus() {
 // ...
 }

 // ...
}
export default logProps(FancyButton);
```

Now let's create a ref and pass it to FancyButton component. In this case, you can set focus to button element.

```
import FancyButton from "../FancyButton";

const ref = React.createRef();
ref.current.focus();
<FancyButton label="Click Me" handleClick={handleClick} ref={ref} />;
```

[↑ Back to Top](#)

## 80. Is ref argument available for all functions or class components?

Regular function or class components don't receive the ref argument, and ref is not available in props either. The second ref argument only exists when you define a component with React.forwardRef call.

[↑ Back to Top](#)

## 81. Why do you need additional care for component libraries while using forward refs?

When you start using forwardRef in a component library, you should treat it as a breaking change and release a new major version of your library. This is because your library likely has a different behavior such as what refs get assigned to, and what types are exported. These changes can break apps and other libraries that depend on the old behavior.

[↑ Back to Top](#)

## 82. How to create react class components without ES6?

If you don't use ES6 then you may need to use the create-react-class module instead. For default props, you need to define getDefaultProps() as a function on the passed object. Whereas for initial state, you have to provide a separate getInitialState method that returns the initial state.

```
var Greeting = createReactClass({
 getDefaultProps: function () {
 return {
 name: "Jhohn",
 };
 },
 getInitialState: function () {
 return { message: this.props.message };
 },
 handleClick: function () {
 console.log(this.state.message);
 },
 render: function () {
 return <h1>Hello, {this.props.name}</h1>;
 },
});
```

**Note:** If you use createReactClass then auto binding is available for all methods. i.e, You don't need to use `.bind(this)` with in constructor for event handlers.

[↑ Back to Top](#)

### 83. Is it possible to use react without JSX?

Yes, JSX is not mandatory for using React. Actually it is convenient when you don't want to set up compilation in your build environment. Each JSX element is just syntactic sugar for calling `React.createElement(component, props, ...children)`.

For example, let us take a greeting example with JSX,

```
class Greeting extends React.Component {
 render() {
 return <div>Hello {this.props.message}</div>;
 }
}

ReactDOM.render(
 <Greeting message="World" />,
 document.getElementById("root")
);
```

You can write the same code without JSX as below,

```
class Greeting extends React.Component {
 render() {
 return React.createElement("div", null, `Hello ${this.props.message}`);
 }
}
```

## Releases

No releases published

## Sponsor this project



sudheerj Sudheer Jonna



<https://buymeacoffee.com/sudheerj>

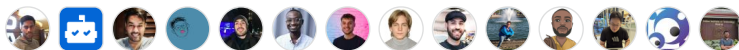
[Learn more about GitHub Sponsors](#)

## Packages

No packages published

## Contributors

113



[+ 99 contributors](#)

## Languages

